Aalto University
School of Science
Master's Programme in Computer, Communication and Information Sciences

Scope Attribution for Static UI Transition Graph Extraction in Single-Page Applications

Duy Khanh Tran

Master's Thesis
Espoo, November 3, 2026

Supervisor: Prof. Petri Vuorimaa
Major: Computer Science

Abstract

A UI transition graph describes the screens of an application and the transitions a user can cause between them. Earlier work extracts such graphs statically from the route declarations of client-side web applications, but none found in the literature reviewed here classifies what the extraction fails to read across a corpus. This thesis makes that measurement on six open-source Angular applications with 14 to ≥909 declared route entries, with four methods that read the sources and one that reads the running application.

The measurement is the main contribution, and its main result is that the deterministic tier is nearly empty. Once the demotion rules the soundness property rests on are applied, 25 constructs yield 150 must edges; 2 of those constructs, one application's toolbar, account for 125 of the edges, and 126 edges rest on an assumption about route order the extractor does not read. The extractor mints a node for every declared screen two separately written static readers find, but a runtime check of the route configuration finds 251 mounted addresses with no node, 242 of them in one application. Every lost construct is assigned to a defect class; after the node-level defects were repaired, attribution of a navigation site to the screens it fires from is the largest class in every application, and almost every class traces to a construct the extractor could read and did not; all but one of those were repaired.

The thesis therefore formulates scope attribution as a dataflow analysis over screen sets, a fixpoint over component nesting, router outlets and a call graph that resolves injected services, reading every provider binding as if it applied everywhere. Its contribution beyond earlier work is modest. The mechanisms earlier work has (the root component on every screen, plus the enclosing routes) reach 99.5 % of a hand-read structural reference of (site, screen) pairs on realworld, partly completed from the graph's output, against 100.0 % for the full analysis; across the corpus the call graph, its fallback join and stand-ins add 36 placed sites, 2.0 % of the total. Attribution recovers 64.3 % of the constructs in its class, almost all as possible rather than certain transitions. Scopes are compared in both directions only on realworld, against a reference that applies the analysis's own relations and guided two repairs; elsewhere the final scopes are unchecked, so the 125 must edges drawn from k8sdash's toolbar rest on an untested scope.

On three applications, the graph detects 45 of 45 injected link deletions, retargets and guard removals, the deletions almost by construction; without attribution fewer changes can be scored at all, but on the 51 mutants both graphs can score, 51 and 50 are detected. Full detection was reached after errors of the harness found in an

earlier run had been corrected. The corpus is small and Angular-only, and the figures describe these six applications only.

Keywords: static analysis; UI transition graph; single-page application; Angular; dataflow analysis; program analysis; software testing

Use of AI tools

Aalto University asks that the use of artificial intelligence tools in a thesis be declared. An AI coding assistant was used in this work in the ways set out below, and I am responsible for the whole of the result.

The extractor under study and the evaluation harness were written with the help of an AI coding assistant. Every number reported in this thesis is produced by running that harness, and the result tables and plots of Chapter 5 are generated from its result files, except the two tables marked as compiled by hand, so that any number can be reproduced from the commits recorded with it. The diagrams in Chapters 1 to 4 were drawn by hand.

The assistant was also used to read the application sources, in three roles. As a reviewer it read the constructs behind an edge and reported where the extractor's reading disagreed with them. In red-team panels it attacked the extractor, the harness and this text, and several of the demotion rules of Section 5.5, the rule by which the extractor decides which guards a navigation runs among them, were written in response to its objections. Every defect it reported in either role was reproduced against the code before anything was changed, and the corrections that followed are reported in Chapter 5 as corrections. Its readings are also part of the evaluation itself: the two readings of the round's must tier in `results-must/review.json` and the reading of the final tier in `results-must/tier-reading.json` were made by the assistant, and Chapter 5 says of each what it can and cannot show, given who made it; the two readings are two runs of one model, so their agreement measures consistency, not correctness.

The assistant was also used during the literature search, to locate candidate sources and to summarise them. No source is cited on the strength of such a summary. Every entry in the bibliography was checked field by field against the publisher page, Crossref metadata, arXiv metadata or DBLP before it was cited, and sources that could not be checked in this way are kept in a separate block in the bibliography file and are not cited anywhere in the text.

Finally, the assistant was used to draft and edit prose, which I then revised. The research questions and the interpretation of the results are my own; the analysis and the evaluation protocol were revised in the course of the work in response to the reviews and panels described above, as Chapter 5 records, and the responsibility for every claim made here is mine.

Contents

Abstract **1**

Use of AI tools **3**

1 Introduction **7**

1.1 Motivation 7
1.2 Research questions and scope 9
1.3 Thesis structure 11

2 Background **13**

2.1 UI transition graphs 13
2.2 Static extraction for Android 14
2.3 Navigation models for web applications 15
2.4 Dynamic state models for single-page applications 15
2.5 Static analysis of JavaScript and TypeScript source 18
2.5.1 Analyses over a source-level syntax tree 18
2.5.2 Abstract interpretation of JavaScript 18
2.5.3 What TypeScript's types contribute 18
2.5.4 Frameworks, not the language, are the obstacle 19
2.6 Formal foundations 20
2.6.1 Modal transition systems 20
2.6.2 Soundiness 20
2.6.3 Dataflow analysis and abstract interpretation 21
2.7 Angular routing, dependency injection and reactive state 21
2.7.1 Routes 22
2.7.2 Components and templates 22
2.7.3 Navigation sites 23
2.7.4 Dependency injection 23
2.7.5 Reactive state 24
2.8 Reactive programming and the limits of static dataflow 25

2.8.1 Propagation order and glitch freedom . . . 25
2.8.2 Reactive code and the tools developers have . . . 26
2.9 Mutation testing . . . 26
2.9.1 The coupling effect . . . 27
2.9.2 Mutation for JavaScript and web applications . . . 27
2.9.3 Reporting a mutation experiment . . . 27
2.10 The gap . . . 28

3 Static navigation analysis and scope attribution 32
3.1 Graph model and soundness . . . 32
3.2 Scope attribution . . . 37
3.2.1 The defect being addressed . . . 38
3.2.2 Scope . . . 38
3.2.3 Input relations . . . 42
3.2.4 Guards as program points . . . 42
3.2.5 Boundary . . . 43
3.2.6 Anchored edges . . . 43
3.3 Implementation . . . 44

4 Evaluation setup and research material 48
4.1 Corpus selection and validity . . . 48
4.1.1 Selection . . . 48
4.1.2 Validity of an open-source corpus . . . 49
4.2 Metrics . . . 50
4.3 Evaluation methods . . . 55
4.3.1 Preconditions . . . 58
4.3.2 Corrections made to the protocol . . . 59
4.3.3 Reproducibility . . . 61

5 Results 64
5.1 The two levels of loss (RQ1) . . . 64
5.2 Which failure is worth an analysis (RQ2) . . . 65
5.3 Scope attribution (RQ3) . . . 69
5.3.1 Checking the scopes against the running interface . . . 73
5.3.2 What the attribution discriminates . . . 75
5.3.3 What each mechanism contributes . . . 76
5.3.4 Agreement with a structural reference . . . 78
5.4 Change sensitivity (RQ4) . . . 81

5.5 What the repairs cost the claim (RQ5) . . . 83
5.5.1 What the losses were . . . 84
5.5.2 What it cost the soundness property . . . 87
5.5.3 What is minted at an address the application does not mount 93
5.6 Dead links, found from the source . . . 96

6 Discussion 98
6.1 Key findings . . . 98
6.2 Limitations and threats to validity . . . 102
6.3 Declared approximations . . . 107
6.4 Transfer to other frameworks . . . 114
6.5 Future work . . . 117

7 Conclusion 120

A Validating the measurement 126
A.1 Nodes the two static readers disagreed about . . . 126
A.2 How the declared-screen set moved . . . 128
A.3 Every remaining site names a reason . . . 130
A.4 The four dead-link corrections . . . 130
A.5 Asking the running framework . . . 134
A.5.1 How the framework was asked . . . 134
A.5.2 What the framework mounts and the extractor does not . . . 135
A.5.3 Nothing is missing that was not declared . . . 136
A.6 What a route table's evidence does not establish . . . 137
A.7 The round's must tier, construct by construct . . . 140

Bibliography 143

Chapter 1

Introduction

A user interface transition graph is a model of an application in which the nodes are the screens a user can see and the edges are the transitions a user action can cause between them. For each edge, the graph records the event that triggers it, the condition under which the transition is available, and the screen it leads to. A graph of this kind makes it possible to answer questions that are difficult to answer from the code alone, for example which screens are reachable from a given screen, or which parts of the user interface are affected when a particular component changes.

Transition graphs have been used in automated testing for a long time. In the Android domain they are used for test generation and for guiding exploration [80, 40, 14], and in web testing they form the basis of crawling-based test derivation [51, 52]. They are also useful for change impact analysis [4, 12], because each transition can be traced back to the code that implements it, and from there to the tests that cover it.

1.1 Motivation

There are two general ways to obtain a transition graph. A dynamic approach executes the application and records the states and transitions it observes. A static approach reads the source code and derives the graph without running anything. Their limitations differ. A dynamically built graph contains only the states the crawler managed to reach, which depends on the available test data, authentication, and the exploration budget, and the graph has to be rebuilt for every new version. A statically built graph covers everything the source declares and can be recomputed on each commit without a running system, but it is limited to what can be decided from the source, and it must over-approximate wherever a target cannot be resolved.

For Android, static extraction is a well-established technique. The window transition graph work derives windows and the transitions between them from bytecode, and resolves the control flow of callbacks to the widget that registered each handler [80, 79, 78]. For the web, navigation models have been extracted statically from server-rendered frameworks, in which pages and links are declared in templates and configuration files [63, 9, 70]. Modern single-page applications, however, have almost always been modelled by executing them. Crawljax builds its state flow graph by firing events against the DOM of a running application [51], and later

tools in the same line improve the exploration strategy without changing where the model comes from [52, 75]. A survey of web testing published in 2025 collects the model-based tools of the past decade [37]. Of the tools it lists, every one whose model construction was examined for this thesis builds that model by executing the application. Static analyses of a modern client-side framework are few. One builds a component dependency graph for React and does not address routing [39]. Three extract navigation, each for a different consumer: one from Angular in order to model-check it, accepting only applications that declare no nested routes, no route parameters and no data-bound navigation [57]; one from Angular sources in order to enumerate the workflows a generated end-to-end suite should cover [65]; and one from the compiled JavaScript bundle of a single-page application in order to check an access-control policy against the screens it can reach [83]. Section 2.4 compares them with the work reported here.

When the dynamic approach was developed, a single-page application had no declarative route table, so its navigation had no explicit representation in the source and was hard to recover from it, and the DOM was the observable the tools used (Section 2.4). Client-side routing frameworks changed this. Angular, React Router and Vue Router all declare routes as data in the source code, connect navigation to those routes through template attributes and service calls, and attach guards to routes as named functions [27]. The artefact needed for static analysis now exists, but how well static extraction works on real applications built with these frameworks, and where it fails, has been measured only against hand-built references for the applications each tool was built on, and not construct by construct.

A statically extracted graph also has consumers outside model-based testing. Tools that drive a graphical user interface with a large language model either explore an application without a model of its navigation structure or build one by exploring it. GPTDroid treats mobile GUI testing as a question-and-answer loop, passing the current screen to the model and replaying the action it returns, and reports higher activity coverage and more bugs than the best baseline on 93 applications [47]. AutoDroid gives the model application-specific knowledge in addition to its commonsense knowledge, held as a UI transition graph that it builds by automated dynamic exploration offline [74]. LLMDroid uses the model to guide exploration towards coverage the test run has not reached [71]. The approach has been applied to web interfaces with a vision-language model and curiosity-driven exploration [73], and a survey of large language models in software testing covers the wider area [72]. Coding agents working on a repository may also benefit from structure of this kind: the SWE-bench issues typically require coordinated edits across functions, classes and files [32], and agent performance depends on the interface through which the repository is navigated [77]. Whether a navigation graph helps such tools is not tested here; these uses motivate the extraction but do not depend on the results of this thesis.

This thesis carries out that measurement for Angular and proposes an analysis for the largest class of failures it finds. What is proposed is a method rather than a product: scope attribution is stated as a dataflow analysis over a lattice of screen sets, and nothing in its formulation depends on a particular extractor.

A method of this kind cannot be assessed without being instantiated, because the questions asked here are empirical: how much of the navigation in real applications it recovers, where it fails, and whether the resulting graph responds to a change in the source. The instrument used is *uigraph*, an open-source static analyser for Angular that I developed before this work began. It plays the part the subject of study plays in an experiment, not the part of a result: the figures reported here describe what the analysis recovers when realised this way, and the limits they expose are limits of the method and of this instantiation, which Chapter 6 separates.

1.2 Research questions and scope

The thesis addresses five research questions. The exact measures are defined in Section 4.2.

RQ1. *How much of the navigation in Angular applications can static analysis recover as deterministic transitions?*

This is measured at two levels, because a transition cannot be recovered without the screens it runs between. The first is node coverage: the fraction of the screens an application declares that the graph holds a node for. The second is the fraction of navigation constructs in the source code that are witnessed by at least one deterministic (*must*) transition, matched at line granularity, so that this fraction is an upper bound. The first is a precondition of the second rather than a numerical bound on it, since a transition needs a node at each end but the two ratios have different denominators, and Chapter 5 reports both, with the warning Section 6.2 carries: refusing to read part of an application raises its reported node coverage, so the figure is only meaningful together with the list of what was not read.

RQ2. *Where does static extraction fail, and why?*

The observed failures are grouped into defect classes. A class is accepted only if at least two lines of evidence confirm it, of which at least one comes from one of the evaluation methods of Section 4.3. Chapter 5 reports, for each class, which of its confirmations are methods and which are observations, because the two are not equally strong. Each construct that is not recovered is then assigned to one class, so the classes can be compared by size within each application.

RQ3. *How does scope attribution affect the coverage and precision of static navigation analysis?*

Scope attribution is the analysis proposed in this thesis. For a navigation site that does not belong to any single screen, it computes the set of screens from which the site can be triggered. One commit of the instrument is run twice on the same inputs, with the attribution pass disabled and enabled, so that nothing but the pass differs between the two runs. Coverage is the recovered share and the conversion ratio, over the same construct denominator. Precision is checked on the deterministic tier, as far as the witness check can read it, and must not decrease; the computed screen sets are also compared with a hand-read structural reference on one application.

RQ4. *Does the extracted graph respond to source changes in the constructs it represents, as change impact analysis requires?*

This is measured with mutation testing. Known changes are inserted into the source, and the graph is checked for the expected difference.

RQ5. *Are the losses a property of the applications or of the tool?*

A defect class names where extraction failed; it does not say whether the navigation was unreadable or merely unread. Each class of RQ2 is therefore taken to its root cause in the source of the application it appeared in, the causes that are the extractor's are repaired where the round could repair them, and the corpus is measured again. A site that still yields no transition must then name the reason, and the script that produces those reasons fails if any site has none, so no site can be left without a recorded reason, though the script checks that a reason is recorded, not that it is correct. One kind of reason is a target the application never declares, which makes the graph a way of finding dead links statically, each candidate then checked against the route configuration the runtime probe recorded; it is reported as a candidate list with counts and not as a defect rate, because a route the readers fail to read is indistinguishable from one the application does not have.

RQ1 and RQ2 are answered by measuring the extractor on a corpus of six open-source Angular applications. The applications are pinned to specific commits and range from 14 declared route entries to $\geq$909. Five evaluation methods are used, chosen so that their blind spots differ, though not so that they are independent. RQ3 is addressed by formulating attribution as a dataflow analysis over a lattice of screen sets, implementing it, and evaluating the result on the same corpus with the same methods. RQ4 is answered on the graph produced after attribution has been applied. RQ5 asks whose the remaining losses are, and it is what decides which instrument the other four questions are answered on: the causes traced to the extractor were repaired except one, ghostfolio's spread of computed routes, which the extractor declares unread, and the figures reported throughout Chapter 5 are those of the repaired instrument. Section 5.5 reports what the repairs cost the soundness property, and what the earlier instrument measured is kept there for comparison rather than as a second set of results. The repairs were found on this corpus and are measured on it; no held-out application tests how far they generalise.

The thesis has four limits of scope. It does not claim completeness. When a navigation target is computed at run time, the full set of transitions cannot be determined statically in general. The thesis therefore states a conditional soundness property over a declared scope, tests it mechanically at each deterministic edge whose destination the checker can read, comparing the construct, its target and its destination but not its guard, and measures recall empirically. The property is not proved, and on this corpus little is left for it to range over. Four corrections each changed the deterministic tier. They concern how a guard is read, how an edge that several constructs cite is judged, and whether a navigation runs under a condition, is refused by a guard or targets the screen already displayed. After the last of them the tier holds 150 edges, in 4 of the six applications (Section 5.5). Nearly every transition the earlier instruments called deterministic turned out to run only once something

had happened (a request had returned, a branch had been taken, a template condition held) or behind a guard the extractor does not evaluate. Taking every guard to let the navigation through would leave 194 must edges instead. Chapter 5 reports each correction rather than only its result. The four corrections took the deterministic tier from the 1024 edges of the instrument this work began with to 150, and they leave RQ1's stricter measure 25 constructs in the corpus, 21 without attribution; the chapter reports both figures, not only the more favourable one. This follows the position of Livshits et al., who argue that a practical static analysis should be described as sound modulo an explicit list of the language features it approximates [48]. It also does not measure the effect of the graph on coding agents or on any other consumer. Such a measurement would require a controlled experiment and is left for future work. A third limit bears on the proposed analysis: the screen sets attribution computes are checked in both directions on one application only. On 5 of the 6 applications, all but ghostfolio, a scope that is too small can be refuted by the rendered interface, though only realworld and bitwarden rendered any judged pair, and Section 5.3.1 finds 4 rendered links the graph does not predict, all in bitwarden and all at sites attribution left unplaced, on an earlier graph and on 30 of 806 screens, while a scope that is too large cannot be, because at the screens that method can observe the graph almost never claims a destination exactly. For one application the screens each site fires from were read off the sources by hand, which gives both directions, and the result qualifies the contribution rather than confirming it: the analysis agrees with 100.0 % of the pairs of that structural reference, whose header sites were partly completed from the graph's output, and asserts none outside it, but the shell rule earlier work already has reaches 94.7 % of them, and with the enclosing routes, which earlier work also follows, 99.5 %. Both figures are those of repairs: the rule the call graph replaced attributed one service's navigation to every screen, and the route reader left one screen's empty-path child off it; this reading caught both, and the repaired analysis was then scored on the same reading (Section 5.3.4). The computed scopes are also wide: narrower than attributing every site to every screen by a factor between 1.7 and 18.4, which at the low end names most of an application (Section 5.3.2). What the application that could be checked exhaustively supports is therefore a small gain in agreement with a structural reference, with no pair outside that reference, on an application whose one counter-example shaped the repair. Whether it holds on the largest application was not tested. Finally, the results apply to Angular only. The graph model does not depend on the framework, and the tool has adapters for other frameworks, but all measurements reported here concern Angular applications.

1.3 Thesis structure

Figure 1.1 shows how the parts of the study fit together.

Chapter 2 presents the background. It introduces the terminology used for transition graphs in different research communities, reviews static extraction for Android, navigation models for web applications and dynamic state models for single-page applications, and describes the Angular mechanisms the analysis has to handle

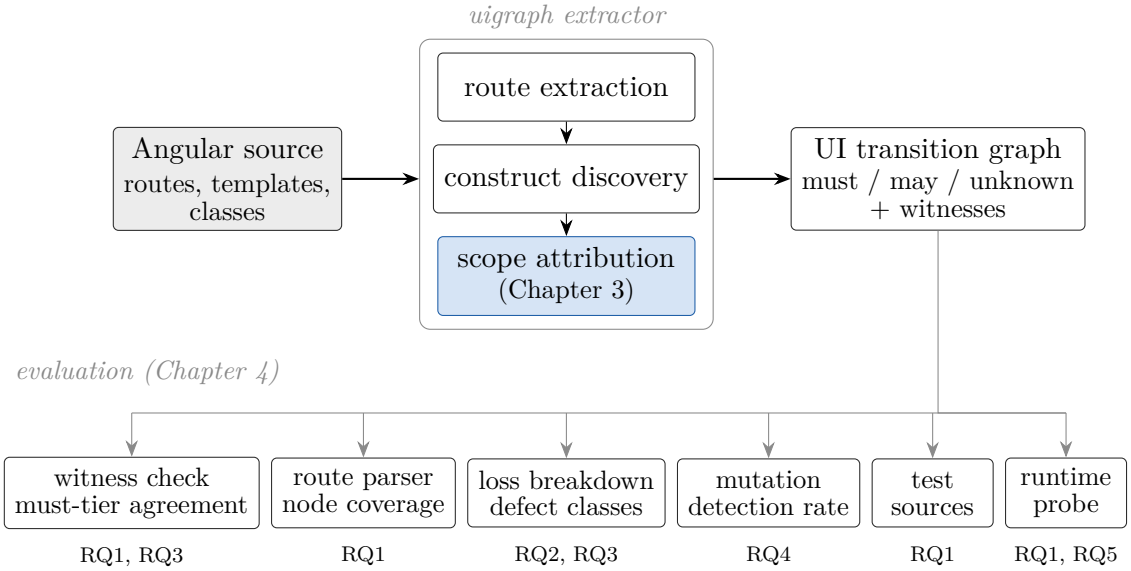


Figure 1.1: Overview of the study. The extractor turns Angular source into a UI transition graph; scope attribution is the analysis this thesis adds. The row below shows the main measurements, each tied to the research questions it answers; the loss breakdown classifies what the other methods find missing rather than being one of the five methods itself.

together with the program analysis ideas it builds on. Chapter 3 presents the graph model and its soundness claim, the scope attribution analysis proposed as a modest extension, and its implementation; the measurement, which is the main contribution, is set up in Chapter 4 and reported in Chapter 5. Chapter 4 describes the evaluation setup: the corpus and why it can be considered valid, the metrics, and the five evaluation methods. Chapter 5 presents the results for each research question: node extraction and the ceiling it sets, then scope attribution and what it recovers, then the defect classes taken to their causes, change sensitivity, what the repairs cost the soundness property, and the dead-link candidates the graph finds statically, each then checked against the route configuration the runtime probe recorded. Appendix A holds the validation of the measuring instruments: the disagreements between the two static readers, how the declared-screen set moved under the corrections, the case-by-case account of asking the running framework, and the record that every remaining site names a reason, recorded rather than verified. It is kept apart from the results because it records the checks made on the instruments, not what the analysis recovers. Chapter 6 discusses the results, the limitations of the study, the constructs the analysis approximates, how the approach transfers to other frameworks, and future work. Chapter 7 concludes the thesis.

Chapter 2

Background

This chapter provides the background needed for the rest of the thesis. Section 2.1 fixes the terminology, since the same kind of graph is called by different names in different communities. Sections 2.2 to 2.4 review the three lines of prior work that are closest to this thesis: static extraction for Android, navigation models for web applications, and dynamic state models for single-page applications. Section 2.5 reviews static analysis carried out over a source-level syntax tree, and what the type annotations of a TypeScript program do and do not contribute to it; this is the tooling style used by the analysis in Chapter 3. Section 2.6 summarises the three formal ideas the analysis builds on. Section 2.7 introduces the Angular mechanisms that the analysis has to deal with and the program analysis problem each of them raises, with dependency injection treated at length because it is the mechanism that hides the most control flow. Section 2.8 reviews reactive programming and the boundary it puts on a static analysis. Section 2.9 reviews mutation testing, which is the method used in Chapter 4 to answer RQ4. Section 2.10 states the gap that the thesis addresses.

2.1 UI transition graphs

A UI transition graph is a directed graph in which the nodes represent user-visible states of an application and the edges represent transitions that a user action can cause between those states. Figure 2.1 shows a small example in the model used in this thesis.

Several research communities have studied this kind of graph, and each uses its own terminology. In Android static analysis it is called a *window transition graph* [80], or, at a finer level of detail, an *activity transition graph* [14] or a *screen transition graph* [40]. In the Ajax crawling literature the term is *state flow graph*, where a state is an abstraction of the DOM [51]. In web engineering the usual terms are *navigation model* and *navigation state machine* [63, 18]. This thesis uses the term *UI transition graph* and treats the other terms as equivalent for its purposes, while noting that they differ in what is counted as a state and in how the graph is constructed.

More important than the naming is how the graph is obtained: *statically*, from source code or binaries without executing the application, or *dynamically*, by running and observing it. It also matters whether the graph represents *guards*, that is,

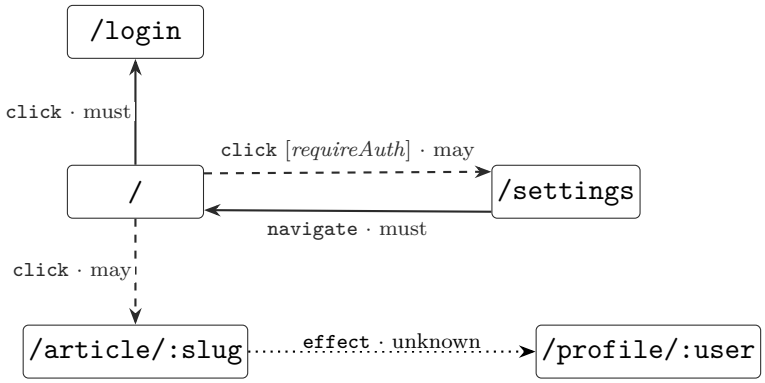


Figure 2.1: A small UI transition graph in the model of this thesis. Each edge carries an event, an optional guard in brackets, and a modality. A *must* edge always fires when its code runs, and a *may* edge is conditional, here on a route guard or on a target that is only partly known. The dotted *unknown* edge shows the model's treatment of a navigation inside a signal effect, whose firing condition cannot be determined statically; the prototype does not implement it (Section 3.2.5). In the measured graphs the unknown modality marks only edges into a sink for destinations that only the run time knows.

whether each edge records the condition under which the transition is available or only that the transition exists. Table 2.1 classifies the work reviewed in the following sections according to both distinctions.

2.2 Static extraction for Android

The largest body of static work on transition graphs concerns Android. Yang et al. introduced the window transition graph, in which the nodes are windows (activities, dialogs and menus) and the edges are transitions caused by callbacks that run on the UI thread; the analysis models the stack of active windows, so that the effect of the back button is captured [80], and a journal version extends the work [78]. A related analysis by the same group constructs a callback control-flow graph whose nodes are pairs of a handler and the widget it belongs to [79]. StoryDroid builds an activity transition graph from `startActivity` calls and uses a simple lexical rule to attribute code inside an inner class to the enclosing activity [14]. GoalExplorer combines a statically derived screen transition graph with dynamic exploration and evaluates the result against manually produced ground truth for 36 applications [40]. UI-CTX identifies the problem of finding the correct code context for a widget's behaviour as its first challenge and solves it with a backward data-flow analysis from the handler to the call that retrieves the widget and its identifier [45].

These approaches have in common the way a navigation is attributed to a screen. In Android, an event handler is registered on a widget object, and at run time that object belongs to exactly one window. Attribution can therefore be treated as an object resolution problem: the widget identifier is resolved, usually by constant propagation of the `getId()` call, and the window is then known [79]. The problem is well defined because the platform's object model gives a widget instance a single owning window.

The limits of this approach become visible in ProMal. Its authors report that a purely static window transition graph contains infeasible transitions, because imprecision in the reference analysis makes several widgets aliases of one another and the over-approximated data flow then attaches a handler to the wrong window. They address this by combining the static analysis with runtime traces and a learned model [46]. Where the owning object cannot be resolved precisely, the Android work therefore turns to dynamic evidence rather than refining the static attribution.

2.3 Navigation models for web applications

Navigation models for the web were first extracted from server-rendered sites, well before single-page applications became common. Ricca and Tonella modelled a web application as a set of pages, frames and forms connected by links and form submissions. Their model includes conditional edges that carry a boolean condition on input values, and it was derived semi-automatically using a crawler together with annotations by an analyst [63]. WebUml recovers a state diagram from HTML, script and server-side pages by combining static source analysis with mutation-driven execution, and derives guards by control-flow analysis of the server code [9]. A patent assigned to Fujitsu describes a fully static method for extracting a navigation model from Struts and JSP sources, including pages that are referenced but do not exist [70]. On the verification side, Haydar et al. model a web application as a system of communicating automata derived from recorded browsing sessions [29], and SiteHopper builds abstracted navigation state machines on the fly for model checking [18]. In addition, design-time notations describe the navigation structure of a site before it is implemented rather than extracting it from code.

This work shows that navigation models, including guards, have been extracted statically from web applications. The models reviewed in this section all concern server-rendered frameworks, where a page corresponds to a file and a link to an anchor element, and none of them deals with a client-side router, a component tree, or dependency injection; Section 2.4 turns to the work that does.

2.4 Dynamic state models for single-page applications

When web applications moved to Ajax and later to client-side frameworks, work on model construction moved to run time. Crawljax executes user events against a running application, compares the DOM before and after each event, and builds a state flow graph whose nodes are equivalence classes of DOM states [51]. Later work in this line changed the exploration strategy but not the source of the model. FeedEx uses feedback from the graph built so far to direct further exploration [52], and FRAGGEN abstracts states as hierarchies of DOM fragments in order to generate better tests [75]. The survey by Kertusha et al. reviews web testing research from 2014 to 2025 [37]. Of the model-based tools it lists, every one whose model construction

was examined for this thesis builds that model by executing the application.

The reason is largely historical. In an Ajax application there was no declarative route table, and one URL could represent many states, because navigation was implemented by handlers that modified the DOM. The DOM was therefore the observable that most directly distinguished one state from another. The source was there to analyse, and static analysis of JavaScript was developed for it [30], but with dynamic dispatch, handlers constructed at run time and the DOM itself to model, recovering navigation from it was hard, and nothing in the source represented the routes explicitly. The dynamic approach was used because it observed directly what the source represented only implicitly.

Client-side routing frameworks changed this situation. The route table is now a declared data structure in the source, a navigation is an attribute or a method call that names a route, and a guard is a named function attached to a route. Four static analyses of such a framework were found during the literature search for this thesis, three of them reading the framework's own sources and one a compiled bundle. Kurant's component dependency graph for React stops at the component rendering graph without addressing routing [39].

The other three do address routing. Oshima and Kishi extract page transitions from an Angular application in order to model-check them with SPIN [57]. Their extraction reads the transition information from the route declarations in `RouterModule`, reads the navigable paths from every component rather than only from the routed ones, and then resolves the parent–child relationships among the root component, the routed component and the custom child components in order to attribute each navigable path to the routed component that contains it. The problem addressed in this thesis is therefore not a new one. That attribution step is scope attribution: a navigable path is attributed to the routed component that contains it, and the root component's own paths to every page, computed statically from the router configuration. Section 5.3.3 measures that combination, as the *rooted* level of the analysis presented here.

What distinguishes the present work is the input the method accepts and what it claims about its output. The authors state that the applications they handle must not use the `children` attribute or route parameters in the routing declarations, and that changes arising from data binding are not treated as page transitions. Those exclusions remove nested route trees and parameterised addresses; how the method treats a navigation whose destination is an expression rather than a literal is not stated in the paper, and is not assumed here. A screen is identified by the name of its routed component rather than by an address, so path composition does not arise, and guards, lazy loading and named outlets are not part of the model. The attribution follows component containment together with the root component's own paths, so a navigation site that reaches a screen through a method call or through a dispatch lies outside the mechanism. The model carries no modality, so a navigation that is always available and one that is available only if a guard admits it are represented identically. The evaluation applies the tool to four constructed applications and to two published ones of 1824 and 6893 lines, with six and twenty-three pages, and reports whether the intended model was produced and whether transition faults,

injected by hand into constructed variants of the first sample application, were detected; the published applications were checked without injection. What the extraction loses on an application it cannot fully read is not measured, which is the question this thesis takes as its subject.

Two recent works read a client-side router configuration statically for other ends. Rima et al. parse Angular sources and templates to build a graph of navigation structure, interaction sites and the conditions (guards among them) that decide whether an interaction is executable, enumerate workflows over that graph, and compile each workflow into a browser test; the graph is a coverage denominator for generated end-to-end tests [65]. That account is taken from the publisher's abstract and from the operational specification in the project's replication package (`softscanner-sw/softscanner_fa`, `docs/paper/approach.md`) and from the anonymised manuscript in the same repository (`docs/paper/main.pdf`), the published chapter itself not having been available to this review, so the published chapter may differ from both. Both documents describe attribution beyond containment. In the manuscript the components active at a route are those activated by the route and its ancestor routes, closed under composition, and the specification adds the bootstrap components to that seed, which covers what the *rooted* and *outlet* levels of Section 5.3.3 place. A handler is followed into helper methods of the same class to depth one and into the first callback of a `subscribe`, and the service a call names is resolved by the type of the constructor parameter. The specification excludes router-outlet modelling and provider resolution beyond module and root providers, and it does not follow a call into a service, so a navigation inside a service member is not placed on the screens of its callers. Zhang et al. parse the compiled JavaScript bundle rather than the sources, take declared routes as vertices and navigation calls as edges, and check an access-control policy against the screens reachable from an authorised one; their graph is declared an over-approximation and is not narrowed, and no navigation site is attributed to the screens it can fire from; the authors note that the graph may contain paths a guard would block, because guards are not represented [83]. Rima et al. compare their output with a ground truth built by hand for the applications they study, which measures what their extraction misses there; neither work classifies the losses by construct or measures them across a corpus. The problem addressed in this thesis is therefore neither new nor settled: the graph has been extracted statically for model checking, for test generation and for policy checking. Earlier work already attributes a site through containment, the shell and the enclosing routes, and follows calls within a class; what none of these does is carry a navigation inside a service member to the screens of its callers across classes, with the provider that binds the injection, give the resulting transitions a modality, or quantify the loss.

The work reported here is located between these analyses and the dynamic tools described above.

2.5 Static analysis of JavaScript and TypeScript source

The analysis in Chapter 3 reads Angular source with the TypeScript compiler API, so it works on a source-level abstract syntax tree rather than on a compiled artefact. This section reviews that tooling style, the abstract interpretation work that put lattice-based analysis of JavaScript on a formal footing, what the type annotations of a TypeScript program contribute, and the evidence that frameworks rather than the language are what limits such analyses in practice.

2.5.1 Analyses over a source-level syntax tree

Analysing the syntax tree of the source, rather than a bytecode or an intermediate representation, is a long-established style. Baxter et al. detect exact and near-miss code clones by comparing abstract syntax trees, using a variant of the common-subexpression detection used in compilers, and apply the technique to a C system of about 400 000 lines [8]. The approach keeps the analysis close to what the developer wrote, which matters when the result has to be reported back at a source location.

SPOON makes this style into a reusable library. It exposes a metamodel of Java source and lets an analysis or a transformation be written in plain Java, without modifying a compiler and without learning a separate formalism [59]. The TypeScript compiler API occupies the same position for TypeScript: it gives a typed syntax tree, a symbol table and a type checker over an ordinary program, which is what allows the Angular adapter to resolve a constant reference in a route table or the declared type of an injected field. The cost of the style is that the analysis sees the program as written, including the parts that a compiler would later rewrite or erase.

2.5.2 Abstract interpretation of JavaScript

Lattice-based analysis of JavaScript itself has a formal treatment. Jensen et al. present a static analysis based on abstract interpretation that infers detailed and sound type information for JavaScript, covering the full language including its object model and the built-in functions, and propose it as a foundation for developer tools [30]. The work shows that the framework of Section 2.6 applies to JavaScript at the level of individual values, which is a considerably harder problem than the one addressed in this thesis. Scope attribution uses the same machinery over a much coarser domain. Its elements are sets of screens, not abstractions of heap objects, so the height of the lattice is the number of screens and does not depend on the structure of the heap.

2.5.3 What TypeScript's types contribute

Angular applications are written in TypeScript, so an analysis can read declared types instead of inferring them. The type system is deliberately unsound. Bierman

et al. give a precise definition of it [10], and Richards et al. describe the annotations as unsound by design [64]. That deliberate unsoundness means a declared type is evidence about the program rather than a guarantee about its execution.

The empirical picture is consistent with that. Gao et al. took bugs that had been fixed in the histories of JavaScript projects, added type annotations to the buggy versions, and checked whether a type checker would have reported them; both Flow and TypeScript detected about 15% of these bugs [25]. Bogner and Merkel mined 604 GitHub projects, 299 in JavaScript and 305 in TypeScript, together more than 16 million lines of code, and found better code quality and understandability for the TypeScript projects, but no lower bug proneness and no shorter bug resolution time; heavier use of the `any` type correlated weakly with worse measurements [11]. The declarations that describe untyped libraries can themselves be wrong: TSCheck compares a declaration file against the library it describes by capturing the runtime object state after initialisation and adding a light static analysis of function code, and found 142 errors in the declarations of 10 libraries [22].

What follows for an analysis of Angular code is a division of labour. A declared type narrows the set of candidate implementations of a call, which is useful and is what the call graph construction in Section 2.7 relies on. It does not establish that the call reaches exactly one of them, so an unresolved or ambiguous type has to widen the result rather than be trusted. In the graph model this appears as the difference between the must tier and the may tier.

2.5.4 Frameworks, not the language, are the obstacle

The practical obstacle to analysing a client-side application is the framework around the code, not the language. Madsen et al. observe that JavaScript applications depend on large frameworks and libraries whose behaviour a static analysis has to know, that hand-written models of them are tedious to produce, incomplete and sometimes wrong, and that the alternative is to infer library behaviour from the way the application uses it; they combine pointer analysis with such a *use analysis* and evaluate it on 25 Windows 8 JavaScript applications [49]. A later analysis by Madsen et al. addresses implicit control flow directly. Their *event-based call graph* for Node.js represents listeners that run when an event fires, and the analyses that build it find event-handling defects such as events that no listener handles and listeners registered too late [50]. An Angular application has the same shape of problem, with the router, template event bindings and observable subscriptions in place of the Node.js event emitter.

Two further static analyses of a framework in the Angular family bear on this work. The one that addresses navigation is Oshima and Kishi's, discussed in Section 2.4. The other, which does not, is Aurebesh, by Ocariza et al. [56]. It defines a consistency model for web applications built on an MVC framework and detects identifier and type inconsistencies between models, views and controllers, reporting a recall of 96.1% and a precision of 100%, and finding 15 real bugs. The work concerns AngularJS 1.x, which is a different framework from the Angular versions this thesis analyses, with a different template language, a different component model and a different injector.

What the two have in common is the problem of relating an identifier in a template to the code that defines it, which Aurebesh solves for a consistency check. Aurebesh targets consistency between templates and controllers rather than navigation, and reports no transition graph.

2.6 Formal foundations

The analysis builds on three ideas from the literature: a transition system in which edges carry a modality, a stated position on what soundness can mean for a practical analysis, and dataflow analysis as the computational framework.

2.6.1 Modal transition systems

The *modal transition system* of Larsen and Thomsen labels each transition as either required (*must*) or merely permitted (*may*) [43]. The graph model in Chapter 3 adopts this distinction and adds a third label, *unknown*, for edges whose modality could not be determined. The distinction has practical consequences: the thesis makes a soundness claim about the must tier only, and the evaluation treats the tiers differently, since the evidence behind a may edge establishes less than the evidence behind a must edge.

Carrying a modality on the edge also changes what an unresolved construct does to the graph. An analysis with a single edge kind has two options when it cannot decide whether a transition occurs: emit the edge and lose precision, or drop it and lose recall. With a modality the same situation produces an edge whose label records how much was established, and a consumer can select the tier it needs. The tiers are reported separately in Chapter 5 for this reason.

2.6.2 Soundiness

Livshits et al. argue that practical static analyses are sound only with respect to a subset of the language, that this is unavoidable in practice, and that the appropriate response is to state explicitly which language features are approximated rather than to claim soundness in general [48]. This thesis follows that recommendation: it states its soundness property only over a declared scope, lists the approximated Angular constructs with their measured frequency in the corpus, and measures recall empirically. The literature reviewed in Sections 2.5 to 2.7 gives the specific reasons why a general claim is not available for this setting. Type annotations in TypeScript are deliberately not guarantees, framework configuration decides which implementation a call reaches, and the firing condition of a reactive computation is not written in the program text.

2.6.3 Dataflow analysis and abstract interpretation

Kildall showed that many program analyses can be stated as the computation of a fixpoint over a lattice of abstract values, where information is combined with a join operation and propagated along the edges of a graph until nothing changes [38]. If the lattice has finite height and the transfer functions are monotone, the computation terminates and its result does not depend on the order in which nodes are visited. Cousot and Cousot gave the general theory behind this approach as abstract interpretation [16], and Nielson et al. give a textbook account [55]. Scope attribution in Chapter 3 is an instance of this framework. The abstract values are sets of screens, the graph is the call graph extended with component nesting, and the join is the union of scopes.

The framework is independent of the language being analysed, and it has been applied to JavaScript as well; Section 2.5 describes that work. What a particular instance needs is a lattice whose height is bounded and a propagation graph. Both are available here: the set of screens is finite, so the lattice of screen sets has a height equal to the number of screens, and the call graph with component nesting supplies the propagation graph. Where the propagation graph itself is incomplete, because a call could not be resolved, the result is incomplete in a way the framework does not detect, which is why the scope has to be declared.

2.7 Angular routing, dependency injection and reactive state

The analysis presented in Chapter 3 operates on Angular applications, and most of its difficulties arise from particular Angular mechanisms. This section introduces those mechanisms and relates each of them to the program analysis problem it creates. The Angular documentation is the reference for the framework details [27]. Listings 2.1 to 2.3 show the constructs discussed below in a small example, which later chapters return to.

Listing 2.1: A route table with the constructs the analysis must handle: a guarded route, a parent route whose children render in its outlet, a lazily loaded route table and a path held in a constant.

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'settings', component: SettingsComponent,
    canActivate: [requireAuth],
    children: [{ path: 'profile', component: ProfileComponent }] },
  { path: 'admin', loadChildren: () => import('./admin/admin.routes') },
  { path: paths.about, component: AboutComponent },   // constant, not a
    literal
];
```

Listing 2.2: The root component's template. The header sits next to the outlet, so its links are present on every screen but belong to none of them.

```
<app-header />                  <!-- shell: rendered on every screen -->
<router-outlet />               <!-- the active route renders here -->

<!-- header.component.html -->
<a routerLink="/settings">Settings</a>
@if (user()) { <a [routerLink]="['/profile', user().name]">Me</a> }
```

Listing 2.3: Navigation from an injected service. The call site belongs to no component, so its screens can only be found through the components that call `logout`.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private router = inject(Router);
  private session = inject(SessionStore);
  logout(): void {
    this.session.clear();
    this.router.navigate(['/login']);
  }
}
```

2.7.1 Routes

An Angular application declares its screens as a tree of route objects. Each route has a `path` pattern and, in addition, either a `component` to render, a `redirectTo` target, or a list of `children`. A route that has children but no component acts as a path prefix and does not itself correspond to a screen. Routes can be loaded lazily using `loadChildren` or `loadComponent`; in that case the child routes or the component are defined in another module or file, which the bundler resolves when the application is built and the router fetches when a navigation first matches the route, or earlier if a preloading strategy is configured. Routes may also have guards, namely `canMatch`, `canActivate`, `canActivateChild` and `canDeactivate`, with `canLoad` still accepted but deprecated in favour of `canMatch`. Guards are functions or injectable classes that return a boolean, a `UrlTree` or `RedirectCommand` representing a redirect, or a Promise or Observable of one; a `canMatch` that returns false makes the router try the next matching route rather than cancel the navigation. Routes may also have resolvers, which fetch data the router awaits before it activates the route; `runGuardsAndResolvers` controls whether they run again when a navigation keeps the route active. Route paths are strings, but they are not necessarily string literals. Applications that keep their route table in one place often write `path: routes.about.path`, and resolving such a path requires following the constant to its definition.

2.7.2 Components and templates

Each screen is rendered by a component, which is a class together with a template. A template can instantiate other components by their selectors, so the components of an application form a nesting graph. A template can also contain a `<router-outlet>` element, which marks the position where the router inserts the component of the

currently active child route. Components that are above the outlet in the nesting graph make up the application shell and are present on every screen, whereas components below a routed component are present only on that screen. Structural directives such as `@if` and `*ngIf` make the nesting conditional. Templates can additionally be passed around as values of type `TemplateRef` and rendered in a different component, and components can be created programmatically at run time.

2.7.3 Navigation sites

In a template, navigation is expressed with the `routerLink` directive, whose value is either a literal path or an array of path segments. In class code, navigation is performed by calling `Router.navigate` or `Router.navigateByUrl`; the target may be given relative to the current route through the `relativeTo` option. Both kinds of navigation site can appear in any component, in any injectable service, and in guards, interceptors and resolvers. The source code does not state anywhere which screens a given site can be triggered from; this information has to be derived.

2.7.4 Dependency injection

Angular creates services and passes them to the components and services that declare them as dependencies. This is dependency injection, a form of inversion of control: a class does not construct or look up its collaborators, but receives them from a container [33, 24]. A dependency is requested by type or by an injection token, and the same token can be bound to different implementations in different injector scopes, using `useClass`, `useExisting` or a factory function.

The pattern is not specific to Angular and is widely used. Yang, Tempero and Melton developed operational definitions of dependency injection and techniques for detecting its use from source code, and applied them to 34 open-source Java applications [76]. Their motivation is the usual argument for the pattern, that it improves extensibility, modifiability, testability and reusability. The configuration that dependency injection introduces is itself an object of static analysis: Laigner et al. catalogue 12 dependency injection anti-patterns together with refactorings, and detect them with a static analyser at a relative recall of 92.19%, reporting that at least 9 of the anti-patterns occur frequently in open-source and industrial projects [41]. Dependency injection configuration is therefore readable statically; it does not state which binding a given call site will see.

For a static analysis this turns a simple method call into a dispatch problem. A call such as `this.nav.goHome()` can only be linked to the method it invokes after the type of `nav` and the binding of that type in the relevant injector have been resolved. This is the call graph construction problem known from object-oriented languages, where a virtual call must be resolved to its possible targets. Class hierarchy analysis resolves such a call using the declared type and its subclasses [17]. Rapid type analysis narrows the result to classes that are actually instantiated [5], and points-to analysis narrows it further by tracking which objects each variable may hold [2]. Grove and Chambers place these algorithms in a common framework that trades

precision against cost [28]. For JavaScript, Feldthaus et al. show that an approximate call graph, which tracks functions only by the property names they are stored under, is precise enough for practical tools [23].

Dependency injection adds a further difficulty, because the binding of a type to an implementation is not written in ordinary code but in framework configuration. The Java literature has measured what this costs a general analysis and what recovers it, and Table 2.2 lists the analyses that model framework behaviour explicitly. Sridharan et al. report that the heavy use of reflection in frameworks makes static taint analysis ineffective, and that adding support for one framework at a time does not scale; their tool instead generates a specification of framework behaviour from the application code and its configuration files, which an unmodified taint engine then consumes, and this found 525 new issues across nine benchmarks [68]. Antoniadis et al. report that major Java analysis frameworks offer almost no support for enterprise applications, capture the concepts those frameworks share, such as components and entry points, in a declarative and largely framework-independent form, and measure about four times higher completeness and about a sixfold speedup against the most precise conventional analysis [3]. Landman et al. put a bound on what can be hoped for: in a literature review and corpus study of Java reflection they find that reflection cannot be ignored in 78% of projects and that resolving it requires unsound assumptions [42].

The closest analogue to resolving Angular's injectors is Jasmine, by Chen et al. [13]. Spring's core mechanisms, dependency injection and aspect-oriented programming, rely on dynamic language features, and the consequence is that a conventional call graph of a Spring application is missing edges. Jasmine extends Soot so that the call graph of a Spring program becomes more complete, and was evaluated on micro-benchmarks and on 18 real Spring programs; it improves on tools built with Soot and with Doop, and in combination with FlowDroid it exposed 12 sensitive-data leak paths. The structural problem is the one that occurs in Angular: an injected field has a declared type, the binding of that type lives in framework configuration, and an analysis that does not read the configuration cannot connect the call to a body. The difference is in where the configuration lives and how far its effect reaches. Angular binds tokens in provider arrays that belong to a module, a route or a component, so the same token can resolve to different implementations depending on where the injection happens. Resolution is relative to an injector rather than global, which makes the provider hierarchy part of the input to the analysis.

The analysis in Chapter 3 follows the advice this literature converges on, which is to model the framework explicitly rather than to treat its configuration as ordinary code. It reads Angular's provider declarations and resolves an injected type in the style of class hierarchy analysis. Where a token can be bound to more than one implementation, all of them are considered.

2.7.5 Reactive state

Recent versions of Angular include signals and effects. A signal is a value that changes over time, and an effect is a computation that runs again whenever a signal

it read changes [26]. This is an instance of reactive programming, which goes back to functional reactive programming [21] and has been surveyed by Bainomugisha et al. [6]. In reactive systems the dependencies between values are recorded at run time, and the runtime propagates updates without explicit calls in the program text. So when a navigation call is placed inside an effect, the condition under which it fires is a data dependence in a reactive graph and not a control-flow condition. A static analysis can find which signals the effect reads; when the effect runs is not determined by the source in general, and the analysis of this thesis does not attempt to bound it. Angular applications also use RxJS observables for the same purpose, in router event subscriptions, HTTP responses and state containers. Section 2.8 reviews what the reactive programming literature establishes about this boundary.

2.8 Reactive programming and the limits of static dataflow

The reactive style gives the runtime, rather than the program text, control over when a computation runs. This section reviews the propagation discipline that reactive runtimes implement, and the evidence about how well reactive code can be reasoned about with tools.

2.8.1 Propagation order and glitch freedom

A reactive program is a graph of values in which an update to one value has to be propagated to the values that depend on it. Cooper and Krishnamurthi embed dynamic dataflow in a call-by-value language, so that an expression evaluates to a node in a dataflow graph [15]. Their account names the central hazard: a *glitch* occurs when a signal is recomputed before all of the signals it depends on have been brought up to date, which makes an intermediate value observable that no consistent state of the program ever had. Their runtime prevents glitches by assigning each signal a height and updating signals in order of increasing height, which is a topological order and therefore requires the dependency graph to be acyclic. Drechsler et al. carry the property into a distributed setting with a change-propagation algorithm that needs no central knowledge of the dependency topology, avoids propagating updates that cannot change a result, and preserves glitch freedom [20]. An Angular signal graph, as described in Section 2.7.5, records its dependencies as they are read, so it is the kind of structure that these propagation algorithms are defined over.

The consequence for a static analysis is that the order in which reactive computations run is a property of the graph that the runtime builds, and the graph is built from the reads that actually happen. The set of signals a given effect may read can be over-approximated from the source. The number of times the effect runs, and the state of the application when it runs, depend on which writes occur and in what order, so they cannot in general be determined exactly from the source; a static analysis could still over-approximate them, and the one presented here does not try.

2.8.2 Reactive code and the tools developers have

The empirical literature on reactive programming reports a split between reading code and reasoning about its behaviour. Salvaneschi et al. ran an experiment with 127 subjects comparing reactive programming against object-oriented code using the Observer pattern, and found comprehension significantly better in the reactive version [67]. The same group reports that the tool support is the weak side. Reactive programs, they argue, need debugging concepts of their own, because a conventional debugger shows a call stack while the question is about a dataflow graph; their Reactive Inspector implements such concepts and outperformed conventional debugging in a controlled experiment [66].

The same picture holds for the Rx family of libraries, which Angular applications use for asynchronous data. Banken et al. report that conventional debuggers have no support for Rx and that developers fall back on inserting console logging; their RxFiddle visualises the structure and the values of an Rx dataflow, and in an experiment with 111 developers it let them finish debugging tasks faster [7]. Alabor and Stolze interviewed 10 professional engineers and observed 4 of them debugging RxJS code, and found that they rely on manually added trace logging rather than on specialised tools; the study is a workshop paper with a small sample, so it indicates rather than establishes the practice [1]. The surface that a tool would have to cover is large: Zimmerle et al. mined RxJava, RxJS and RxSwift usage on GitHub and Stack Overflow and found that 95.2% of the Rx operators are used in practice, with the stream abstraction itself the largest of the 23 Stack Overflow topics they identified, at 36.4% of the posts [84].

An analysis that aims to recover navigation from an Angular application inherits this boundary. A navigation placed in an effect or in a subscription has a firing condition that is a data dependence in a reactive graph. The read set is visible in the source; the firing is not determined by it exactly, and this analysis does not approximate it. Chapter 3 therefore specifies the modality *unknown* for such edges and keeps the read set as the symbolic guard, which is the treatment the soundiness position of Section 2.6 prescribes for a construct that cannot be decided. No pass in this version emits that modality for an effect or a subscription; the unknown edges it does emit are those whose destination it cannot resolve to a screen.

2.9 Mutation testing

RQ4 asks whether the extracted graph responds to source changes in the constructs it represents, as change impact analysis requires. The method used to answer it is mutation testing, applied in an unusual direction: the mutants are inserted into the application source, and the artefact whose reaction is measured is the extracted graph rather than a test suite. This section reviews the technique, the evidence for the assumption it rests on, its use on JavaScript and web applications, and the reporting practice that Chapter 4 follows.

2.9.1 The coupling effect

Mutation testing inserts small syntactic changes into a program and measures how many of them a test suite detects. The original argument for it is the *coupling effect* stated by DeMillo et al., namely that tests which uncover simple errors are often also effective at uncovering much more complex errors [19]. Without that assumption a mutant would say nothing about the faults a developer actually writes.

Jia and Harman give the standard survey of the field, covering approaches, tools and empirical results; their trend analysis suggests that the techniques and the tools are reaching maturity and applicability [31]. Papadakis et al. survey the following decade of work on the fundamental problems of the technique and list the problems that remain open [58]. The evidence that mutants are a usable proxy comes from Just et al., who used 357 real faults in 5 open-source applications totalling 321 000 lines of code and found that mutant detection correlates significantly with real-fault detection, independently of code coverage, while also discussing the limitations of the analysis [34].

2.9.2 Mutation for JavaScript and web applications

Mutation analysis has been adapted to JavaScript and to web applications, where the cost of generating and running mutants is a central concern. Mirshokraie et al. guide mutant generation with a combination of static and dynamic analysis, directing it at code that is error-prone or likely to affect the output, in order to reduce the cost and the number of equivalent mutants, and they propose mutation operators specific to JavaScript and the web [53]. The journal version adds a ranking of functions and reports that more than 93% of the generated mutants are non-equivalent and that more than 75% of the surviving non-equivalent mutants fall in the top 30% of the ranked functions [54]. Praphamontripong and Offutt define operators for faults in the way web components interact with each other, covering client-to-server and server-to-server interaction, implement them in a tool called webMuJava, and report a small case study suggesting that the resulting tests find web application faults; the work appeared at a workshop [60].

On the tooling side, Stryker is a mutation testing framework for JavaScript and TypeScript. No peer-reviewed publication describing it was found, and the project does not cite one; its documentation describes implementations for JavaScript and TypeScript, for C# and for Scala, more than 30 supported mutations, a runner-agnostic design and an Apache-2.0 licence [69]. The absence of a paper is the reason the mutation procedure in Chapter 4 is specified in the thesis rather than delegated to a tool's defaults.

2.9.3 Reporting a mutation experiment

Papadakis et al. also give best-practice advice on the use of mutation in empirical studies of testing [58]. Chapter 4 follows it by stating the operator set it uses, how the mutants were produced and sampled, which mutants were excluded and why, and

the detection rate for each operator separately instead of one aggregate figure. The operators it uses change the navigation structure of the application: renaming a route path, removing a guard, deleting a `routerLink` and retargeting a `Router.navigate` call. The oracle is the difference between the graph extracted before the mutation and the graph extracted after it, compared against the change that was authored, so the expected outcome is known without reference to the tool's own output.

This use of mutation puts the thesis in the change impact analysis tradition rather than in test adequacy measurement. Arnold and Bohner define a framework for comparing impact analysis approaches along three dimensions, namely how an approach is used, how it works internally, and how effective it is [4], and the collection edited by Bohner and Arnold is the standard reference for the area, covering source-code dependency analysis and traceability analysis [12]. Chianti is the classic tool of this kind: it decomposes the difference between two versions of a program into atomic changes and reports, for each affected test, the changes that affect it; on a year of version history from one project it found that an average of 52% of the unit tests were affected, each by only 3.95% of the atomic changes [62]. Lehnert's taxonomy classifies impact analysis approaches along more detailed criteria than the earlier frameworks and is useful for placing a new one, although it appeared at a workshop [44]. A transition graph whose edges carry source witnesses supports the same kind of query at the level of screens, and the mutation experiment measures whether the graph changes when the source it was derived from changes.

2.10 The gap

Static extraction of navigation models from web applications has been directed largely at server-rendered frameworks, in which a page is a file and a link an anchor; client-side routing frameworks have mostly been modelled dynamically instead. The literature search for this thesis found three static extractions of a client-side navigation graph, two from a framework's own sources and one from a compiled bundle, each building its graph for a different consumer. Oshima and Kishi's is the one that fixes the terms of the problem addressed here (Section 2.4). It does so, however, only for applications that declare no nested routes, no route parameters and no data-bound navigation, with attribution computed over component containment and the root component's own paths, with no modality on the transitions, and with no measurement of what the extraction fails to read. The other two put the graph to work without narrowing it: one as the coverage denominator of a generated end-to-end suite [65], one as the reachability relation of an access-control check, declared an over-approximation [83]. Rima et al. already read nested and lazily loaded route trees and attach guards to edges as constraints. What this review found no earlier work combining is attribution that follows calls across classes and into services under their provider bindings, a modality on each transition that separates what is unconditionally available from what a guard or an unresolved scope makes uncertain, and a measurement of the resulting loss, classified by construct, on applications large enough for the difficult constructs to occur. The Android literature is the nearest static relative. It attributes navigation by resolving the owning object, and

when that resolution is imprecise it falls back on runtime traces. A navigation site in a web application frequently has no owning object that could be resolved. In Android the owning object of a handler usually identifies the window, so attribution reduces to object resolution in the common case; fragments, background tasks and aliased widgets are the exceptions, and the Android work handles them with dynamic evidence. That is why an Android technique cannot simply be reused.

The literature reviewed in Sections 2.5 to 2.9 fills in why the problem is open and what is available for addressing it. The analysis machinery is not missing: abstract interpretation of JavaScript is established [30], source-level analysis over a typed syntax tree is a standard tooling style [8, 59], and the Java literature has shown repeatedly that framework behaviour can be modelled explicitly to recover call edges that a general analysis loses [68, 3, 13]. Applying this to a client-side framework in order to recover navigation has been done, within the restrictions set out above [57] and, more recently, for test generation and for access-control checking [65, 83]; what this review found no instance of is an attribution that carries a navigation site inside a service to the screens of its callers across classes, or a measurement of the loss classified by construct across a corpus. The other peer-reviewed static analysis in the Angular family targets AngularJS 1.x and checks template and controller consistency rather than navigation [56], and the type information that TypeScript adds narrows a dispatch without settling it, because the type system is unsound by design [10, 64]. The reactive literature marks where the boundary of the approach lies, since the firing of a reactive computation is decided by a graph the runtime builds and not by the program text [15, 7]. Mutation testing supplies a method for asking whether an extracted model reacts to source changes [31, 58], and for a statically extracted navigation graph of a client-side framework it has been applied only to faults injected by hand into constructed applications [57]; an operator-level mutation study of such a graph was not found in this review.

Table 2.1: Prior work closest to this thesis, classified by target, extraction mode, guard representation, and the evaluation basis each reports, which is a reference, a metric or the tool it was compared with, as the work states it.

Work	Target	Static	Guards	Evaluation basis
Yang et al. [80, 78]	Android windows	yes	no	manual, 6 apps
Yang et al. [79]	Android callbacks	yes	context	manual vs. dynamic
StoryDroid [14]	Android activities	yes	no	hand-built, 10 apps
GoalExplorer [40]	Android screens	hybrid	no	manual, 36 apps
ProMal [46]	Android windows	hybrid	no	40 apps, F1
UI-CTX [45]	Android widgets	yes	branches	2000 labelled widgets
Ricca and Tonella [63]	server-rendered pages	semi	yes	manual
WebUml [9]	server-rendered pages	hybrid	yes	case study
Tkachuk and Rajan [70]	Struts/JSP	yes	outcome	not stated
Crawljax [51]	Ajax DOM states	no	no	manual
FeedEx, FRAGGEN [52, 75]	DOM states	no	no	coverage
Kurant [39]	React components	yes	n/a	F1, no routing
Oshima and Kishi [57]	Angular pages	yes	no	6 apps, faults
Rima et al. [65]	Angular workflows	yes	conditions	7 apps, tests run
Zhang et al. [83]	SPA routes, from bundles	yes	no	4 apps, vs. ZAP

Table 2.2: Static analyses that model framework behaviour explicitly, and the mechanism each of them resolves.

Work	Framework	Mechanism modelled
F4F [68]	Java web frameworks	reflective calls, from a specification generated from code and configuration
JackEE [3]	Java enterprise frameworks	components and entry points, declaratively
Jasmine [13]	Spring	dependency injection and aspect-oriented programming
Aurebesh [56]	AngularJS 1.x	identifiers shared between template and controller

Chapter 3

Static navigation analysis and scope attribution

This chapter presents the analysis that is the main contribution of the thesis. Section 3.1 defines the graph model and states the soundness property claimed for it. Section 3.2 presents scope attribution, the analysis proposed in answer to RQ3. Section 3.3 describes how both are implemented in the extractor. The evaluation of the analysis is described separately, in Chapter 4.

3.1 Graph model and soundness

Definition 3.1 (UI transition graph)**.** *A UI transition graph is a labelled transition system* (S, Σ, G, M, E) *with*

$$E \subseteq S \times \Sigma \times G \times M \times S.$$

S *is the finite set of* screens*, the states of the system and the nodes of the graph.* Σ *is the finite set of* event kinds *that can be raised at a screen, such as the activation of a* `routerLink` *or a call to* `Router.navigate`*.* G *is the set of* guards*, each held as the symbolic source text of the condition under which an edge may be taken.* M *is the set of* modalities*, which has exactly three elements,*

$$M = \{must, may, unknown\},$$

and whose elements are confidence labels: a modality states how firmly the extractor asserts an edge, and never denotes a screen, a route or a navigation target. An edge $(s, \sigma, g, m, s') \in E$ *reads: at screen* s*, event* σ *under guard* g *leads to screen* s'*, asserted with confidence* m*. Both the source* s *and the target* s' *are elements of* S*, namely the first and the last component of the tuple. Constructs in the source that agree on source, event kind, guard and target yield the same edge, which carries the strongest modality among them, with* M *ordered unknown* $<$ *may* $<$ *must, so* E *holds at most one modality for each* (s, σ, g, s')*; the paragraph on witnesses below says how the constructs are traced.*

What the materialised graph carries beyond the model. The extractor emits two node kinds that are not screens, and the edge relation above does not range

Table 3.1: What the published graphs are made of: nodes by kind, where an anchor stands for a whole subtree of screens and *other* counts unplaced origins and dynamic targets together, and edges by modality. These are the attributed graphs; the node column of Table 5.1 counts the graphs of the same snapshot without attribution, which hold no anchors and fewer *other* nodes.

	nodes			edges		
Application	screens	anchors	other	must	may	unknown
realworld	10	2	0	9	56	0
alfresco	27	1	26	0	36	19
ghostfolio	84	6	96	0	266	137
k8s dashboard	67	1	76	125	81	103
bitwarden	98	4	147	1	373	155
mifos	606	14	204	15	10590	206
All	892	28	549	150	11402	620

over them. An *unplaced origin* stands for a component, with the navigation sites in it, that the analysis could not place under any screen; a *dynamic target* stands for a destination only the run time knows. Both are kept in the output rather than dropped, because a site the analysis cannot place is a loss Chapter 5 has to count, and a destination it cannot name is one the run time could still take. Table 3.1 reports how much of each graph they are. Over the corpus 549 of the 1469 nodes are of these two kinds, 37.4 % of them, and 620 of the 12172 edges reach a dynamic target. A further 73 edges leave an unplaced origin and are held at *may*, so they sit inside a tier Chapter 5 reports while their source is not a screen. An edge that reaches a dynamic target asserts that the screen it leaves has an outgoing navigation whose destination the analysis could not name, and asserts nothing about which screen is reached. Claim 3.1 and the conversion ratio on which RQ1 is defined range over the edges whose source and target are both screens.

A *must* edge is claimed to fire for every input on which the enclosing code executes; a *may* edge is possible but conditional; *unknown* marks an edge whose modality could not be determined. The third modality carries the destinations the analysis cannot name. In the materialised graph it labels only edges that lie outside E, namely edges into the `u_` sink of Section 5.5, which records that a screen is left by an event whose target only the run time knows. The pinned extractor does produce such edges: 620 of the 12172 edges in the corpus, in 5 of the 6 applications. Carrying this modality and reaching a node that is not a screen coincide on every edge of every corpus graph, and the census refuses to report a graph in which they come apart. The conversion ratio of RQ1 and the soundness property of Claim 3.1 are defined over the other two tiers, so the unknown tier is counted in the edge totals and in no rate. The model also assigns *unknown* to navigations inside signal effects (Section 3.2.5), which may have a screen as target and so lie in E; no pass of this version emits one, so in the measured graphs the two still coincide. Guards are stored as source text and are never evaluated by the tool. A consumer of the graph can read a guard, but is not expected to decide whether it holds. This keeps the set G opaque and means that

the model does not need to represent application state.

Alongside the modality the artefact records a numeric *confidence* on each edge, which is not part of the model and carries no claim this thesis makes. It is a finer version of the same ordering: 1 for a must edge; 0.6 for a guarded edge with a literal target, or 0.5 when the guard is asynchronous; at most 0.5 for every other may edge, whatever demoted it; and 0 for an edge into a dynamic target. Where several constructs yield one edge, the edge carries the modality and the confidence of the most certain of them, and each construct it cites records the modality it earns on its own, so the edge is as certain as the best evidence for it and no construct is cited as stronger evidence than it is. Every figure in this thesis is reported over the modalities, never over the confidence.

Screens and routes. The set S is derived from the route configuration, and not every route contributes a screen. Write *Routes* for the set of routes the application declares and $component(r)$ for the component that route r renders. Then

$$S \;=\; \{\, screen(r) \;\mid\; r \in \mathit{Routes} \text{ and } component(r) \text{ is defined} \,\},$$

where $screen(r)$ is the node for route r, identified by the full path that concatenates the paths of r and of its ancestors. A route that declares `children` but no `component` is only a path prefix, as Section 2.7 describes: it contributes no element of S, and its path is carried into the full paths of its children. A route whose only effect is a `redirectTo` renders nothing and likewise contributes no element of S. In Angular a redirecting route is only a redirect: the router refuses a configuration that declares `redirectTo` and `children` together, and likewise `redirectTo` with a `component`, so a redirecting route can never also be the layout under which a child renders. Both refusals were read off the router's own configuration validator in the version vendored with this work, Angular 18.2.14; no corpus repository is checked out where the invariant could be re-read, so the statement holds only for that version. The same is not true of every client-side router. Vue Router applies a record's redirect only when that record is the last one matched, so a redirecting parent does render its component for a matching child. Section 6.4 returns to this, because the model's step from "redirects" to "renders nothing" is a fact about Angular rather than about routers. Because $screen(r)$ is identified by the full path, the map $screen(\cdot)$ is injective on the routes whose full paths are pairwise distinct. Angular also permits two component-bearing routes with the same full path, told apart at run time by `pathMatch` or by a `canMatch` guard. The model assumes that this does not occur: the extractor keeps the route declared first under a full path and drops the later ones. For Vue and for React the extractor also declares the drop, as a note carrying the file and line of the record it discarded, so a graph built from a table with a repeated address says so rather than being quietly one screen short; the note is true of the record and over-stated about the graph, because where both records name the same component the address still renders it. Angular's extractor drops silently, which is one of the asymmetries Section 6.4 returns to. Two routes with different paths that render the same component give two screens, which is the case that makes the attribution of Section 3.2 necessary.

Witnesses. Every edge of static or runtime origin carries a witness: the file and line of a construct that produced it, or an observation identifier. Where several

constructs produce the same edge, one of them is kept as the witness and the others are recorded as merged witnesses, so that every construct behind an edge can be traced (Section 4.2). No edge is added to the base graph without a witness, and edges added manually are kept in a separate overlay. The tool's specification states this as an invariant, and the first evaluation method checks it mechanically.

Claim 3.1 (Soundness of the must tier)**.** *Every must edge corresponds to a navigation that occurs, for every input on which the enclosing code executes, over the declared route set, under the route-order assumption of Section 6.3, and for a site whose component is not placed by a join over an ambiguous selector, within the following execution model: the event is a plain primary activation that navigates the current browsing context, no other listener the event reaches starts a navigation, and no call the reader cannot resolve to a body in the application, nor any work the handler schedules for later, starts one.*

The execution model is part of the claim and not a detail of the implementation. Section 6.3 lists the cases it excludes: a call into a library that cancels the navigation by starting another, a second listener on the element or an ancestor, and an activation with a modifier key or into another browsing context. On such an execution a must edge can name a navigation that does not complete, and the claim says nothing about it.

That assumption carries much of the tier: without it the must tier holds 24 of its 150 edges, on 22 of 25 constructs.

The evaluation tests a necessary condition of this claim rather than the claim itself, and the distinction decides how the figures of Chapter 5 may be read. The witness check of Section 4.3 re-reads the source at every location an edge cites and asks whether the construct found there names the destination the edge names. A must edge whose construct that earns must names a different destination cannot satisfy the claim, so a failure of the check there refutes it; agreement does not establish it. The other constructs a must edge cites are evidence for may and are not judged against the claim. Whether the navigation occurs *for every input on which the enclosing code executes* is decided by the rules that place an edge in the must tier instead of the may tier. A site is demoted to may in the following cases.

- **Guards and resolvers.** A guard the navigation runs demotes it, whatever the guard would decide, since the guard half of the rules is read off the route configuration and never evaluates a guard. Which guards a navigation runs is decided the way the router decides it. A route the source and the target share stays active, and its `canActivate` and resolvers run again only if its path or matrix parameters can change (the default `paramsChange`) or its `runGuardsAndResolvers` policy otherwise directs. A navigation runs the `canMatch` of every route its address is matched through; the `canActivate`, `canLoad` and resolvers of every route it enters, `canLoad` until the route's code has been loaded, which the extractor, not knowing whether it has, takes as every time; the `canActivateChild` of every route above those; and the `canDeactivate` of every route it leaves. A child at the empty path that the target's own address resolves to is one of the routes it enters. A resolver is

awaited by the router and demotes the edge as an `await` does, and a guard list the reader cannot bring back to an array demotes it as a guard does.

- **Target and source addresses.** The target is the address the source already displays: the router ignores such a navigation in its default configuration, or follows it only when a parameter or the query differs, so the extractor holds it at may as a matter of policy, which is the safe side for an application that configures otherwise; a shell or anchor edge is judged per screen and drawn per screen where the screens disagree. The source is never displayed at its own address because a redirect answers it. The address is one that two declared routes, or a route and a redirect, can answer, since Angular takes whichever it meets first. The base a relative target resolves against cannot be read. The URL names an auxiliary outlet, whose route the adapter does not read.

- **Conditional execution.** The site runs only on some executions of its own handler: under a template control-flow block, a structural directive, an `<ng-template>`, an ICU case or a container that renders its content lazily; under a branch, in a loop or a `catch` clause, in an optional call or a default value, inside a callback; after an `await`, an early `return` or `throw` or a `break` out of its block; in a function declared at module level or a member of its class that no handler calls unconditionally; in a component mounted under a template condition on that screen; or in a route component behind a conditional outlet. A handler is a member the framework calls itself, a lifecycle hook or a member `@HostListener` registers, or one the template binds to an event as a bare call outside any template condition.

- **A later navigation may cancel it.** The router cancels a navigation when another starts before it has finished, so a navigation is demoted when code that runs after it, before the event is handled, may navigate. That code is the rest of the expression that holds it (the body of a call it is an argument of, a right operand, a branch of a conditional whose condition it is), the statements after it in its block and in every block around it, a `finally` block around it, and what runs once its function returns: what each caller in the class does after calling that function, handler or not, and for a constructor or lifecycle hook the hooks Angular calls after it. Code there may navigate when it holds a navigation call, in a branch or a callback as much as at the top, or a call to a member of the class, a function of the file or a method of an injected service whose body does. An awaited navigation has finished before the rest of its function runs, so what follows it there does not count, but its callers go on as soon as the function first awaits, so what they do still counts. A call the reader cannot resolve to a body in the application is taken not to navigate (Section 6.3).

An exception from an earlier synchronous call is not counted, since every call can throw; an `await` is, because a rejected request is an ordinary outcome of the code that awaits it. The router's own fetch of a lazily loaded route is not counted either: a failed chunk load is a fault of the deployment rather than an outcome of the application, and the extractor takes the code as deployed. The extractor's unit

tests cover each of these rules in both directions, a site the rule demotes and one it leaves alone, and a mutation sweep deletes each rule in turn and checks that some test fails (Section 5.5); they share the extractor's reading of each construct, and no second classifier re-derives them. What stands in for one, for the condition rules, is a review in which the assistant the declaration describes read the constructs behind the round's must tier in the source, twice and separately, without the extractor's readers, and recorded a verdict for each (Section 5.5). Section 5.5 reports the corrections to the extractor's reading. Three made the tier smaller, each hidden by the one before; the fourth, found by review rather than by the witness check, closed gaps in the demotion rules, replaced the third's merge rule, which had demoted an edge one of its constructs made certain, and replaced a guard rule that demoted an edge for guards the router does not run on it. What the check establishes is therefore narrower than the claim: that every cited site exists in the source and agrees on its target. The stronger half of the claim rests on the demotion rules and is not independently verified.

This is the only soundness property claimed in the thesis, and it holds only relative to the declared route set. No such property is claimed for the may and unknown tiers. The witness of a may edge shows that the corresponding call site exists in the source, but it does not show that the transition is ever taken. This difference between the tiers matters for the evaluation. Whether the target found at a witness location agrees with the target of the edge is a meaningful question for must edges, and for may edges whose target is a literal. It is not a meaningful question for edges that the extractor produced by over-approximation, and treating it as one leads to spurious failures, as described in Section 4.3.2.

3.2 Scope attribution

This section specifies more than the prototype contains, and the division is the same throughout. Component nesting, router outlets, the shell rule, anchored edges and the call graph with dependency injection, read at the granularity of class members with bindings taken application-wide, are implemented in the extractor and are the parts evaluated in Chapter 4. The dispatch relations that carry NgRx effects and router event subscriptions, guards that return a `UrlTree`, the injector hierarchy that decides which binding of a token applies where, and navigation inside signal effects and observable subscriptions, which the model labels *unknown* (Section 3.2.5), are part of the model only: they are specified here to define the analysis, and the extractor approximates or omits them as Section 6.3 lists. A navigation site no implemented relation reaches contributes no edge, and the evaluation counts it in the defect class "Attribution, service" of Table 4.2. Section 3.3 gives the same division from the implementation side.

3.2.1 The defect being addressed

In the version of the extractor used for the baseline measurements, a navigation site is attributed to a screen only if it occurs directly in the component that the route for that screen renders. Navigation sites anywhere else are not attributed to any screen. This one cause explains three symptoms in the baseline: shared components contribute no edges, because the application shell places the header next to the router outlet and not inside the subtree of any screen; a component mounted at several routes contributes one edge instead of one per route; and applications that navigate from injected services produce no edges at all.

3.2.2 Scope

Every navigation site is anchored at a *scope*: the set of screens from which it can fire. A scope is thus an element of the powerset $\mathcal{P}(S)$, and the abstract domain of the analysis is $\mathcal{P}(S)$ ordered by inclusion, with set union as join,

$$X \sqsubseteq Y \iff X \subseteq Y, \qquad X \sqcup Y \;=\; X \cup Y,$$

which is the powerset lattice over the screens. Its least element is the empty scope,

$$\bot \;=\; \varnothing,$$

a site that fires from no screen at all. The greatest element is $\top = S$, every screen.

Two different things are called a scope, and the rest of this thesis keeps them apart. Write $\mathit{scope}^*(x)$ for the *true scope* of a site x, the set of screens from which x really can fire, which is a property of the application and not of any analysis; and $\mathit{scope}(x)$ for the *computed scope*, the set the fixpoint of Section 3.2.3 returns. A screen in $\mathit{scope}^*(x) \setminus \mathit{scope}(x)$ is a scope too small and a screen in $\mathit{scope}(x) \setminus \mathit{scope}^*(x)$ is a scope too large. No true scope is established anywhere in this thesis. For one application a third set is read off the sources by hand, the *structural reference* $\mathit{scope}^R(x)$, which applies the analysis's own relations without its implementation (Section 5.3.4). Agreement between *scope* and scope^R shows that the relations are implemented as stated; it does not show that either equals scope^*. Everywhere else the thesis reports the computed scope and says so.

The empty computed scope is likewise two situations, and conflating them would hide a defect inside a property of the application. A site is *unreachable* when $\mathit{scope}^*(x) = \varnothing$: the code exists and no screen can reach it, so contributing no edge is the correct answer. A site is *unresolved* when $\mathit{scope}(x) = \varnothing$ because no rule of the analysis reaches it (a navigation in a service nothing is known to inject, or behind a construct the reader could not follow), and the true scope is simply unknown. Both yield $\bot$, so $\bot$ alone does not say which holds. The extractor therefore does not let an unresolved site vanish silently. A site in a component that no rule places is anchored at an unplaced origin, a node that carries the component's file, and its edges are held at *may*; a site in any other class whose scope is empty contributes no edge, and the extractor records a note naming its file and the reason. Either trace is what makes

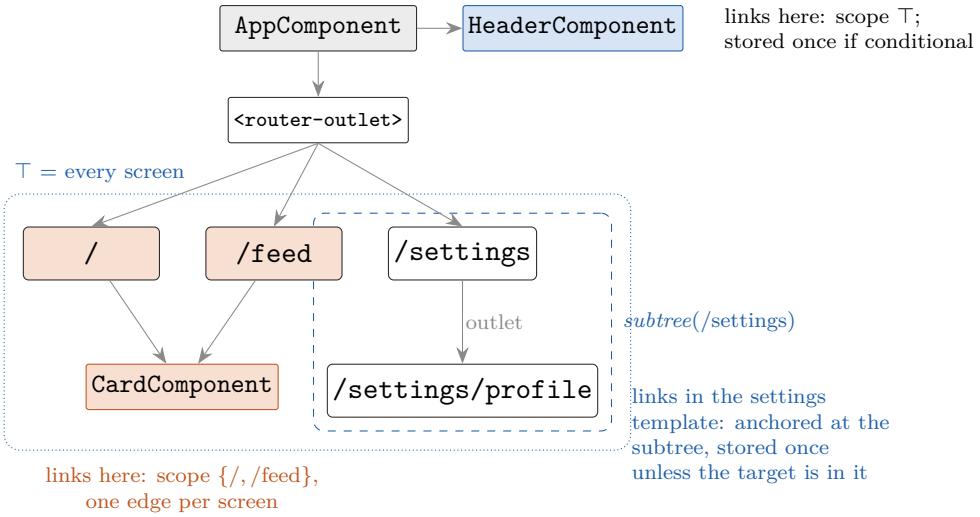


Figure 3.1: Scopes in a small application. The header is rendered next to the outlet, so its links fire from every screen. The card is rendered inside two unrelated screens, so its scope is the union of the two, which is none of the named elements. The settings template is rendered on its own screen and on its child screen, which is exactly *subtree*(/settings). Grey arrows show component nesting and outlets.

the unresolved sites countable in Chapter 5 rather than indistinguishable from code that is genuinely unreachable.

Three elements of the domain are named, because attribution anchors edges at them. Writing $r' \succeq r$ for the reflexive ancestor relation on routes, that is r' is r itself or an ancestor of r in the route tree, they are

$$\{\mathit{screen}(r)\}, \qquad \mathit{subtree}(r) \;=\; \{\, \mathit{screen}(r'') \in S \;\mid\; r \succeq r'' \,\}, \qquad \top \;=\; S,$$

and they form the chain

$$\bot \;\sqsubseteq\; \{\mathit{screen}(r)\} \;\sqsubseteq\; \mathit{subtree}(r) \;\sqsubseteq\; \top.$$

The chain is stated for a route r with $\mathit{component}(r)$ defined. For a prefix route, which has `children` but no `component`, $\mathit{screen}(r)$ is undefined and only $\mathit{subtree}(r)$, the set of the screens of its descendants, is defined. The chain then reads $\bot \sqsubseteq \mathit{subtree}(r) \sqsubseteq \top$. These names are a representation device for anchoring edges, not a restriction of the domain. The join of two scopes is their union, not the least named element above both: a component mounted at two unrelated leaf routes has the scope $\{\mathit{screen}(r_1), \mathit{screen}(r_2)\}$, which is none of the three named elements. The names matter for storage, where an edge anchored at $\top$ or at $\mathit{subtree}(r)$ is kept once rather than once per screen. Table 3.2 assigns a scope to each Angular construct. Figure 3.1 shows the three kinds of scope in a small application, and Figure 3.2 shows the domain for three of its screens.

The row for services is why a simple traversal of the component tree is not enough. A service has no scope of its own. Its scope is the join of the scopes of the places that call it, which may themselves be services. Attribution is therefore computed as a fixpoint over the input relations of Section 3.2.3. The outlet relation gives the initial

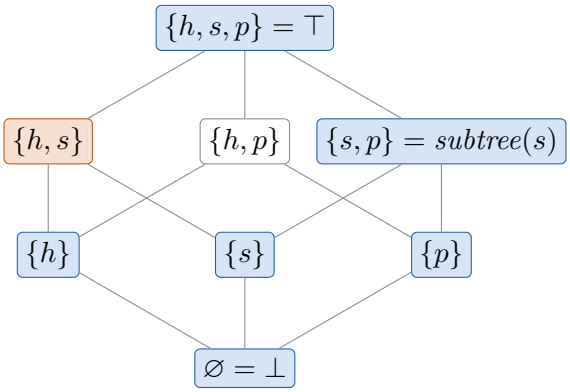


Figure 3.2: The scope domain $\mathcal{P}(S)$ for three screens: h for `/`, s for `/settings` and its child p for `/settings/profile`. Shaded elements are the named scopes at which edges are anchored. The join is set union: a component rendered in both h and s gets $\{h\} \sqcup \{s\} = \{h, s\}$, marked in orange, which is not a named element, so its edges are stored per screen. The height of the lattice is $|S|$.

scopes, and scopes are propagated along the component nesting relation N, from each component to its own members, along the call graph CG, along the dispatch relation D, and from each navigation site to the entry guards of the routes its target lies under, with $\mathcal{P}(S)$ as the abstract domain. This is a monotone dataflow problem in the sense of Kildall [38, 55], and two properties of it do separate work. Finite height gives termination. The height of $\mathcal{P}(S)$ is the number of screens $|S|$, so the scope of one node can rise at most $|S|$ times before it reaches $\top$, and the iteration can make only finitely many changes. Monotonicity of the transfer functions gives the result: the iteration reaches the least solution of the constraint system, and that solution does not depend on the order in which nodes leave the worklist. The transfer functions here are monotone because the only operation they apply to a scope is a join with another scope, and union is monotone in both arguments.

A worklist solver re-examines a node once for each rise of its own scope, and the work done for a node is proportional to its out-degree in $N \cup Mem \cup CG \cup D \cup T$, where Mem links each component to its members and T links each callable holding a navigation site to the entry guards that navigation runs. A run therefore makes at most

$$O\Big(|S| \cdot (|N| + |Mem| + |CG| + |D| + |T|)\Big)$$

transfer-function applications, after an initialisation that seeds the routed components, the shell, the `canDeactivate` guards and the interceptors in $O(k \cdot |S|)$, where k is the number of seeded nodes. Each application is a union of two subsets of S, which costs $O(|S|)$ with scopes held as bit sets, so the running time of the solver, initialisation included, is bounded by $O(k \cdot |S| + |S|^2 \cdot (|N| + |Mem| + |CG| + |D| + |T|))$. This is the bound for the full model, and it covers the solver only: building the relations it runs over, which includes parsing the sources and resolving calls, is a separate cost not bounded here. $|Mem|$ is the number of members of components and is at most the number of callables, and $|T|$ is at most the sum over navigation sites of the number of their targets, times the depth of the route tree times the largest number of entry guards on one route.

Table 3.2: Scope of each construct that can contain a navigation site.

Construct	Scope
component above the router outlet (the shell)	$\top$
component of route r	$\mathit{subtree}(r)$
component mounted at routes $r_1, \ldots, r_k$	$\mathit{subtree}(r_1) \sqcup \cdots \sqcup \mathit{subtree}(r_k)$
component rendered in the template of another component	join of the scopes of the components that render it. The prototype applies this row only to components that no route mounts (Section 3.3)
member (method, property initialiser, lifecycle hook) of a component	the scope of the component
method of an injectable service	join of the scopes of its callers
`canActivate`, `canActivateChild` or `canMatch` on route r	join of the scopes of the navigation sites with a known target in $\mathit{subtree}(r)$, an approximation of the screens a navigation into r starts from
`canDeactivate` on route r	$\mathit{subtree}(r)$
HTTP interceptor	$\top$
NgRx effect for action a	join of the scopes of every dispatcher of a

The part of the analysis that is implemented runs over the nesting, outlet, membership and call-graph relations only, so its worklist is bounded by the bound above with D and T empty; it is not the full model restricted to those relations, since the restrictions, fallbacks and stand-ins of Section 3.3 are computed outside the worklist. The call-graph pass, a worklist of set-valued joins along member call edges, is bounded by it. The nesting and outlet part is cheaper than that specialisation, because reachability from each source needs no set-valued joins. Scopes are propagated along the component nesting graph from the shell and from the component of each screen, which is plain reachability and costs

$$O\Big(|S| \cdot (|C| + |N|)\Big),$$

for the traversals, one of the nesting graph per routed component, layout and shell, where C is the set of components and the number of such sources is taken as $O(|S|)$; adding the source's screen set to each component reached costs a further $O(|C| \cdot |S|)$ per source. What makes the propagation necessary in both cases is that scopes travel transitively and the scope of one node can rise more than once, once for each screen that reaches it. Cycles are not the reason. The nesting, outlet and call-graph relations the prototype implements may contain cycles, since mutually recursive templates are legal, and the worklist handles them; a cycle in CG or D likewise only means that a node can be re-examined after one of its own successors.

3.2.3 Input relations

Four relations, in order of increasing difficulty, provide the input to the fixpoint computation. *Component nesting*: $A \to B$ when A's template instantiates B's selector, resolved within the correct import scope; matching selector text globally is unsound, since two components may share a selector string in different scopes. *Router outlets*: components on the path from the root to the outlet form the shell and take $\top$; components below the component of route r take its scope *subtree*(r); a named outlet carries a second mount, which makes auxiliary routes representable without separate treatment. *Call graph with dependency injection*: a call links to a method only after resolving the receiver's type and its binding in the relevant injector. Resolution follows class hierarchy analysis [17], with Angular's provider declarations as the source of bindings, as recommended for framework-heavy code [3]. Where a token admits several implementations the analysis joins over all of them, which is one source of *may* modality. *Dispatch relations*: a dispatch is treated as a call-graph edge from every dispatcher to every handler, which admits NgRx effects and router event subscriptions under the same fixpoint.

Algorithm 1 gives the computation, and its propagation steps are the transfer-function applications bounded above. A step adds a screen to a scope only when an edge of N, *Mem*, *CG*, D or T carries it there, and scopes only grow, so what the algorithm computes is the least solution of the constraint system that Table 3.2 states. In the model a component that is both mounted by a route and rendered in the template of another component therefore holds the join of the two scopes. Section 3.3 reports where the prototype departs from this: it does not propagate into a component that a route mounts, and it adds fallbacks that are not constraints of the table, so its result is not the least solution of this system or of any restriction of it.

3.2.4 Guards as program points

A guard that returns a `UrlTree` is itself a navigation, typically to a login screen. Its source is not the guarded route: a `canActivate` or `canMatch` guard on r runs while the router leaves the screen the navigation started from, so the redirect edge leaves that screen, the origin of an edge into *subtree*(r), and only a `canDeactivate` guard redirects from a screen of *subtree*(r) itself. Treating guards only as labels on inbound edges misses this edge class entirely. Further, the guard of an edge in the model is the conjunction of the conditions on the containment path from screen to call site (structural directives) with the guards the router runs on that navigation (Section 3.1). Reporting only the second part gives a weaker condition than the true one. The prototype does not materialise that conjunction in the graph that is measured. There, a structural condition in a template, or an enclosing `if` or conditional expression around a navigation call, demotes the edge to may but is not written into its guard, so the guard of a screen-to-screen edge holds only the router's guards. Only the optional mode that anchors navigations at the controls that trigger them, which no table of Chapter 5 uses, keeps the enclosing condition of a handler's navigation as source text beside the router's guards. The graph evaluated

in Chapter 5 therefore records that a navigation is conditional without recording the condition. In the baseline, for example, a link that is rendered only inside `@if (isUser())` was labelled with the authentication guard of its target route. That guard is satisfied by any logged-in user, including users who are not the owner and who therefore never see the link.

3.2.5 Boundary

A navigation inside a signal effect has a firing condition that is a data dependence on a reactive graph. As Section 2.7 notes, only the read set is statically visible. The model gives such edges the modality *unknown* and records the set of signals read by the effect as the symbolic guard; no pass in this version emits one. The analysis does not try to determine when these edges fire. It marks this limit explicitly instead of giving a result that looks more precise than it is.

3.2.6 Anchored edges

If the result of attribution were stored explicitly, a single link in the application shell would produce one edge for every screen. Where every site has a single target, the size of the graph would then be bounded by $|S| \cdot |\mathit{sites}|$, where *sites* is the set of navigation sites; for the largest application in the corpus that bound is 605×1343. A site whose target is only partly known can reach several screens, so in general

$$|E_{\mathit{expanded}}| \;\leq\; \sum_{x \in \mathit{sites}} |\mathit{scope}(x)| \cdot |\mathit{Targets}(x)|,$$

which is $O(|S|^2 \cdot |\mathit{sites}|)$ when a site of scope $\top$ reaches every screen. Anchoring removes the factor that comes from the source scope, not the one that comes from the targets. Instead, each edge is stored once, anchored at its scope, and the set of edges leaving a particular screen is computed when it is requested:

$$\mathit{edgesFrom}(r) \;=\; E_{\top} \;\cup \bigcup_{r' \succeq r} E_{\mathit{subtree}(r')} \;\cup\; E_{\{\mathit{screen}(r)\}},$$

where E_X is the set of edges anchored at scope X and $r' \succeq r$ is the reflexive ancestor relation on routes introduced above, so that the term for $r' = r$ contributes $E_{\mathit{subtree}(r)}$ itself. The query is made for a screen, so $\mathit{component}(r)$ is defined, while r' ranges over all ancestors of r, prefix routes included. An edge whose scope is some other screen set is stored at each of its screens and reaches $\mathit{edgesFrom}(r)$ through the $E_{\{\mathit{screen}(r)\}}$ term. Figure 3.3 contrasts the two representations.

Claim 3.2 (Expansion)**.** *The expansion of the anchored edge set equals the materialised edge set.*

The claim is tested rather than proved: Section 5.3 compares the two sets on the corpus and on fixtures.

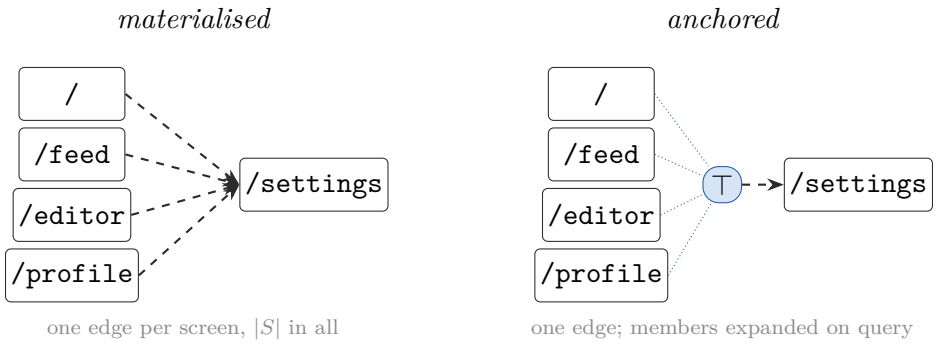


Figure 3.3: A header link to `/settings`, rendered under a condition and so held at may on every screen, stored two ways. Materialised, it becomes one edge per screen, including one from `/settings` itself, which is not drawn. Anchored, it is stored once at the scope node $\top$, and *edgesFrom*(r) adds it to the edges of any screen r when asked. Claim 3.2 states that both forms describe the same transitions. Only a reason for may that every screen shares keeps the anchored form: an unconditional link is demoted from `/settings`, the screen it already displays, but not from the others, so the extractor stores it as one edge per screen.

3.3 Implementation

The extractor, *uigraph*, is available as open source. It consists of a core that is independent of any particular framework and a set of adapters, one per framework. The core defines the graph model, its representation on disk, and the operations over graphs. Each adapter maps the source constructs of one framework onto the shared model, following a fixed interface. The Angular adapter reads route configuration, templates and class code with the TypeScript compiler API, and emits nodes for screens and edges for `routerLink` attributes and `Router.navigate` calls, each with the rule identifier and source location that produced it as witness.

Scope attribution is implemented as a separate pass in the Angular adapter, enabled with the option `-attribution`. With the option off, the extractor produces exactly the baseline edge set; this was checked on every application in the corpus. The pass implements three of the four input relations of Section 3.2: component nesting, router outlets and, as the second departure below describes, the call graph. The dispatch relation is not implemented.

Component nesting. Every component in the application, excluding test hosts and stories, is read with its element selectors and its template. An edge $A \to B$ is added when A's template contains an element with B's selector. When a selector belongs to more than one component, the candidates are narrowed to those that A's file imports; if none is imported, the analysis joins over all of them and records a note. Edges from such a join are not capped at *may*, so a site in one of the candidates can be placed on a screen that renders another; Claim 3.1 excludes such edges (Section 6.3); whether any corpus must edge comes from one was not measured. An attribute selector is matched the same way: a directive is rendered by the templates that carry its attribute, so it has a scope in the same way as a component does.

Router outlets. The route tree is recorded during route extraction. A route component's template is rendered on its own screen and on every descendant screen, so its navigation sites get the scope $subtree(r)$. A component that is not routed, is not placed on a screen by the nesting relation and holds the primary `<router-outlet>` is the application shell, and everything it renders gets the scope $\top$. A component that is not routed and not referenced, but whose subtree contains the outlet, is most likely mounted by a route the extractor cannot see, for example through a helper function that wraps a route table. It is also treated as shell, but its edges are capped at *may*, so that Claim 3.1 is not put at risk by a scope that may be too large.

The pass departs from Algorithm 1 in two respects. The first is that it does not propagate a scope into a component that a route mounts: a routed component keeps $subtree(r)$ and does not also receive the scope of a component whose template renders it. Traversal does pass through a routed component, so a component nested beneath it still receives the scopes of the components above; only the routed component's own sites are left at $subtree(r)$. Taken alone, this restriction can only shrink a routed component's scope relative to the full system, so through it a navigation site in a routed component can be attributed to fewer screens than the model allows, never to more. Two other rules of the pass can enlarge a scope beyond the model's: components that nothing references and no route renders but whose subtree holds the primary outlet are treated as shell and given $\top$, with their edges capped at may, and an ambiguous selector that no candidate's import resolves joins over all candidates, which Claim 3.1 excludes. A scope that is too small loses edges and cannot invent them, which is why the restriction cannot put Claim 3.1 at risk; what it costs is recall, and the constructs it strands are counted in Chapter 5.

The second departure is in how the call graph of Section 3.2.3 is read. Algorithm 1 ranges over every component and callable, and the pass does too: its nodes are the members of classes, and a member of a component runs on that component's screens. A call `this.f.g()`, where `f` is a constructor parameter, a typed field, a field initialised with `inject(T)`, or an `@Inject(TOKEN)` parameter, is resolved through the declared type of `f` to the class that type names, to the classes a provider declaration binds it to with `useClass` or `useExisting`, or, for an abstract class no provider binds, to its subclasses in the application, which is class hierarchy analysis [17] restricted to the declared type. The departure is that bindings are read across the whole application rather than relative to the injector that resolves them, so where two injectors bind a token differently the join covers both; that can only widen a scope. A factory or value provider is not read: a call on its token resolves to the token's own class if the application declares one, and a call on a type declared outside the application contributes no edge. The seeded call-graph iteration on its own reaches the least solution of its constraints. The scopes the extractor emits for service sites add two fallbacks to that solution. A member no caller reaches falls back to the join over every component that injects its class, the per-class approximation an earlier version used for every service. A member the iteration does reach can still receive more screens: where a caller the graph does not see, one that nothing calls and nothing seeds, reaches the member, the join of that caller's class is added, or this class's own join if nothing places the caller. Neither fallback is a constraint of Table 3.2, and the first is not monotone in the call graph: a member whose only caller is found receives

that caller's screens instead of the wider join, so adding a call edge can narrow its scope. The emitted service scopes are therefore the least call-graph solution widened by these fallbacks, for reached members as well as for unreached ones, and not the least solution of the constraints in Table 3.2. Service sites are held at *may* either way, because the calls on the way carry conditions the analysis does not follow.

Anchored edges. Edges with scope $\top$ are stored once at a node of kind `scope`. Edges whose screen set is exactly $\mathit{subtree}(r)$ for a screen r with at least one descendant are stored once at a scope node that lists the members of that subtree; the prototype does not anchor at a prefix route that is not itself a screen, whose screen sets are stored per screen. Other screen sets are stored as one edge per screen. The core library provides the expansion query $\mathit{edgesFrom}(r)$ and a function that materialises all anchored edges, which is used to test Claim 3.2.

The dispatch relation is not implemented in this version, so a navigation in an NgRx effect has no scope from the model itself. Two approximations stand in where the model has no relation the pass implements, and Section 5.3.3 withholds them below the *full* level, so no lower level counts their sites. The relation T is not implemented, and a guard is given instead the screens of the routes that name it, as a bare identifier, in `canActivate`, `canActivateChild`, `canMatch` or `canDeactivate`: the screen of each such route, not $\mathit{subtree}(r)$, and nothing for a route that is no screen, such as a componentless prefix. For a `canDeactivate` guard that is narrower than the model's scope; for an entry guard it is not the model's rule, since the redirect leaves the screen a navigation starts from, so the stand-in can be too small where navigations into the route start elsewhere and too large where they do not start at the route itself. An effect, being registered for the lifetime of the application, is given $\top$. Edges from both are capped at *may*. A class the call graph does not reach and nothing injects still has no scope, and the evaluation counts those in the defect class "Attribution, service" of Table 4.2. The pass is covered by unit tests for each of the cases above, including a test that the baseline is unchanged when the option is off.

One change was made to the baseline itself. For components with an inline template, the witness of an edge pointed at the line where the template starts rather than at the construct. The witness now points at the construct's own line. This can change witness locations but no edge. On the corpus, the baseline graphs with and without this change are identical, including witnesses.

Algorithm 1 Scope attribution as a worklist fixpoint over $\mathcal{P}(S)$. The relations marked *model only* belong to the full analysis and are not implemented in the prototype evaluated in Chapter 5; the prototype's other departures, among them the stand-ins for guards and effects and the rule for routed components, are listed in Section 3.3.

Require: routes R with route tree; for each route r its entry guards $\mathit{entry}(r)$, named in `canActivate`, `canActivateChild` or `canMatch`, and its `canDeactivate` guards $\mathit{leave}(r)$; components C; nesting $N \subseteq C \times C$; HTTP interceptors I; call graph CG; dispatch relation D; guard relation T, linking a callable x to $g \in \mathit{entry}(r)$ when x holds a navigation site whose target lies in $\mathit{subtree}(r)$

Ensure: $\mathit{scope}(x) \in \mathcal{P}(S)$ for every component and callable x; the callables include the members of components and of services

for all x **do** $\mathit{scope}(x) \leftarrow \varnothing$
end for
for all $r \in R$ and c the component of r or a layout wrapping it **do**
    $\mathit{scope}(c) \leftarrow \mathit{scope}(c) \cup \mathit{subtree}(r)$ ▷ own screen and outlet descendants
end for
for all c on the nesting path from the root component to the holder of the root-level primary outlet **do**
    $\mathit{scope}(c) \leftarrow S$ ▷ application shell, $\top$
end for
for all $r \in R$ and $g \in \mathit{leave}(r)$ **do**
    $\mathit{scope}(g) \leftarrow \mathit{scope}(g) \cup \mathit{subtree}(r)$ ▷ *model only*: runs as the user leaves r
end for
for all HTTP interceptor $i \in I$ **do** ▷ *model only*: registered for the application
    $\mathit{scope}(i) \leftarrow S$
end for
$W \leftarrow$ every x with $\mathit{scope}(x) \neq \varnothing$
while $W \neq \emptyset$ **do**
    take x from W
    for all y with $(x, y) \in N$ **do** ▷ x's template renders y
        PROPAGATE(x, y)
    end for
    for all members m of x, if x is a component **do** ▷ a member runs where its component is shown
        PROPAGATE(x, m)
    end for
    for all y with $(x, y) \in CG \cup D \cup T$ **do** ▷ CG: member level, bindings read application-wide; D, T: *model only*
        PROPAGATE(x, y)
    end for
end while
procedure PROPAGATE(x, y)
    if $\mathit{scope}(x) \not\subseteq \mathit{scope}(y)$ **then**
        $\mathit{scope}(y) \leftarrow \mathit{scope}(y) \cup \mathit{scope}(x)$; add y to W
    end if
end procedure

Chapter 4

Evaluation setup and research material

This chapter describes how the analysis of Chapter 3 is evaluated. Section 4.1 presents the corpus of applications and discusses why it can be considered a valid basis for the measurements. Section 4.2 defines the metrics used to answer each research question. Section 4.3 describes the five evaluation methods, what each of them can and cannot show, and the arrangements that keep the results reproducible.

4.1 Corpus selection and validity

4.1.1 Selection

Ten candidate applications were screened against six criteria. The criteria were applied in order of increasing cost, so that a candidate could be rejected cheaply before more expensive checks were made. The criteria were: (1) a real application rather than a library or template; (2) a nested, lazily-loaded route tree; (3) guards present, since guards are what produce conditional transitions; (4) end-to-end test configuration for a browser driver; (5) retrievable history in which tests and source change together; (6) an active repository. Six candidates were retained (Table 4.1). Criterion (4) was checked as configuration rather than as a usable suite: at the pinned commits only alfresco has specifications the test-source cross-check can read, so that method covers alfresco alone, and ghostfolio and bitwarden carry no browser-test configuration at all, so for them the criterion was not met. It was therefore not applied uniformly: `gravitee`, below, was rejected for a thin UI suite while two applications with none were kept. Each of them is fixed to a specific commit so that the measurements can be repeated on the same code.

Four candidates were rejected. They are listed here together with the reason for rejection, since the rejected candidates are part of the selection protocol. `angular/components` is a component library with no product router. `vendure` has 169 end-to-end specifications, all of which exercise the GraphQL server and none of which drive the user interface. `gravitee` declares 618 routes but has 14 UI specifications. `SAP/spartacus` has a large Cypress suite, but its routing is configuration-driven in a way that the extractor does not currently support, so it was set aside as a possible candidate for later work. That reason is tool support, which is not one of the six criteria, so this rejection was made on a ground added

Table 4.1: The corpus. Routes are the route entries reached by the independent route parser of Section 4.3, as it stands after the corrections of Section 5.5. A count marked ≥ is a lower bound: the parser records a blind spot for that application, so the entries behind it are not in the figure, and Table 5.16 gives how many such blind spots each application has. An earlier version of this table gave kubernetes/dashboard as a lower bound of 42 because the parser could not then follow its lazily loaded modules; it now reaches them. The marks that remain are on alfresco, ghostfolio, bitwarden, mifos, on these grounds: children not resolvable to a route array; matcher route: path not statically knowable; spread route source: not followed.

Application	Commit	Routes	Role in the study
gothinkster/angular-realworld-example-app	`dd99ed2c`	14	size floor; all literal paths
alfresco/alfresco-content-app	`b88d47f4`	≥57	87 Playwright specs including a navigation suite
ghostfolio/ghostfolio	`9884fa0b`	≥135	constant-valued route paths; self-hostable
kubernetes/dashboard	`0ba796d9`	140	mid-size control
bitwarden/clients (web)	`e40ce6f0`	≥168	many guards, guard factories among them
openMF/web-app	`7f781796`	≥909	size ceiling

after screening began.

The screening also produced an observation that influenced the design of the evaluation. Among the candidates examined, the size of the route tree and the density of end-to-end tests were inversely related. The largest applications had few or no tests that exercise the user interface, while the application with the most extensive navigation-oriented test suite was of medium size. As a result, no single application in the corpus provides both a large route tree and execution-based ground truth. The evaluation therefore applies the methods that require test suites only to the applications that have them, and the methods that do not require tests to all six.

4.1.2 Validity of an open-source corpus

Open-source corpora are common in empirical software engineering, and their threats to validity have been studied. Kalliamvakou et al. show that a large share of GitHub repositories are inactive, personal or not software projects, that stars do not measure quality, and that forks inflate counts [35, 36]. The remedy they recommend is to filter the candidates using explicit criteria and to report those criteria, which is what the selection procedure above does. The threat that remains is one of external validity. Open-source applications may differ from proprietary enterprise applications in size,

in routing style and in the mechanisms used for navigation. In addition, I selected the corpus and I wrote the extractor, which introduces a possible selection bias.

Three measures address these threats. The selection protocol is reported in full, including the rejected candidates; the six criteria were fixed before any results were obtained, but one candidate was rejected on tool support and the test-suite criterion was not met by two of the retained applications. The corpus is also stratified across nearly two orders of magnitude in the number of routes, so that any effect of application size is visible and not averaged away. Finally, the commits are fixed, and the evaluation harness is released with the thesis so that the measurement can be repeated rather than taken on trust. None of these addresses a further threat: the repairs of the second round were found on this corpus and are measured on it, with no held-out application (Section 6.2), so the figures after the repairs are fitted to it. A fourth measure is planned: a case study on a proprietary Angular application from industry, reporting only aggregate figures; this would be the most direct response to the external validity threat, and it is discussed further in Chapter 6. There is also no standard benchmark corpus for single-page applications. The web testing survey confirms that evaluations in this area are carried out on collections of applications chosen for each study [37]. For this reason the corpus and its manifest are reported as one of the outputs of the thesis.

4.2 Metrics

Each research question is answered with one main measure. They are defined here.

Navigation construct. A navigation construct is a `routerLink` binding in a template, or a call to `Router.navigate` or `Router.navigateByUrl` in class code, in the application source excluding tests. Constructs are found by a separately written parser. It shares no code with the extractor, but Section 5.5 qualifies what that independence covers: for the route-path forms the second round added, the two now resolve the same constructs by the same rules, and three of the parser's own corrections were made because it disagreed with the extractor. Attributes such as `routerLinkActive` are not constructs; counting them by substring was one of the corrections described in Section 4.3.2.

Conversion ratio (RQ1). The conversion ratio is the fraction of in-scope navigation constructs that are witnessed by at least one must edge. A construct is matched to a witness on its own line or on the line either side, to absorb offsets in the extractor's locations, so two constructs on adjacent lines can both be credited by one witness, and the ratio is an upper bound at line granularity. Query-only navigations and the two exclusions of Table 4.2 are left out of the denominator as well as the numerator. When several constructs produce the same edge, the extractor keeps the most certain as the edge's witness and records the others as merged witnesses, each with the modality it earns on its own, so every construct that yields an edge can be traced; on the current instrument a construct counts as converted only where it earns must itself, and that is how the current tables of Chapter 5 count. The superseded round tables quoted there, measured before witnesses carried a modality,

counted every construct a must edge cites. The ratio is computed over constructs rather than edges because the number of edges depends on representation: the same construct can be stored as one anchored edge or as one edge per screen (Section 3.2). An earlier version of this measure divided the number of must edges by the number of constructs. It was replaced for that reason (Section 4.3.2).

Agreement between a scope and the rendered interface (RQ3). The measures above check nodes, edges and witnesses. None of them can be asked the question the contribution turns on: is the set of screens attribution computed for a navigation site the set that site can really fire from? The runtime method answers one direction of it. A scope says a site fires from certain screens, so the harness navigates to each screen the graph holds, lets it render, and reads the in-app links the DOM actually carries; those are compared with the edges the graph's own per-screen query yields at that screen.

The two directions are not equally decidable. A link the application renders at a screen that the graph does not have leaving it is unambiguous: the link is there, so either the site was attributed to too few screens or it was never read. The opposite case, an edge the graph has and the DOM does not show, is weaker in two ways. A template renders links conditionally and this harness signs in to nothing, so an absent link may be correctly attributed and merely unreachable in the state observed. And where the analysis could not resolve a site's target to a single address it contributes every address the static prefix admits, recording the edge at reduced confidence; such an address is not a claim that the screen links there, so the screen not linking there refutes nothing. The harness therefore judges this direction only over the predictions that name a destination exactly, and publishes how many those are, so a share is never read off a denominator that cannot support one.

Two bounds belong with the figure. The oracle sees a link only where the rendered page carries a readable target, which means an anchor's `href` or a static `routerLink` attribute; a `Router.navigate` call in an event handler leaves nothing in the DOM to read, so the sites it covers are the template-sourced ones. And a screen whose guard redirects it to a sign-in page without a backend renders another screen's links, so its scope cannot be judged from what rendered; such screens are counted apart. On this corpus that is most of them, and Section 5.3.1 gives both numbers rather than the share alone. A third bound is the corpus: the harness serves one built directory at the server root, which ghostfolio's per-locale build does not fit, since each of its bundles declares a base path of its own. That application is measured by every other method here and by none of this one.

The ablation and the site–screen ground truth (RQ3). Two further measurements answer a question the methods above leave open: whether the attribution computes a set of screens that agrees more closely with a structural reference than a simpler analysis does, or only a larger one.

The first withholds the mechanisms of Section 3.2 one at a time and extracts the whole corpus at each level. *Owned* attributes a navigation site to the screen whose routed component's own file holds it, which is direct ownership and is what resolving the owning object yields in the Android literature. *Nesting* adds component containment: a component reached through template element selectors is attributed

to the screens of the components that render it. *Rooted* adds the shell rule, so a component the application's root renders is attributed to every screen; since Oshima and Kishi assemble their page information from the root component as well as from the components beneath it (Section 2.4), that level, and not containment alone, is the one that measures how far the prior work reaches on this corpus. *Outlet* adds router-outlet descent, and *full* adds the member-level call graph, which places a service's navigation on the screens of the members that call it, together with its fallback join and the stand-ins for guards and effects. Each level adds to the one before it in this fixed order, so a step measures what a mechanism adds given those below it, and the *full* step is not one mechanism alone. A sixth level, *class*, is run beside the nested ones rather than among them: it replaces the call graph with the rule the call graph superseded, which joins a navigating service over every component that injects its class, so that what the call graph changes can be measured. The nested levels are nested because the implementation withholds results rather than computing them differently, and the harness checks that on the real graphs rather than assuming it: a (site, screen) pair present at one level and absent at the next is a defect in the switch, and no table is written when one is found.

The second supplies what no method above has: a reference scored in both directions. For one application the screens each navigation site fires from were read off the sources by hand, under the rules stated in the file's method field, and recorded, with the derivation for each site, in `route-oracle/ground-truth/realworld.json`. This is the structural reference of Section 3.2.2, called the ground truth below for brevity. Every level is then scored against it in both directions, which is the comparison the runtime oracle cannot make: a pair the reference has and the level does not is a scope too small by the reference, and a pair the level has and the reference does not is a scope too large by it. Three limits belong with it. It is one application, chosen because it is small enough to read exhaustively. It is the author's reading, so it is not independent of the tool's author; it is published with its reasoning so that a reader can check it against the same sources. Its rules are the analysis's own relations: a component is on every screen whose routed component renders it through template selectors, what the shell renders is on every screen, and a navigation inside a service member is on the screens its callers are on. A template condition or a guard that admits a screen only in one sign-in state does not narrow a scope under those rules, so the ground truth tests whether the implementation applies the relations correctly, not whether the relations match what the running application renders. And it is checked in the one direction available: the scorer reports every site the graph places that the ground truth does not list, which is the ground truth's own blind spot. That check found nine sites missing from its first version, because the search that enumerated them discarded every line carrying `routerLinkActive` and this application writes both attributes on one anchor.

Which ratio is "coverage". Several of the measures below are fractions of something over something, and more than one of them has been called coverage in the literature this thesis builds on. They are not interchangeable, and on this corpus they differ by a factor of fifty, so each is named here with the population it is a fraction of. *Node coverage* is declared screens that have a node, over the screens the independent parser declares. This figure rises if less of an application

is read, which is why the blind-spot count is printed beside it. The same quantity over the addresses the running framework mounts is a different and smaller figure, reported separately in Section A.5. The *conversion ratio* is navigation constructs witnessed by a *must* edge, over all in-scope constructs; a construct a must edge cites only as evidence for may does not count, as the definition above states. *Recovered* is the same numerator widened to an edge of any tier, and on an application whose screens sit behind guards the two are far apart: almost every transition such an application has is conditional, so a corpus can be mostly recovered while almost none of it is deterministic. *Accounted* is recovered plus the sites that name a reason for not being recovered, and it is the only one of these that measures whether the losses are declared rather than how much is recovered. Where this thesis says coverage without qualifying it, it means node coverage; every other figure is named.

Loss breakdown (RQ2). Each construct is matched to the edges whose witness cites it. A construct that is in scope, cited by no must or may edge and not accounted for is *lost*, and it is assigned to exactly one defect class by the first rule in Table 4.2 that applies. Because the rules are ordered, the classes partition the lost constructs and their sizes can be compared within each application. Guard defects do not appear in this table, since a wrong guard mislabels an edge rather than removing it. No method here checks a guard label directly: the witness check compares the construct, its target and the destination of a must edge, not whether the transition is unconditional, so a must edge that should have been conditional passes it. Guard handling is exercised only by the guard-removal mutation operator and by the source review of Section 5.5.

The class *other* is reached in unrelated ways: a template site whose owner cannot be resolved, a route component whose route has a node elsewhere, and a construct no rule matches. The harness tallies them as one class, so its size counts constructs left for inspection by hand and is not the size of one cause.

The nested branch decides a class from the state of the construct's *owning* route component. A construct can therefore be assigned to one of the node classes and still be converted once attribution gives it a scope through the shell or a subtree, because the scope it needs is not the one its owner would have supplied. Section 5.3 measures how often that happens.

A call of the form `navigate([], {queryParams})` changes only the query string of the current screen. It is not a transition between screens, so it is reported separately and not counted as a loss.

Change in coverage and must-tier precision (RQ3). The conversion ratio and the loss breakdown are measured on one commit of the extractor, run twice on identical inputs with the attribution pass disabled and enabled. Measuring one commit rather than two rules out any difference between the runs other than the pass itself, which two versions could not guarantee; Section 3.3 reports that with the pass off the extractor reproduces the baseline edge set exactly on every application. *Must-tier precision* is the fraction of must edges whose witness line contains a construct of the claimed kind with the claimed target, *and* whose destination node sits at an address the independent route set declares. The second half of that test was added late and is the reason it is stated here in two parts: asking only whether

Table 4.2: Rules that classify a construct, in the order the harness tests them. The first three rows take a construct out of the denominator, the next one counts it as recovered, in the tier that cites it, and the two after that as accounted for; only a construct that reaches the rows below is lost. The indented rules are the nested decision taken when the construct's owning route component has no node.

Class	The construct ...
Query only	changes only the query string of the current screen (`navigate([], {queryParams})`), so it is not a transition and is not counted as a loss
Named outlet	targets an auxiliary outlet (`{outlets: ...}`)
No commands	names no destination at all
Converted, must or may	is cited as witness by an edge of that tier
Accounted, dynamic	is cited by an edge into a sink whose destination only the run time knows
Accounted, unplaced	is recorded by the extractor as a site it read but could not place
Attribution, service	has no owning component, or its owner is a class that is not a component: a service, guard, interceptor, resolver or effect. A site in a template whose owner cannot be resolved falls to *other* instead
Attribution, shared	is in a component that no route renders directly
Owner is a route component, but that component has no node:	
Component unlinked	a node for the owner's route exists, but the extractor did not link it to the component
Constant path	every declaration of the owner's route gives its path as a constant reference rather than a literal
Other	some declaration of the owner's route does have a node
Lazy boundary	some declaration of the owner's route is in a lazily loaded routes file
Node missing	a declaration exists but none of the above applies, or neither parser found a route for the owner
Constant target	owner has a node, but the construct's own target is not a literal
Other	matches none of the above; inspected by hand

the witness agrees with the edge's destination takes the destination for granted, so a node minted at an address the application never mounts passes every question the check asks. This is how one defect went unnoticed: one application's lazily loaded reports module was read at the project root, so the graph held `/:filter` where the application mounts `/reports/:filter`, and edges pointed at that address while the check reported the tier clean (Section 4.3.2). No count of those edges is given, because the defect is repaired and the graph that carried it no longer exists, so the figure cannot be re-measured against anything. RQ3 is answered positively only if the recovered share and the conversion ratio, both over the same construct denominator, do not fall and the recovered share rises, and must-tier precision does not fall. Because an edge whose destination the check cannot read is neither a pass nor a failure, precision is stated as bounds: the share of must edges that pass, over

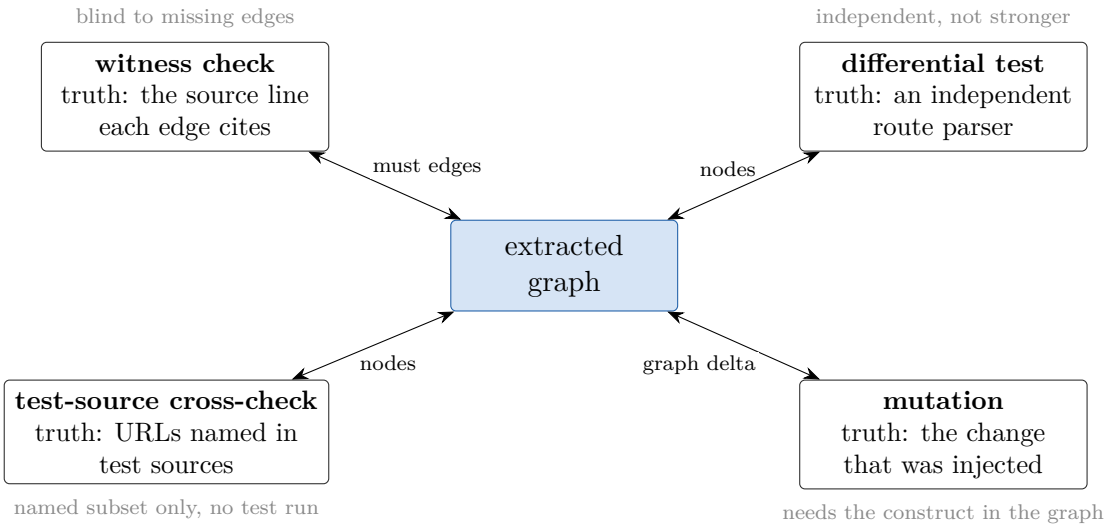


Figure 4.1: The four evaluation methods that read the applications' sources, the part of the graph each one checks, and where its ground truth comes from. The fifth, the runtime route probe, is not drawn here because its ground truth is the running application rather than any part of the graph. No method sees everything; the grey notes give the blind spot of each, as listed in Table 4.3.

all of them, as the lower bound, and the share that do not fail as the upper bound. The criterion is read as no detected failure in either run; it cannot be read as a precision that did not fall where the check reads only part of the tier, and the two runs' bounds are shares of different sets of must edges, so they are not compared as rates.

Node-level recall. The fraction of distinct URLs named in an application's own end-to-end test sources that have a matching node in the graph. It is a cross-check on node extraction, which all edges depend on.

Detection rate (RQ4). The fraction of injected source changes that produce the predicted change in the graph, reported per mutation operator with its n. A change whose construct the graph does not contain is excluded rather than counted as a miss, since it can only repeat a coverage gap measured under RQ1. The rate is therefore conditional on coverage, and the number of excluded mutants is reported alongside it.

4.3 Evaluation methods

Five methods are used. None establishes correctness alone, so Table 4.3 states what each can show and where it is blind. Figure 4.1 shows which part of the graph each method checks and where its ground truth comes from.

Four of the five read the application's sources. That shared footing is their common weakness: two readers of the same text can be wrong in the same way, and an agreement between them then provides no evidence. The fifth was added for that reason and is described last.

Table 4.3: Evaluation methods, the quantity each measures, and the blind spot each has.

Method	Quantity	Blind spot
Witness check	Must-tier precision. For every must edge, the witness file is re-parsed at the claimed line, the construct kind and target are compared, and the destination node's address is tested against the independent route set.	Cannot see missing edges. Target agreement is invalid for over-approximated may edges, whose witness proves only that the call site exists. The destination test is only as good as that route set: where both tools read a route table at the wrong prefix, they agree on a wrong address and the test passes. This is the gap the runtime route oracle below was built to close, and it was not hypothetical: Section A.5 reports 10 addresses on which both readers agreed and the running application mounted none of them.
Differential test	Disagreements with a separately written route-configuration parser.	Diagnostic, not a metric. The parser is independent, not stronger: its first version missed whole route tables in several applications until it was corrected (Section 4.3.2), and for the path forms added in Section 5.5 it is no longer independent of the extractor at all.
Test-source cross-check	Node-level recall against a fourth reader of the source. The end-to-end specifications of the application are parsed and the URL literals in them collected; a URL the application's own test suite names is an address its authors expect to exist, and the extractor is asked whether it minted a node there.	Reads test source text and runs nothing, so a collected string is an address a test *names*, not one a test was observed to reach, and whether the suite passes is not established by this method. Tested subset only. A direct `goto` is not a UI transition, so visit pairs are not counted as edges. Textual extraction under-samples click-driven navigation.

continued on the next page

Table 4.3 continued from the previous page

Method	Quantity	Blind spot
Mutation test	Detection rate over four operators: rename a path, remove a guard, delete a `routerLink`, retarget a `navigate`. Ground truth is the authored delta, so nothing is circular. Detection is judged on the node and edge sets and on the witness set, since several constructs can produce one edge.	A mutant whose construct is absent from the baseline graph re-detects a coverage gap and is excluded, as is a guard mutant on a screen that no edge enters. The screens a guarded route yields are looked up by what its subtree names, so a component several routes render maps to every screen those routes declare; that over-approximates, which errs towards scoring a mutant rather than dropping it. Detection is therefore conditional on coverage, and the attributed and unattributed graphs do not exclude the same mutants, so their overall rates are not taken over one set: the comparison between them is also reported over the mutants both can score, paired by operator, file, line and subject. The sample is drawn from the graph, so it is not independent of the artefact under test. Sampling is seeded and stratified by operator.

continued on the next page

Table 4.3 continued from the previous page

Method	Quantity	Blind spot
Runtime route oracle	Node-level soundness against the framework itself. Each application is built in its development configuration, loaded in a headless browser, and asked for the route configuration the Angular router holds; every lazily loaded subtree is expanded by calling the application's own `loadChildren` and reading the routes out of whatever it resolves to. The addresses the extractor mints are then compared with the addresses the framework mounts. Nothing in this method re-reads the route files, which is the point: the answer comes from the application's own code. Which addresses in that configuration count as mounted is itself a reading, so it is checked against the framework's behaviour: the harness navigates the running application to a list of addresses and records what the recogniser made of each one, before any `canActivate` guard runs and after the `canLoad` and `canMatch` guards that take part in matching.	Requires the application to build and boot, which fixes the method to applications that can be built without their server. It reads the configuration rather than the rendered screen, so a route that a guard always refuses still counts as mounted, and a route whose component fails to render is not distinguished from one that renders. An address a route claims through a `UrlMatcher` cannot be decided from the configuration at all, since the function is not spelled out there. A workspace application that composes the library a measurement is scoped to mounts the shell's own routes as well, so part of any difference is scope and not reading.

4.3.1 Preconditions

Two properties of the extractor were checked before any of the measurements were taken. The first is determinism: two extractions from the same unchanged source produce identical sets of nodes and edges, when compared on identifier, route, event, guard and modality after sorting. Without this property the differences observed in mutation testing could not be distinguished from noise. The second is the absence of accumulated state: extracting into an existing database gives the same graph as extracting after the database has been deleted. Without this property the results for a mutant could be affected by the earlier baseline run.

4.3.2 Corrections made to the protocol

Eleven corrections were made to the protocol while it was being developed. They are described here because each of them would otherwise have led to an incorrect figure in this thesis, and because the underlying mistakes are general ones. Every figure quoted below is the one a superseded instrument produced, recorded so that the correction can be judged; none of them is reproducible with the harness this thesis ships, and none is cited anywhere else as a result. The few comparisons elsewhere that do rest on an earlier state of the extractor name the commit and read their figures from that commit's own generated results (Section 5.5).

In the test-source cross-check, a URL parameter named in a test was at first allowed to match a literal segment of a route, which produced matches that did not correspond to real screens. After the rule was tightened, the recall figure for one application fell from 86.7% to 66.7%. A negative test, which names a URL that must not exist, is now also excluded, which gives the 71.4 % reported in Chapter 5.

The first version of the witness check reported a precision of 44.7% for one application. On inspection, all 177 edges it had flagged were may edges with a confidence of 0.5 that referred to a single source line. They were the result of one over-approximated call site being connected to every declared route. The must tier of the same run had 69 correct edges out of 69.

In mutation testing, a mutant whose target does not appear in the baseline graph cannot show anything about the graph's sensitivity to change; it only repeats a coverage gap that has already been counted elsewhere. Three of the five mutation misses for one application were of this kind and were moved to a separate category.

The first denominator of the conversion ratio counted the substring `routerLink` in templates, which also matches the attributes `routerLinkActive` and `routerLinkActiveOptions`. For alfresco this counted 113 constructs where the parser finds 34. Constructs are now counted by the parser that also produces the loss breakdown, so the ratio and the breakdown share one definition.

The first version of the route parser used by the differential test did not look inside wrapper calls such as `Route.withShell([...])`, did not follow a lazily loaded NgModule to the routing module it imports, and walked lazily loaded route tables a second time without their prefix. On mifos it therefore reported that every declared screen had a node. After the correction it finds 610 declared screens, of which 58 have a node. The extractor was the same in both runs.

The conversion ratio itself was first defined as the number of must edges divided by the number of constructs. After attribution, this ratio rose from 5.1% to 75.7% for mifos, because a construct in a component rendered on many screens produced one edge per screen. The rise therefore came from how edges were stored. It is now computed per construct, and edges whose screen set is a whole subtree are stored once (Section 3.2).

The mutation harness patched the source by searching for the text of the construct, so a mutant aimed at one of several identical constructs edited the first of them instead of its own. In realworld the five guard mutants collapsed onto two sites, because the five `canActivate` properties are written with only two distinct texts.

The harness also judged detection on the node and edge sets alone, which scored the deletion of one of two identical links in a template as a miss, although the graph had behaved correctly: it kept the edge, which the second link still produces, and dropped only the witness that pointed at the deleted line. Detection is now judged on the witness set as well, which is the property change impact analysis actually needs. Three smaller changes followed from the same review. A retargeting mutant now points at another declared route instead of an invented one, so that it cannot be confused with a missing target; a later correction replaces the whole commands array rather than its first element, since retargeting only the head of `['/b', id]` composes an address no route declares, and scores the mutant on the edges the edited line witnesses. A guard mutant on a screen that no edge enters is excluded, because a guard is represented on the edges entering the screen it guards, so such a mutant can only repeat the edge-coverage gap already counted under RQ1. A rename mutant counts as detected when the renamed route appears in the graph, without also requiring the original node to disappear, since the same path segment may be declared in a second routing module. With the offset and witness corrections in place, on the same seed and the same sample, the detection rate for realworld changed from 71.4% to 95.2%. Sampling is seeded and stratified with a fixed quota per operator, and the detection rate is reported per operator with its n.

The eighth correction is to that guard exclusion rule, and it was found by reading the graph at the sites the rule had let through rather than by a figure that looked wrong. The rule asks whether any screen the guarded route yields is entered by a baseline edge, and it identified that route by the nearest path literal preceding the `canActivate` in the source. For a route that has children, that literal is the last child's path rather than the route's own; for an empty-path child it is the empty string, which is a segment of every absolute path and so satisfied the test whatever the application; and for a route whose path is a constant reference or a template literal there is no literal at all, which skipped the test. Four guard mutants that cannot be observed at all were therefore scored rather than excluded and reported as misses, which the discussion then explained as a defect in how the graph represents a guard; three of them guard screens no baseline edge enters and the fourth a route that yields no screen. The rule now matches the braces of the enclosing object literal to find the route, over a copy of the source with comment and string contents blanked so that a brace inside either cannot close the object early. It reads what the route's subtree renders (a component class, or the module a `loadComponent` imports) and looks those up against the class and file each screen node carries. A route that declares no component and no populated children is excluded as well, since it yields no screen for a guard to be recorded on, and the detection check is keyed on the same screens rather than on a substring of a path literal. On the superseded instrument the run scored 68 of the 70 sampled mutants and detected 59, with 5 producing no change in the graph at all; Chapter 5 reports what the corrected rule gives on the same seed and the same sample.

The ninth and tenth corrections are to how a removed guard is judged, and both followed a change in the extractor rather than a figure. Once the extractor gated an edge only on the guards its navigation runs (Section 5.5), a guard on a route that source and destination share no longer appeared on the edge between them, and

the rule that a removal must change every edge entering the guarded screens scored such a removal as wrong although the graph had done what the model requires. A removal is now detected when some edge loses a guard it carried, and excluded when no baseline edge carries the guard at all, since then no edge enters the guarded route from outside it. The tenth is that the comparison counts guards instead of collecting them: the extractor writes a guard factory without its arguments, so two guards on one navigation can print alike, and removing one of three identical texts left a list that a comparison of sets read as unchanged. Neither changes which mutants are sampled; both change how a sampled one is scored.

The eleventh is to the construct denominator, and it was found by a generator that refused to write rather than by a figure. The parser took a construct off any line its pattern matched, so a link inside an HTML comment, and a sentence of a code comment that names `router.navigate`, were constructs, and the extractor, which reads neither kind of comment, could only lose them. The loss classifier found no reason for the first, a link mifos had commented out, and the run that regenerates the round-two tables stopped on it. Comments are now blanked before the parser reads a line, with the compiler's own scanner for TypeScript so that a `//` inside a string is not taken for one; over the corpus that removes those two constructs, one in bitwarden and one in mifos, and nothing else.

Five of the eleven corrections were found the same way, by a figure that looked implausibly good or bad: the conversion ratio that rose to 75.7%, the precision that fell to 44.7%, the coverage that reached 100%, the construct count that tripled, and the detection rate that moved by more than twenty points. Each turned out to be a property of the measurement rather than of the extractor. A surprising figure is therefore checked against its inputs before it is written down, and the remaining figures in this thesis were checked in this way.

4.3.3 Reproducibility

The four methods that read the sources are re-run by two scripts, run at one commit of the extractor, each of which refuses to start while the extractor's packages carry uncommitted changes. `regen-round.sh` runs `run-all.mjs` twice (with the attribution pass disabled and enabled), which refuses an application whose checkout is not at its fixed commit and writes a results file together with the raw output of each method; the conversion, coverage, loss, remaining-loss, before-and-after and round-two tables are generated from those two runs. `remeasure.sh` maps the corpus again and generates what is read off the graphs themselves: the census, the expansion check, the witness check of the must tier, the ledger of the round's tier, the guard-only run, the call-graph counts, the ablation with its width table and the ground truth, and the mutation run. It ends by publishing the sweep of the demotion rules, which a third script, `mutation-sweep.mjs`, records apart because it runs the adapter's test suite once per rule (about forty minutes), and which is refused if it was recorded on other rules than the commit's. Every table of Chapter 5 that describes the final instrument comes out of these scripts at one commit; the few that describe an earlier instrument are generated from that instrument's own recorded results,

and the runtime method's from its own, as below. An earlier set of the round-two tables did not: the runs they were generated from were lost, the archived runs of the same commit gave a different loss ledger, and those tables were replaced by a regeneration rather than carried forward. The scripts need two things they cannot supply: the dependencies of the harness directory, which is outside the workspace and is installed with `npm ci` there, and a checkout of the extractor at the evaluated commit, which is passed to it. The extractor is run from its TypeScript source rather than from a build. An earlier arrangement built it instead, which required rewriting each package's `exports` field from source to build output before the built command could resolve its own sibling packages; that rewrite edited the checkout being measured, and it was the one step of this evaluation that could not be recorded in the repository. Running from source removes it.

The fifth method, the runtime route oracle, is re-run by its own documented command, because it needs each application built in its development configuration and served to a browser, which is a step the two scripts cannot take on a machine that has not built them. Its result files record no commit or runtime version, unlike the others, so the provenance guarantee below does not extend to them.

Each figure in the results file of the first four methods is recorded together with the commit of the extractor, the commits of the corpus applications, the version of the harness, and the version of the runtime. Every result table in this thesis is generated from those files; where the prose states a measured figure it reads the same generated macro the table does, so a regeneration cannot leave a sentence behind. Several generators refuse to write at all rather than emit a figure they cannot account for: the census will not print its table unless every route entry falls in exactly one named bucket and the buckets sum to the route total, and the before-and-after comparison will not print unless every construct assigned to the attribution class is either recovered or appears in the classification of the remaining losses. That ensures the accounting closes; it does not ensure that each class it lands in explains the construct's cause.

This arrangement has two limits. The evaluated commit of the extractor must be one a reader can fetch, so the figures are reproducible only once that commit is published; and the version of the harness decides several figures independently of the extractor, so a result file is only comparable with another made by the same harness version, which is why each one records it.

Table 4.4 lists, for each group of generated tables, the extractor commit and harness version its inputs were measured with and the recorded data it is regenerated from. The table itself, the corpus table and the corrections of Section 4.3.2, which report the superseded instrument, are written by hand. All figures of the final instrument were measured at extractor commit `c42f220`, harness 1.9.0 and mutation harness 1.9.1; the source of the Angular adapter is identical at `920e1aa`, where the sweep of the demotion rules was recorded. Three groups are historical by design and are labelled as such where they are printed: the first-round node audit of Appendix A, the earlier instrument with the round's own must tier, and the runtime scope and route checks. They were not re-measured on the final instrument, and the figures taken from them describe the instrument or graph named in the table. The runtime

Table 4.4: Provenance of the generated tables. *Final* means measured on the instrument every other figure describes; *historical* means measured on an earlier instrument or graph and printed as a record of it.

Tables	Status	Measured with	Regenerated from
Loss ledger, before and after, round two, plots	final	`c42f220`, harness 1.9.0	`results-round2/final14.tgz`; the plots also `results-rq4/`
Census, must tier, witness check, guard-only, route order, call graph, expansion	final	`c42f220`	the corpus graphs, via `remeasure.sh`
Ablation, structural reference	final	`c42f220`; generated at later commits with identical `packages/`	`results-ablation/ablation.tgz`
Mutation (RQ4)	final	`c42f220`, mutate 1.9.1	`results-rq4/`
Demotion-rule sweep	final	`920e1aa`, same adapter source[a]	`results-must/mutation-sweep.json`
Earlier instrument and the round's must tier	historical	`eb9dab80` and before	tables read from git at `f17a302` and `7d109c7`
Runtime scope check (Table 5.10)	historical	an earlier graph, before the call graph; scope check 1.3.0	`route-probe/results/`
Runtime route check (Tables 5.19, 5.20)	historical, 29–30 Oct	probe 1.0.0, comparison 1.1.0; extractor commit not recorded	`route-probe/results/`
First-round node audit (Table A.1)	historical	harness 1.2.0; recorded extractor `4505fd2a`, absent from the repository	committed table only
Construct prevalence (Table 6.1)	no extractor	pinned corpus commits	`approx-counts.mjs` over the sources

[a] `packages/core` differs between the two commits; the adapter tests the sweep runs import from it, so the sweep result is carried over on the adapter source alone.

route check records no extractor commit and was made on 29 and 30 October; its node total equals the final screen count, but that agreement is the only link to the final graph. The construct prevalence counts read the application sources only and do not depend on the extractor.

Chapter 5

Results

The figures in this chapter were produced by the harness of Section 4.3 on the corpus of Section 4.1. The numerical result tables are generated from its result files; the summaries compiled by hand are explicitly marked. The main conversion, loss, before-and-after and graph-composition tables describe the final evaluated extractor snapshot. The baseline is that code without scope attribution; the attributed version is the same code with the pass enabled. Each generated file records the snapshot it measures. Historical results are identified separately in Section 5.5; they explain the repairs rather than supplying a second population for the final comparison.

Two conditions of the measurement hold for every figure below. The corpus source trees are extracted without their dependencies installed, so the TypeScript compiler cannot resolve types from the framework or third-party packages, and type-directed steps are weaker than in an installed project. The corpus was also selected with the tool in view: `SAP/spartacus` was rejected because its configuration-driven routing is not supported (Section 4.1). The figures therefore describe this configuration on these six applications, and are likely to be optimistic for Angular applications in general (Section 6.2).

5.1 The two levels of loss (RQ1)

RQ1 is answered at two levels, and the two give very different answers. The first is whether the graph holds the screens the application declares at all; the second is how much of the application's navigation becomes a deterministic transition between them. The first is a precondition of the second, because a transition cannot start or end at a screen the graph does not have, but not a numerical bound on it: one ratio counts screens and the other constructs, so a missing screen costs the second ratio only the constructs that name it.

Table 5.1 gives the second measure on the final snapshot. The constructs that earn must number 21 in the whole corpus without attribution and 25 with it, in 4 applications with attribution and none on the other 2. Edge counts in that table must be read per tier and not summed as distinct constructs: one unresolved site can produce many over-approximated may edges.

Table 5.2 and Figure 5.1 give the first measure, comparing the nodes the extractor

Table 5.1: Construct-to-transition conversion per application, baseline. Conversion is the fraction of in-scope constructs whose own witness earns must; the edge columns count edges. *Nodes* counts every node of the baseline graph: the screens of Table 5.2 together with its unplaced origins and dynamic targets (Table 3.1 counts the attributed graphs, which add anchors and more of the latter). The baseline and attributed graphs are extracted from the same final snapshot on identical corpus inputs.

Application	Constructs	In scope	Nodes	Must	May	Conversion
realworld	40	39	10	1	24	2.6%
alfresco	34	27	37	0	7	0.0%
ghostfolio	198	186	112	0	60	0.0%
k8s dashboard	72	68	74	0	3	0.0%
bitwarden	131	112	113	1	89	0.9%
mifos	1343	1343	684	15	5572	1.4%

mints against the screens an independently written parser finds declared. Against that denominator node extraction is complete: on all six applications every declared screen has a node. Two qualifications apply, because the figure invites a reading that neither of them supports. First, the denominator is the screens a *static reader* can enumerate. Against the addresses the running framework actually mounts, the same node set is 892 of 1146 (Section A.5), and almost all of the difference is one construct in one application that the extractor declares unread. Second, the two static readers reached their agreement partly by being corrected against each other during this work, so their exact agreement is weaker evidence than it was when they were written independently; the runtime method is the external check for that reason, and in the other direction it finds no node at an address where the framework mounts nothing. That was not true of the extractor this evaluation began with, which read one application as a single node for the 84 screens it declares and the largest as under a tenth of its screens; Section 5.5 reports what had to be repaired to reach the figures in this table, and Appendix A.1 the handful of addresses the two readers still disagree about.

Completeness here is a claim about the screens the readers can *see*, and the blind-spot column bounds it: a route entry neither reader can resolve to an address is outside the denominator, and the applications that have such entries say so in their own soundiness notes. Refusing to read more of an application would raise this column rather than lower it, which is why it is printed beside the coverage figure and not instead of it.

5.2 Which failure is worth an analysis (RQ2)

Table 5.3 lists the defect classes and the evidence for each, separating the named evaluation methods from the other observations. Four of the five accepted classes rest on one named method: attribution, which three observations carry to four lines of evidence, and guards, named outlets and constant paths, which one observation each carries to two. The attribution class leans most on observations, with one method and

Table 5.2: Node extraction against the independent parser, in both directions: declared screens that have a matching node, and nodes for which the parser holds no declared screen. The blind-spot column counts route entries the parser could not follow, which bounds what either direction can show.

Application	Declared	Nodes	Matched	Coverage	Unmatched	Blind spots
realworld	10	10	10	100.0%	0	0
alfresco	26	27	26	100.0%	1	10
ghostfolio	84	84	84	100.0%	0	2
k8s dashboard	67	67	67	100.0%	0	0
bitwarden	98	98	98	100.0%	0	1
mifos	605	606	605	100.0%	1	1
All		892			2	

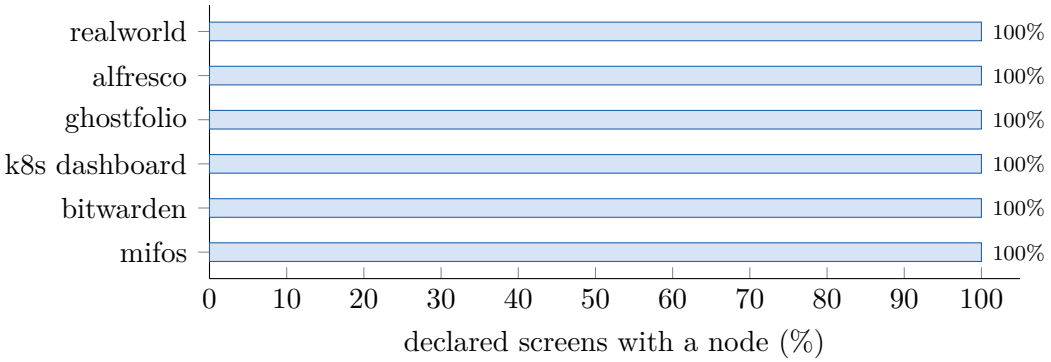


Figure 5.1: Node coverage: the share of declared screens that have a node in the graph. It is complete on every application; the blind-spot column of Table 5.2 bounds what that means, since a route entry neither reader can resolve to an address is outside the denominator.

three of them, and it is the class the analysis of Chapter 3 addresses. Its observations are not weak evidence in themselves, since two applications producing no edges at all is hard to explain any other way, but they are not independent instruments. The five classes found in the first round all have at least two confirmations. The quantitative breakdown below revealed a sixth candidate, a node that exists but is not linked to the component that renders it. It is confirmed only by the comparison between the route parser and the graph, so it is reported as a candidate until a second method confirms it.

Table 5.4 and Figure 5.2 assign every navigation construct to one outcome using the rules of Table 4.2. Of the 1818 constructs in the corpus, they show how many convert onto each tier and why the rest do not. A construct is counted on the must tier only when its own witness earns that modality. Guard defects are measured separately (Section 4.2).

Attribution is the largest loss class in every application in the corpus, and by a wide margin: of the 1818 constructs, the two attribution classes hold 454 between them, and no other class the loss rules name holds a tenth as many. The destinations only the run time knows, which the table counts apart because they are no defect of

Table 5.3: Defect classes and the evidence for each. A class is admitted on two lines of evidence, at least one from a named evaluation method; the methods column counts only those. This summary is compiled by hand; it is not harness output.

Class	Evidence	Methods	All
Attribution: navigation outside route components is invisible	mutation misses (at commit f17a302); multi-mount edge absent; manual audit; zero-edge applications	1	4
Lazy module boundary not followed	route parser; test-source cross-check; mutation	3	3
Guards unreliable in both directions	witness check at commit f17a302; source survey of the guard factory and the call-form `canActivate` (Section 5.5)	1	2
Named router outlets not modelled	test-source cross-check; source survey	1	2
Constant-valued route paths not resolved	route parser; node-count collapse	1	2
Candidate: node not linked to its component	route parser against graph	1	1

Table 5.4: Outcome of every navigation construct, baseline. Columns: realworld, alfresco, ghostfolio, k8s dashboard, bitwarden, mifos. All rows are measured on the final snapshot with attribution disabled.

Outcome	rw	alf	gf	k8s	bw	mif	Total
Converted, must	1	0	0	0	1	19	21
Converted, may	16	3	55	3	53	956	1086
Attribution, shared component	21	10	63	51	37	224	406
Attribution, service	1	8	9	5	15	10	48
Lazy boundary	0	0	0	0	0	0	0
Constant path	0	0	0	0	0	0	0
Component unlinked	0	0	0	0	0	0	0
Constant target	0	0	0	0	0	5	5
Node missing, other	0	2	3	1	0	34	40
Other	0	0	6	0	2	3	11
Dynamic destination	0	4	50	8	4	92	158
Source the analysis cannot place	0	0	0	0	0	0	0
Query only (not a transition)	1	0	11	4	19	0	35
Excluded: names no destination	0	0	1	0	0	0	1
Excluded: navigates a named outlet	0	7	0	0	0	0	7
All constructs	40	34	198	72	131	1343	1818

the extractor, come to about a third as many. Most of the attribution class comes from components several screens render rather than from services.

That ranking is a result of the repair campaign rather than of the baseline. When this evaluation began, two node-extraction defects dominated the corpus: one application declared every route path as a reference into a constant table, which collapsed its graph to a single node, and the largest application hid most of its route tables behind lazy module boundaries the extractor did not follow, while a further large group of its screens had nodes with no link to the component that renders them. Those classes are empty in Table 5.4 because the causes were found and repaired; what that cost, and the evidence that the repairs were genuine, are reported in Section 5.5. The ranking matters because it justifies spending an analysis on attribution rather than on any other class: once the extractor reads what the applications declare, the navigation it still cannot place is almost entirely navigation with no single owning screen.

The classes also interact, and the interaction runs both ways. A construct in a shared component that is only used inside screens which have no node is counted as attribution, although attribution alone cannot recover it. In the other direction, a construct whose own route component has no node would be counted as a node defect (none remains in the final baseline, though some did at commit f17a302), yet attribution can still reach it through a shell or subtree scope and convert it. The 454 constructs the attribution classes hold are therefore neither a floor nor a ceiling on what scope attribution recovers. Section 5.3 measures both directions.

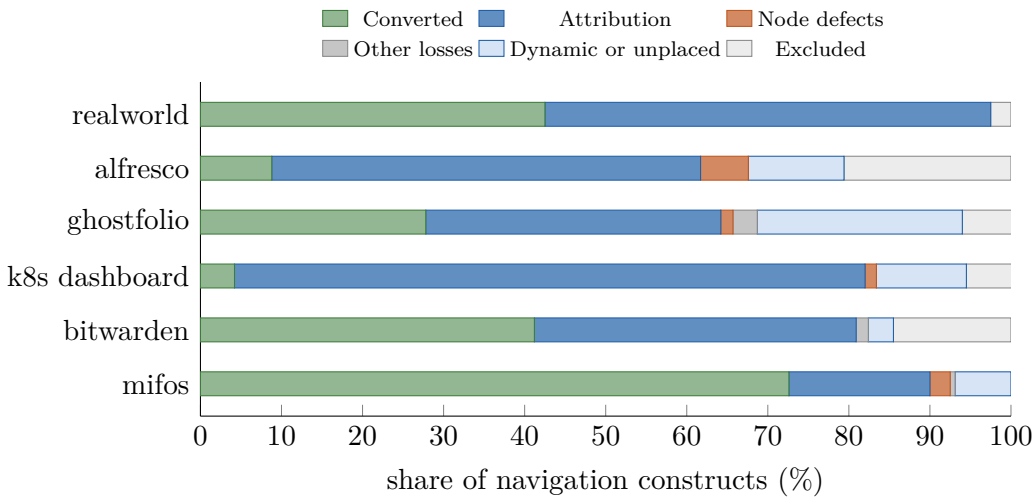


Figure 5.2: What happens to the navigation constructs of each application without the attribution pass. Attribution losses are the largest class in every application; of the node classes, the lazy-boundary, constant-path and component-unlinked ones are empty on this instrument, and the remaining one holds few constructs, most of them in mifos. The exact counts per class are in Table 5.4. *Converted* counts constructs witnessed at must or may, as that table does; the conversion ratio of Figure 5.3 counts must only.

Table 5.5: Results of the witness check, on the final attributed tier, and of the test-source cross-check. Mutation is reported separately in Section 5.4, on the attributed graph. This summary is written by hand from the harness output rather than generated.

Method	Result	n
Witness check, final attributed must tier	0 detected failures; 62 edges have a destination the checker cannot read. Unread edges are not passes, and target agreement does not establish Claim 3.1.	150
Test-source cross-check, alfresco	10 of 14 URLs the specifications name have a node; one negative test excluded.	14

Table 5.5 reports the witness check on the final attributed tier and the test-source cross-check. Section 5.5 gives the must tier its own treatment, because what the check reads and how an edge cited by several constructs is judged both changed in the course of this work, and the figure is not comparable across those changes.

5.3 Scope attribution (RQ3)

The analysis under evaluation is the one Chapter 3 states. Every navigation site is anchored at a *scope*, the set of screens from which it can fire, so a scope is an element of the powerset lattice $\mathcal{P}(S)$ ordered by inclusion with union as join (Section 3.2.2). A site written inside the component of a leaf route has that one screen as its scope; written in the component of a route that has children, it has $subtree(r)$, because

Table 5.6: Before and after scope attribution on the final evaluated snapshot. Conversion in percent and its change in percentage points; must-tier edges whose destination the witness check read and passed; and constructs assigned to attribution in the baseline that recover onto must or may. A construct's tier is the modality its own witness earns. The remaining must edges are unread destinations, all 62 of them on k8sdash, which are neither passes nor failures; no edge failed. Conversions from other baseline classes are in Table 5.8.

	Conversion			Must, read and passed		Attribution losses, recovered		
Application	Before	After	pp	Before	After	Before	must	may
realworld	2.6%	7.7%	+5.1	1/1	9/9	22	2	20
alfresco	0.0%	0.0%	+0.0	–	–	18	0	4
ghostfolio	0.0%	0.0%	+0.0	–	–	72	0	32
k8s dashboard	0.0%	2.9%	+2.9	–	63/125	56	0	8
bitwarden	0.9%	0.9%	+0.0	1/1	1/1	52	0	16
mifos	1.4%	1.4%	+0.0	15/15	15/15	234	0	210
All						454	2	290

that component stays mounted while any descendant is active (the shell rule of Section 3.2). A site written in a component that several screens render, or in a service that several components call, has no single owning screen, and its scope is the join of the scopes of the places that reach it; because a service may call another service, the result is a fixpoint over the component nesting relation, the call graph and the dispatch relation, with termination from the finite height $|S|$ of the lattice (Section 3.2). Edges are not materialised per screen: each is stored once, anchored at its scope, and the edges leaving a screen are computed when asked, which is what keeps the graph off the $|S| \cdot |sites|$ bound (Section 3.2.6).

The effect of the pass is measured by running one commit of the extractor twice over the same inputs with the pass disabled and enabled, so nothing but the pass differs. Three quantities decide the answer: how much navigation the pass recovers, whether it costs the deterministic tier anything, and what it still cannot place. The third shows where the analysis as implemented stops, and Section 5.5 returns to it once the extractor underneath has been repaired.

Table 5.6 and Figure 5.3 compare the extractor with and without scope attribution on identical inputs. The comparison is made construct by construct: a construct is *recovered* if it was lost to attribution in the baseline and is witnessed by an edge after attribution.

Attribution recovers 292 of the 454 constructs the loss rules assign to it, 64.3 % of that class, and takes the graphs from 5919 edges to 12172. Measured instead by RQ1's deterministic conversion ratio the gain is far smaller: 2 of the recovered constructs earn must and 290 earn may, and 2 of the six applications gain any conversion at all. The must constructs rise from 21 to 25: besides those two recoveries, both on realworld, two k8s dashboard constructs that the baseline already converted at may are raised to must, which the recovery count does not include. These are different measures of recovery, which is why both are reported: a gain in represented navigation need not be a gain in deterministic transitions when targets or execution conditions remain uncertain. Counted in edges the must tier grows from 17 to 150,

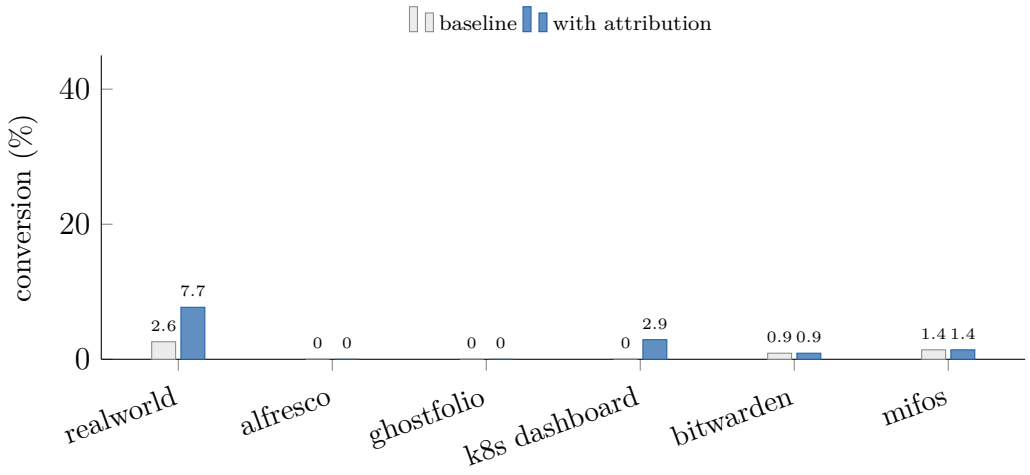


Figure 5.3: Conversion ratio before and after scope attribution, on identical inputs.

and the edges added are the links of two application shells drawn once for each screen the shell renders (Section 5.5), which is the reason the ratio is computed over constructs.

The attributed tier has 0 detected witness-check failures and 62 edges whose cited destination the check cannot read; the latter cannot be treated as agreements. The separate source review is reported in Section 5.5. Following each construct by its identifier across the two runs finds 0 that leave the converted classes, so no conversion of the baseline was lost. This measures recovery of represented constructs, while the incomplete mechanical comparison limits what can be concluded about must-tier precision.

The witness check also records the edges outside the must tier. Chapter 3 excludes them from the precision claim, because target agreement is not a meaningful test for an edge the extractor produced by over-approximation, and its figures are reported with the round's soundness table so that the exclusion is visible rather than assumed (Section 5.5): most locations the graph cites outside the must tier are over-approximations, for which the check claims no agreement, and 44.8 % of the corpus's edges come from one over-approximated call site in the largest application, one edge per possible target (Section 4.3.2). This count of locations grows with the may tier and says nothing about the must tier, which is why Chapter 3 excludes it from the precision claim, and no ratio is given for it here.

The anchored representation was checked on the same graphs. Claim 3.2 requires the expansion of the anchored edge set to equal the materialised edge set. The check cannot be a comparison between the expansion query and the library's own materialisation function, because that function is a loop over the query and the two can only disagree about which nodes they iterate. The anchored edges are therefore materialised independently, by walking the edge list and fanning each scope-anchored edge over the screens its anchor stands for, with its own deduplication.

The two agree on every application the check was run on, which Table 5.7 reports: the check refuses to write that table at all if the two sets differ anywhere. Those ratios are also the only measurement of what the representation saves, and that

Table 5.7: The anchored edge set against its expansion. *Anchored* is what the graph stores, *expanded* what the per-screen query yields, and *independent* a second materialisation written for this check. Claim 3.2 requires the last two to be equal; the check refuses to emit this table if they differ on any application.

Application	Screens	Anchored	Expanded	Independent
realworld	10	65	97	97
alfresco	27	55	100	100
ghostfolio	84	403	1166	1166
k8sdash	67	309	694	694
bitwarden	98	529	714	714
mifos	606	10811	33418	33418

saving is the reason the representation exists. On mifos the materialised edge set is 3.1 times the size of what is stored. What the check exercises is the member walk and the deduplication key; the definition of a scope's membership is shared by both paths and is not tested by it, so the claim is supported on 6 applications and on fixtures rather than proved. On 5 of the 6 the library's own materialisation function returns a larger set than either, which is the iteration difference predicted above and not a disagreement about the edge set. Where expansion makes two edges with the same source, target and event, they are joined into one, the strongest modality leading and the witnesses united, so that the result does not depend on the order the anchors are walked in; the check compares modality and witnesses as well as endpoints, and 1414 such joins occur across the corpus.

Table 5.8 gives the same recovery by the class each construct held in the final baseline and by the modality its own witness earns. Of the attribution-class recoveries, 2 earn must and 290 earn may. The diagnostic classes should not be interpreted as independent causes: a shared component can be assigned to attribution even when a missing target node prevents its recovery.

The historical comparison illustrates that interaction. On the extractor this evaluation began with, commit f17a302, 41 of 79 deterministic conversions, 51.9 %, came out of node-defect classes, against 38 from attribution classes. These figures come from that earlier commit's recorded results; they do not supply counts for the final comparison.

Table 5.9 classifies each construct that stays lost.

Every one of the 162 constructs that stays lost is in the table, and the two tables close the class between them: 292 recovered plus 162 remaining is the 454 the rules assigned, which the generator asserts before it will write either table. All but 2 carry a reason; those 2 have a scope and a destination the graph holds, and the diagnosis names no reason they stay lost. The reasons describe both limitations of the implemented analysis and limits of the source population. Some components have no recovered route or template path to a screen; this is an empty computed scope, not proof that the application never renders them. Some sites lie beyond the member-level call graph's supported receiver and provider forms. Other sites have a scope but a dynamic destination or a target for which the graph has no node. The

Table 5.8: Where recovered constructs came from on the final snapshot. Rows are their baseline loss classes; columns give the modality each construct's own witness earns after attribution. Constructs are followed by identity, not matched by source location.

Baseline class of the construct	must	may	total
Attribution, shared component	2	271	273
Attribution, service	0	19	19
Other, no class rule applied	0	2	2
Out of the attribution classes	2	290	292
Out of the node classes	0	0	0
Out of the remaining classes	0	2	2
All conversions gained	2	292	294

Table 5.9: Why attribution losses remain after attribution.

Reason	rw	alf	gf	k8s	bw	mif	Total
Owner component not reached from any screen	0	8	11	5	18	1	43
In a service or guard, not placed	0	6	4	4	8	7	29
Scope found, target screen has no node	0	0	23	2	6	3	34
Scope found, target fully dynamic	0	0	2	37	2	13	54
Scope found, not explained	0	0	0	0	2	0	2

final table records the reasons remaining after the implemented call graph, rather than losses measured before it was added.

5.3.1 Checking the scopes against the running interface

Everything above measures what attribution recovers. Whether the *set of screens* it computes for a site is the set that site fires from is a different question, and it is the question on which the contribution depends. The runtime method of Section 4.3 answers one direction of it: navigate to each screen the graph holds, let it render, read the in-app links the page carries, and compare them with the edges the graph's per-screen query yields there. Table 5.10 retains the recorded observations and graph predictions from an earlier evaluation. This browser experiment was not repeated, and its predictions were not recomputed for the final call graph. The section therefore documents what that earlier graph missed; the final-snapshot ground-truth and ablation comparisons below are separate measurements.

A scope claim has two ways to be wrong, and they are not equally testable. It can be too small, so that the application renders a link the graph does not attribute to that screen; the rendered interface decides this case. It can be too large, so that the graph attributes a site to a screen the application never fires it from; the rendered interface decides this case only where the graph claims a destination exactly. The two are reported separately below, and the second is reported with its denominator, because that denominator turns out to be what limits the method.

Table 5.10: Recorded scope predictions of an earlier graph against the rendered interface, on the 5 applications the harness could serve. These predictions were not recomputed for the final call graph. *Judged* counts screens that rendered themselves rather than redirecting to a sign-in page; the rest cannot be judged from what rendered, because a redirected screen shows another screen's links. *Not predicted* is the only column that counts against the analysis; the other direction is reported in the text, because it can be judged only where the graph names a destination exactly.

Application	Screens	Judged	Pairs rendered	Predicted by the scope	Not predicted	Raw anchors
realworld	10	7	25	25	0	59
alfresco	26	3	0	0	0	0
k8s dashboard	67	1	0	0	0	0
bitwarden	98	18	8	4	4	8
mifos	605	1	0	0	0	0
All	806	30	33	29	4	67

In the first direction, of the 33 (screen, target pattern) pairs the running applications render on the screens that could be judged, 29 are predicted by the scope the analysis computed (87.9 %), and behind these predictions stand 67 raw anchors. On realworld, which renders most of its screens without a backend, every rendered pair on every judged screen is predicted; the pairs that are not predicted are all bitwarden's. The unit of this check is the screen and the destination, not the site: the graph's edges leaving a screen are pooled into one set of destination patterns and set against the links the screen renders, and neither the witness nor the site enters the match. Two sites on one screen that name the same destination are therefore one pair, and a site attribution left off that screen is hidden whenever another site supplies the same destination. A confirmed pair shows that the screen's outgoing destinations cover what it renders; it does not show that each site behind them was attributed to that screen. An unpredicted pair is not affected by the pooling, since no edge of the screen reaches that destination at all, so the 4 are direct evidence of scopes that are too small. The evidence is limited in size: it covers 30 screens of 806, because 776 redirect to a sign-in page when there is no backend behind them, and 5 of the six applications are measured at all. ghostfolio is the one left out: it builds a separate bundle per locale, each declaring a base path of its own, and the harness serves a single directory at the server root, so its screens cannot be addressed the way the other five are. The oracle does not scale to this corpus; it settles one direction of a question, on part of it, that nothing else here settles at runtime.

The 4 links it does not predict are the informative part, and they share one cause. All four are in bitwarden: its `/login` screen renders links to `/hint` and `/signup`, and two further screens render a link to `/signup`, none of which the graph has leaving those screens. The graph does hold those edges, with the right witness (`libs/auth/src/angular/login/login.component.html`), but anchors them at an origin node rather than at a screen, because the component that renders them lives in a workspace library outside the application directory and attribution computed the empty scope for it. So the failure is not a scope that names the wrong screens;

it is a scope the analysis declined to compute, which the running application then shows to be non-empty. Those sites are the *unplaced* class of Table 5.17, and this is the first measurement that any of them has a screen.

The other direction does not support a single figure. 80 predictions run that way: the graph holds the edge and the page showed no link. Three causes are mixed in that count, and two of them are not defects. The first is interface state: realworld's header is wrapped in `@if (authState === 'authenticated')` and offers *New Article*, *Settings* and the profile link only to a signed-in user, and this harness signs in to nothing, so the probe cannot see those links and the absence says nothing about the graph. The second is data: a tag page navigated to with a placeholder segment lists no articles, so it renders none of the article links the graph attributes to it. The third is target over-approximation, which is a property of the destination rather than of the scope. realworld's `[routerLink]="['/profile', article.author.username]"` resolves to `/profile/:username` and to `/profile/:username/favorites`, because a two-segment link with a hole admits the deeper route; the edge is recorded as a may edge at half confidence, which is the graph stating that it does not claim the address.

Only the predictions that name a destination exactly can be refuted by a page not linking there, so those are the ones the oracle judges in this direction. There are 7 of them across the judged screens, against 102 over-approximated, and 0 of the 7 were not rendered. A denominator of 7 supports no rate, and reporting one would be the same error as the negative percentage of Section 5.5. The result in this direction is therefore narrower than in the first: the over-attribution direction was not refuted on this corpus, and the reason is that the graph almost never claims an exact destination at a screen this oracle can observe. The measure is reported as several numbers rather than as a precision figure so that the two directions stay separate.

5.3.2 What the attribution discriminates

The oracle above cannot say whether the scopes are too large, so this subsection asks the question in a form that does not need a running application: how much narrower is the computed attribution than the widest attribution there is? Assigning every navigation site to every screen recovers every construct by construction, needs no analysis, and is useless, because the per-screen query would return the whole site list on every screen. The ratio between the two is a measure of what the analysis contributes, and Table 5.11 reports it.

The analysis is between 1.7 and 18.4 times narrower than that baseline, and the low end of that range is the figure that qualifies RQ3. On k8sdash the average site is attributed to 39.5 of 67 screens, which is 59 % of the application: the per-screen query there excludes about two screens in five and admits the rest. The widest margin is mifos, where 43059 attributions are computed where the baseline would assert 793760. Across the corpus the mean scope runs from 4.8 to 39.5 screens.

Two things follow, and the second is the less favourable to the analysis. The first is that the trivial baseline is not a competitor for the must tier: an unguarded site in a shared component attributed to every screen would yield a must edge from screens

Table 5.11: The computed attribution against assigning every site to every screen. A site is a source construct a witness cites; an edge anchored at a scope is counted over the screens that scope stands for, since counting it at its anchor would understate the analysis's breadth. *Mean scope* is the attributed pairs per site.

Application	Screens	Sites	Attributed pairs	Mean scope	All-screens pairs
realworld	10	39	188	4.82	390
alfresco	26	19	117	6.16	494
ghostfolio	84	189	4104	21.71	15876
k8sdash	67	62	2450	39.52	4154
bitwarden	98	167	1785	10.69	16366
mifos	605	1312	43059	32.82	793760

that never render it, so Claim 3.1 would fail by construction rather than be tested, and the finding that none of the readable must edges attribution added fails the witness check is only meaningful where the attribution is narrow. Of the 133 must edges it added, 62 are unread, all on k8sdash, where the scopes are widest. The second is that on part of this corpus it is not very narrow. A scope that names three fifths of an application is a weak statement about where a site fires, and the coverage RQ3 reports is bought with scopes of that width. Nothing in the evaluation shows those wide scopes to be wrong, since the oracle above could not test the direction in which they would be wrong. Nothing shows them to be right either, and a reader should take the recovered share of Table 5.17 together with the widths in this table rather than on its own.

5.3.3 What each mechanism contributes

The width above says how far the attribution is from doing nothing. It does not say whether the mechanisms that produce it earn their place, and the answer is not the same for all of them. Table 5.12 extracts the corpus at 5 nested levels, each withholding every mechanism above it. The withholding covers the stand-ins of Section 3.3 as well as the implemented relations: a navigation inside a guard, attributed to the routes that name it, and an effect placed at the shell stand in where the prototype implements no relation of the model, so a level below *full* does not carry them either. Without that, the lowest level would not be the ownership baseline it is read as: it would credit direct ownership with sites that only the stand-ins place, and the contribution of the levels above it would be correspondingly understated.

The levels are not evenly spaced, and the step with the largest effect on (site, screen) pairs and on reference recall is not one this thesis adds. Direct ownership places 1354 sites, 75.7 % of what the full analysis places, and containment raises that only to 1619. The shell rule is that step: *rooted* places 1738 sites, 97.2 % of the full figure, and carries the pair count from 2663 to 43027. Everything above that level (outlet descent, which an ancestor-route closure in earlier work also covers, and the member-level call graph that places a service's navigation) raises the site count from 1738 to 1788, 50 sites or 2.8 % of the corpus total: outlet descent adds 14 sites, and

Table 5.12: The corpus extracted at each attribution level. *Sites* counts the source constructs the level places on at least one screen, *pairs* the (site, screen) attributions it asserts. The levels are nested, and the harness refuses to write this table unless each one's pairs are a subset of the next one's. *Screens* counts the screens at an address a link can name, so the catch-all screens of alfresco and mifos are left out of it and of the pairs, and the column falls short of the census's total by those screens.

		sites / (site, screen) pairs				
Application	Screens	owned	nesting	rooted	outlet	full
realworld	10	17 / 26	25 / 48	38 / 178	38 / 187	39 / 188
alfresco	26	11 / 33	14 / 37	14 / 37	14 / 37	19 / 117
ghostfolio	84	111 / 121	129 / 155	170 / 3599	182 / 3712	189 / 4104
k8sdash	67	11 / 12	59 / 275	59 / 275	59 / 2250	62 / 2450
bitwarden	98	124 / 134	149 / 389	154 / 879	156 / 1448	167 / 1785
mifos	605	1080 / 1431	1243 / 1759	1303 / 38059	1303 / 41532	1312 / 43059
All	890	1354 / 1757	1619 / 2663	1738 / 43027	1752 / 49166	1788 / 51703

the full level 36, 2.0 % of the corpus total, which is the call graph, its fallback join and the stand-ins for guards and effects together, all of which only that level carries. The call graph's own share is smaller: against the coarser rule it took the place of, which joined a navigating service over every component that injects its class and which it keeps as the fallback for a member no caller reaches, it places 7 more sites, 0.4 % of the corpus total, and 331 more pairs. Run beside the nested levels rather than among them, that class-level join places 1781 sites, and it differs from the call graph in pairs on 5 of the six applications: 10 pairs only the class join asserts and 341 only the call graph does. The class join reaches a service only from a component that injects it, so a service another service calls, or one bound to an injection token, is placed only by the call graph.

That ordering matters for the comparison with earlier work, because the shell rule is not a mechanism this thesis introduced. Oshima and Kishi assemble their page information from the root component as well as from the components beneath it (Section 2.4), so *rooted*, and not *nesting*, is the level that stands for what they reach; Rima et al.'s specification also closes a route's components over those of its ancestor routes (Section 2.4), which is what *outlet* adds. Both are baselines that reimplement those mechanisms inside this extractor, not runs of either tool, so the comparison measures what the added mechanisms place on the same inputs and does not show that the tools themselves would place fewer. A comparison drawn against containment alone would be a comparison with a method nobody proposed. Against the nearest of them, *outlet*, what this thesis adds is the *full* step alone: 36 sites, 2.0 % of the corpus total.

Outlet descent is the mechanism whose contribution is mostly width: it places 1752 sites against rooted's 1738, the same count on 4 of the six applications and 14 new sites over the corpus, while raising the pair count from 43027 to 49166. A table of counts cannot say whether that width is correct; the next subsection addresses that question.

Table 5.13: Each attribution level against the hand-read structural reference for realworld. *Missing* is a pair the reference has and the level does not; *excess* is a pair the level has and the reference does not. Precision and recall are agreement with that reference, not with what the running application renders.

Level	Correct	Missing	Excess	Precision	Recall
owned	26	162	0	100.0 %	13.8 %
nesting	48	140	0	100.0 %	25.5 %
rooted	178	10	0	100.0 %	94.7 %
outlet	187	1	0	100.0 %	99.5 %
full	188	0	0	100.0 %	100.0 %
class	188	0	9	95.4 %	100.0 %

5.3.4 Agreement with a structural reference

For realworld the screens each navigation site fires from were read off the sources by hand (Section 4.3): 39 sites and 188 (site, screen) pairs. Scoring every level against that gives the comparison no other method here can make, because it has both directions rather than one; Table 5.13 reports it. The reading applies the analysis's own relations by hand, so it tests the implementation of those relations and not whether they describe what the running application renders (Section 4.3). It is also not independent of the graph: its first version was completed with nine header sites the scorer reported from the graph. And it is dominated by the shell: 13 of its 39 sites are placed on every screen and supply 130 of the 188 pairs, which every level from *rooted* upward places trivially, so the pair-level recall of those levels rests mostly on them. Over the other 58 pairs, recall is 82.8 % for *rooted*, 98.3 % for *outlet* and 100.0 % for *full*.

No nested level attributes a pair the reference does not contain. What separates them is how much they find. Direct ownership recovers 13.8 % and containment 25.5 %, both far short. Adding the shell rule, the level that stands for the prior work's reach, recovers 94.7 % at 100.0 % precision, leaving 10 pairs unfound and inventing none. Precision here is against a reference whose header sites were added from the graph's output, so it is weakest as evidence exactly where the shell dominates.

Against *rooted*, everything above that level reduces 10 missing pairs to 0 and asserts no pair outside the reference. That was not true of the analysis before the call graph, and the last row, the class-level join the call graph replaced, shows where it failed: it asserts 9 pairs outside the reference, all at one site, the `navigate` inside `UserService.logout`, taking precision to 95.4 %. The class join attributes a navigating service to every screen whose components inject it. Ten components inject this one, the header among them, and the shell renders the header on every screen, so the site is attributed to every screen the application has; its only caller is a button on the settings screen. That was the first measurement in the thesis to show a computed scope too large rather than merely unverified, and the call graph, which follows the call from that button to the member, places the site on the settings screen alone. The order of these steps limits what the clean row shows. The call graph was written after this ground truth had exposed the defect, and it is scored

on the same ground truth, so the 100.0 % precision is consistent with the repair and is not an independent test of the call graph, whose remaining errors would sit where this application has nothing to test them. The same holds for recall. Until the last corrections to the rule that places an empty-path child on the screen above it (Section 6.3), *full* missed five pairs, all at one screen, `/profile/:username`, whose article list is placed by a child route at the empty path and renders on the parent's address; at the final commit it misses 0. Of the changes between the two runs, only that rule moves a (site, screen) pair; the others change a modality, which this scoring does not read. That rule, too, was corrected with the defect in view, so the clean row is consistent with two repairs and tests neither independently.

Two things follow for RQ3. The first is that most of the recovery attributed to this analysis is produced by mechanisms earlier work already had: 94.7 % of the 188 pairs are reached by the shell rule and 99.5 % once the enclosing routes are added, so the *full* step, the call graph with its fallback join and stand-ins, adds 1 pair on this application, and the *full* level takes recall to 100.0 %. The second is that this is one application of six, the smallest, and the corpus-wide widths of Table 5.11 are not licensed by it. On k8sdash the mean scope is 59 % of the application, and no ground truth exists to say whether that width is earned. What the ground truth establishes is narrower than soundness: the additional mechanisms raise agreement with the structural reference on this application without a pair outside it, after the one scope it found too large and the one screen it found missed had both been repaired with it in view. What it cannot establish is that the same holds on an application of six hundred screens.

Scope attribution therefore raises measured recovery. Its attributed must tier has 0 detected witness-check failures among 150 edges, with 62 unread destinations. This is limited evidence for RQ3's precision requirement: unread edges do not pass the comparison, and target agreement cannot establish execution on every input (Section 5.5). Three qualifications belong with that answer, and all three are visible in the tables. The first is that the great majority of the gain is on the may tier, so what attribution mostly adds is navigation the graph says is possible, not navigation it says is certain. The second is that the scopes are tested in both directions on one application only. The recorded runtime check exposed 4 missing pairs on its earlier graph, all on bitwarden; of the 5 applications it served, only realworld and bitwarden rendered any judged pair. It is not a new check of the final call graph, and cannot establish whether a scope is too large, because at the screens it can observe the graph almost never claims a destination exactly, and because a link a template renders conditionally is absent for reasons that have nothing to do with the scope. The ground truth of Section 5.3.4 does test both directions against the structural reference: of the 188 pairs it records, the analysis asserts none outside it, where the class-level join it replaced asserted 9 at one site, and misses 0. That result is for realworld alone, the smallest application of the six, and both repairs were made after the ground truth had shown the defects. The third is the width reported in Section 5.3.2: the computed scopes are narrower than the trivial attribution by a factor between 1.7 and 18.4, and at the low end of that range a scope names most of the application. A site attributed to too many screens can produce edges whose readable witnesses agree, because the construct is at that location and does name

that target, so no witness-based method can find it. The recorded rendered-interface comparison exposed 4 missing pairs at sites the earlier graph had left unplaced; it does not establish the corresponding error set of the final graph. That the scopes are not too large is not established, and Section 6.2 states it as the most serious limitation of the work.

The same mutants were also run against the baseline graph, which answers a question RQ3 leaves open: whether attribution changes how sensitive the graph is to a source change. The answer has two parts that should be kept apart. The first is the one that was expected. Many mutants cannot be scored against the baseline graph at all, because the construct they change produces no baseline edge: 22 are excluded there against 6 on the attributed graph, and the clearest case is the link-deletion operator on realworld, where every candidate sits in the application shell and none of them produces a baseline edge. Attribution widens the part of the source the graph responds to at all.

The second is the detection rate per scored change, and it needs a comparison the two raw rates below cannot make. Of the mutants each graph can score, the attributed graph detects 67 of 67 (100.0 %), against 50 of 51 on the baseline (98.0 %). Those rates are taken over different mutant sets, so their difference mixes a detection effect with the eligibility effect above. The paired comparison isolates it: of the 51 mutants both graphs can score, identified by operator, file, line and subject, the attributed graph detects 51 (100.0 %) and the baseline 50 (98.0 %). The graphs disagree on 1 of them, a guard removal on realworld's empty-path `editor` child that the baseline run scores and misses, and Section 5.4 shows that this disagreement comes from how the harness matches the guarded screen rather than from the graph. Outside the paired set, 16 mutants are scorable only against the attributed graph and 0 only against the baseline; 0 constructs are mutated in both runs into different things. On this corpus, then, attribution leaves the detection rate per scored change essentially unchanged, and its effect on change sensitivity is the widening of what can be scored at all.

The paired gap has moved twice as the apparatus was corrected. The first measurement of this corpus found that attribution widened the graph's reach without making it more sensitive per construct, an artefact of two defects. One was in the extractor, which did not read a guard carried by a route a wrapper function returns. The other was in the mutation harness, whose retargeting operator required the graph to *add* an edge to the new destination: attribution fans a navigation site over every screen it can fire from, so a retarget to a destination those screens already reach removes the old edges and adds none, and a correct response was scored as a wrong one. With both repaired, the previous run showed a paired gap in favour of attribution: six disagreements, five of them retarget mutants whose operator replaced only the head of the commands array, which the baseline graph did not respond to. With the operator corrected (Section 5.4) the baseline detects those five as well, and the gap is the single disagreement above, the one the two runs had in common.

Table 5.14: Mutation sample. Candidates found, mutants sampled under the per-operator quota, and mutants excluded as confounded with a coverage gap.

Application	Candidates	Sampled	Excluded
realworld	26	25	0
alfresco	25	16	1
bitwarden	221	32	5

5.4 Change sensitivity (RQ4)

Mutation testing requires the application to be extracted again once per mutant, so this part of the evaluation was run on three of the six applications: realworld and alfresco, which carry the earlier mutation figures, and bitwarden, whose routes carry many guards, guard factories among them. Every figure in this section comes from the attributed graph with harness mutate 1.9.1, a fixed seed and a quota of eight mutants per operator. The corrections made to the mutation harness are described in Section 4.3.2.

Table 5.14 shows how the sample was drawn. Over the three applications the harness found 272 candidate mutations, sampled 73 of them under the quota and scored 67. 6 were excluded: for 5 of them no baseline edge enters any screen the guarded route yields, and 1 guards a route that declares neither a component nor children, so it yields no screen for a guard to be recorded on. An excluded mutant carries no information about sensitivity: it acts on a screen no extracted edge enters, which may be a coverage gap or a screen reached only by its address, or on a route that declares no screen. The exclusions are not spread across operators: all 6 are guard removals, so 11 of the 17 sampled guard mutants are scored, and the rule removes exactly the cases in which a removal could not be observed. The guard rate below is therefore conditional on the graph entering the guarded screen. The exclusions are few on the attributed graph but not on the graph without attribution: of the 73 mutants sampled against the graph produced without attribution, 22 are excluded and 51 remain, which is the same coverage gap seen from the mutation side. Where the quota capped an operator, the candidates left unsampled are reported rather than passed over.

Table 5.15 gives the detection rate per operator, and Figure 5.4 shows the same outcomes summed over the applications. The rename operator is set aside first, because its detection is close to guaranteed by construction: a rename mutant is excluded unless the baseline graph already holds a node for the segment it renames, and a screen is identified by its full path (Section 3.1), so a parsed literal path must yield a different node once renamed. It tests node extraction rather than the graph's response to a change in behaviour, and it supplies 22 of the 67 scored mutants. The three other operators are detected in 45 of their 45 scored mutants, and they are not equally informative. A deleted `routerLink` is sampled only on a line a baseline edge cites and is detected when that line stops being cited, which, once the attribute text is gone, fails only if the extractor misreads its own input; the evidence for RQ4 rests mainly on the retargets and the guard removals. The counts are 14 of

Table 5.15: Detection rate per mutation operator on the attributed graph, as detected over scored mutants. A dash means the operator found no candidate in that application.

Operator	realworld	alfresco	bitwarden
Rename a route path	6/6	8/8	8/8
Delete a `routerLink`	6/6	–	8/8
Retarget a `navigate`	8/8	4/4	8/8
Remove a guard	5/5	3/3	3/3
All operators	25/25	15/15	27/27
Detection rate	100.0%	100.0%	100.0%

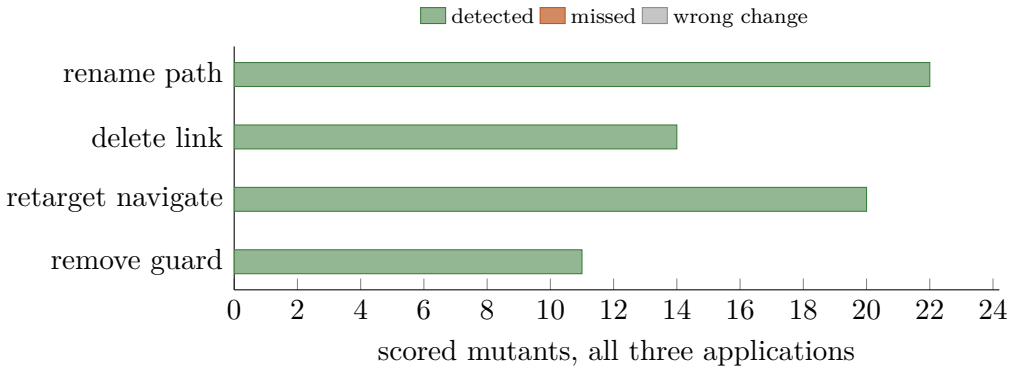


Figure 5.4: Mutation outcomes by operator, summed over realworld, alfresco and bitwarden, on the attributed graph. Excluded mutants are not shown; 6 of the 73 sampled were excluded here against 22 of 73 on the graph without attribution.

14 deleted `routerLink` bindings, 20 of 20 retargeted `navigate` calls and 11 of 11 removed guards. The samples are small, in single digits per operator and application, so the pattern says more than the rate.

Deleting a `routerLink` moved the witness set rather than the edge set in 5 of its mutants, because a second identical link in the same template still produces the edge. That is the behaviour the graph should have, and it is the reason detection is judged on witnesses as well as edges. The operator found no candidates in alfresco, which binds `[routerLink]` instead of writing a literal attribute. Against the graph without attribution the same sample yields only 7 scored mutants, of which 7 are detected: the candidates in the application shells produce no baseline edge and are excluded.

A retargeting mutant replaces the whole commands array of a `navigate` call with a declared route that names no parameter, and is detected when the edited line witnesses an edge to the new destination and none to a destination it reached before. Under the previous version of the harness every retarget the graph did not detect was an artefact of the operator: it replaced only the head of the array, so four mutants composed an address no route declares and one left its target dynamic (Section 4.3.2). The destination is now drawn from the declared routes alone, so both configurations patch each site alike and the paired comparison compares the same edit.

Every guard removal the run can score is detected: 11 of 11, on all three applications, each judged by whether some edge loses a guard the removed list names. A guard is recorded on the edges that enter the guarded screen, so removing it can be observed only where the graph both holds that screen and enters it. All 6 exclusions are that condition failing: for 5 of them the graph holds the screen and no edge enters it, and 1 guards a route that declares no component and no populated children. The harness finds a route's screens by the component it renders, and it now matches a class by the file the routing module imports it from: bitwarden declares two classes named `AccountComponent`, and matching by name alone had counted edges into the user's `/settings/account` as entering the organisation's, so a guard whose screen no edge enters was scored as a miss. Among the scored guards the empty path makes no difference: 3 of the 3 on a route whose own path is the empty string are detected.

One limit of the same matching remains, and it shows in the run without attribution. realworld's `editor` route has two children, at the empty path and at `:slug`, that load the same component file, so the harness cannot tell their screens apart. Without attribution no edge enters `/editor`, but one enters `/editor/:slug`, so the guard on the empty-path child is scored and missed rather than excluded. That is the one paired disagreement, and it is a limit of the harness rather than of the graph.

The mutation run therefore does not confirm the defect class *guards unreliable in both directions* of Section 5.2 on this instrument, and it does not contradict it either: every guard it could observe behaved as the model requires, and the 6 it could not observe carry no information about guard representation. The evidence for that class is what Section 5.5 records at commit f17a302: a guard carried by a route a wrapper function returns, and a `canActivate` element written as a call rather than as a name, neither read by the extractor as it stood then and both read now.

The three applications do not declare their routes in the same way: realworld and alfresco export route arrays, while bitwarden declares its routes inside NgModule routing modules through `RouterModule.forRoot` and `forChild`. No operator fails under either style, so nothing in this experiment distinguishes them.

Of the 67 scored mutants, 67 were detected, and 50 of 51 without attribution; on the 51 mutants both graphs can score, the figures are 51 and 50. In this sample the graph responded to every scored change in the constructs it represents. A construct it does not represent cannot respond, and such mutants are excluded rather than scored, so the rate is conditional on RQ1's coverage.

5.5 What the repairs cost the claim (RQ5)

The defect classes of Section 5.2 are a diagnosis, and the value of a diagnosis lies in the repair it leads to. This section reports what happened when the list was treated as a worklist: every class was taken to its root cause in the source of the application it appeared in, and each cause that turned out to be the extractor's was fixed, with one exception, ghostfolio's spread of computed routes, which the extractor declares unread. The same question was asked of each one, and the method exists to make it

possible to ask: is the navigation unreadable, or was it merely unread?

The tables in this section report the final snapshot used for the main RQ1–RQ3 comparison. Historical measurements appear only where explicitly named to explain a repair. Their extractor or review cohort is recorded with the corresponding artefact, and their counts are not substituted into the final tables. The generated files identify the extractor and harness snapshot that produced each table. Where an oracle shared a blind spot with the extractor, it too was corrected; the harness manifest records those corrections. Reproducing an earlier comparison requires its recorded results or its earlier toolchain, not a run of the final extractor. The final baseline and attributed runs use the same route denominator in Table 5.16. Earlier revisions of that denominator did change; Appendix A records those historical disagreements separately, because a ratio whose denominator moved is not an isolated extractor improvement. Several of the oracle's corrections matter enough to name, because each one had been scoring the extractor against paths the oracle itself had failed to read: a route path written through a constant, an enum or Angular's `$localize`; a first-party import resolved through the workspace's `baseUrl`; and a route array named rather than written out. Before the last of these the oracle saw almost none of k8s dashboard's declared screens, and scored nearly every node the extractor minted there as unmatched. Three more were found in the round reported in Section 5.6 and belong in the same list, all of them in the denominator rather than in the extractor: the oracle had been reading Storybook fixture route arrays as declared screens; it had been reading the route tables of a sibling application reached through a tsconfig alias as this application's; and it had been recording a path held in a route table wrapped in `Object.freeze` as an unresolvable expression, which is the correction that matters most, because the alternative considered first was to drop those routes from the denominator as unreadable. They were readable one call away, and shrinking the denominator would have raised the coverage figure without any improvement in the extractor.

5.5.1 What the losses were

Every class but one resolved to a construct the extractor could read and did not. The exception is ghostfolio's custom `UrlMatcher`, which computes a route's path in a function and which no reader of a route table can resolve, and both tools report it as a blind spot rather than as a route. The runtime method of Section A.5 finds a second route the sources leave undecided, mifos's choice between two route tables by a value injected at deployment, and one construct the extractor could read and does not: ghostfolio's spread of computed routes, behind which 242 of the 251 mounted addresses without a node sit. The causes fall into three groups.

The route table was not reached. An NgModule loaded by `loadChildren` keeps its routes in a separate `*-routing.module` that it imports, so stopping at the module file found nothing and the routing module was then scanned at the project root, where every screen in it was minted at its own relative path. A route array held inline by `RouterModule.forChild([...])` has no variable to find it by, and its elements may name routes (`[ERROR_ROUTE]`) rather than write them. A loader may reference

Table 5.16: Declared screens that have a node on the final extractor, with scope attribution off (Before) and on (After); the repairs of the loss list are in both columns. A blind spot is a route the oracle knows is there and cannot address: a lazy boundary whose route array it could not find, a path expression it could not resolve to a literal, a route whose path is computed by a custom matcher, or a route source spread into a table from a call the parser does not follow, which is the kind that now accounts for most of the column.

Application	Before	After	Declared	Blind spots
realworld	100.0 % (10/10)	100.0 % (10/10)	10	0
alfresco	100.0 % (26/26)	100.0 % (26/26)	26	10
ghostfolio	100.0 % (84/84)	100.0 % (84/84)	84	2
k8s dashboard	100.0 % (67/67)	100.0 % (67/67)	67	0
bitwarden	100.0 % (98/98)	100.0 % (98/98)	98	1
mifos	100.0 % (605/605)	100.0 % (605/605)	605	1

its module directly (`loadChildren: () => VaultModule`) instead of importing it dynamically, and a barrel may re-export the route array rather than import it. A wrapper such as mifos's `Route.withShell([...])` is a call, not an object, and the array it is given was skipped whole.

The path was named, not written. `path: internalRoutes.account.path` is a compile-time string, and so is an enum member and a `$localize` segment with its metadata block. Following the name needs the workspace's `paths` aliases, since the table commonly lives in a library outside the application directory, and it needs the destructuring a route file reaches it through.

The destination was named, not written. A target may be a field, a dotted chain into the route table, an element of a commands array, the return of a builder, or a value in a lookup the component fills. Resolving these is what separates a link from a dead end, with one qualification that cost a soundness violation before it was added: a field seeded with a literal is mutable state, not a constant, and following it turns a run-time value into a must edge.

Node extraction is not the bottleneck it was when this work began. At an earlier snapshot the extractor completed on one application and left four of the six at coverage rates below 20%. The repairs above brought every one to completion; the shift matters, because an analysis that decides where a navigation can fire depends on having a node for the screen it fires from, and a construct whose source screen has no node is a loss attribution cannot repair. That is the bind the lowest-coverage applications were in, and the threshold was crossed by resolving the constructs the route table was reached through rather than by inferring nodes the framework definition does not supply. The gaps that remain are of two kinds, and both are declared rather than inferred. The custom matcher and the deployment-time choice between tables depend on information a static reader of the sources does not have. The spread of computed routes does not: its routes are in the source, and the extractor does not yet read them, so it is an unsupported construct the extractor could close.

Table 5.17 gives the construct accounting, and it is reported in four parts rather

than as one ratio, because a single number cannot separate three different things. A site is *recovered* when it becomes a transition with a destination. It is *dynamic* when the destination is one only the run time knows, such as a value from the backend or a history entry; the graph then records the transition as an edge into that screen's `u_` sink, with modality unknown, rather than dropping it or guessing. It is *unplaced* when the destination is readable but the screen it fires from is not, in which case the edge is anchored at an `o_` origin node that says so. What is left carries a recorded reason but no transition; the check enforces that a reason is recorded, not that it is correct. 43 sites leave the denominator entirely, on three grounds, none of which is a transition between screens: 35 pass an empty commands array and so change the query of the route they are already on, 7 navigate a secondary outlet, which the model represents as a second mount (Section 3.2.3) and the extractor does not implement, so no screen is minted for it, and 1 names no destination at all. All three are decided from the construct's own text before the graph is consulted, so the set is the same in both runs; the generator refuses to write these figures if the grounds do not sum to the total. Both runs are measured by the same ledger with the same exclusions, so the two columns of Table 5.6 are ratios over the same 1775 sites. That was not so for most of this work. The grounds were tested after the graph had been consulted, so a site attribution gave an edge to escaped the ground its own text had put it in, and the after column was a ratio over more sites than the before column. Appendix A.2 says a coverage figure should never be allowed to do this. The comparison now refuses to print at all if any construct changes its ground between the two runs.

The restriction is enforced on the route side as well. A route that declares a named `outlet` is reached as `(name:path)` and never at the primary address, so it contributes no screen; before that was enforced it could take the primary address from the sibling that really renders there, by being declared first, and the graph named the wrong component at a real address without saying so. bitwarden declares such routes: a login screen and a registration screen each pair a primary component with a second one on a named outlet at the same empty path. The repair is therefore not hypothetical, though it moves no count there because the primary sibling already held the address. The restriction reaches a named outlet's *subtree* as well, which it did not until the runtime check of Section A.5 showed what the omission cost. alfresco mounts its viewer as `{ outlet: 'viewer', children: [...] }`, and the routes beneath it were minted at the primary address, `/view/:nodeId` and `/search-libraries/view/:nodeId`, which Angular reaches only as `(viewer:...)`: both tools read the subtree the same way, so the coverage figure could not expose it, and a navigation to either address in the built application is answered by the catch-all. An outlet is inherited, so nothing below such a route has a primary address either, and neither tool mints one now.

Of 1775 in-scope sites, 78.9 % are recovered, 15.0 % are accounted as dynamic and 4.9 % as unplaced, so 98.8 % are accounted for in all, against 71.3 % in the final baseline with attribution disabled. This comparison holds the extractor snapshot and in-scope population fixed. k8s dashboard illustrates why recovered and dynamic must be reported separately: a low recovered fraction can coexist with many sites whose destination depends on runtime data. The dynamic sites do not come from

Table 5.17: Construct accounting after the loss list was worked. Percentages are of the in-scope sites.

Application	Sites	Excl.	In scope	Recovered	Dynamic	Unplaced
realworld	40	1	39	100.0 %	0.0 %	0.0 %
alfresco	34	7	27	25.9 %	25.9 %	44.4 %
ghostfolio	198	12	186	47.8 %	43.5 %	7.5 %
k8s dashboard	72	4	68	16.2 %	72.1 %	8.8 %
bitwarden	131	19	112	62.5 %	15.2 %	18.8 %
mifos	1343	0	1343	88.2 %	8.3 %	2.5 %
All	1818	43	1775	78.9 %	15.0 %	4.9 %

Table 5.18: Final-snapshot witness-check outcomes and must conversion after the repairs. The must column counts edges whose destination was read and passed; the rest are unread destinations, neither passes nor failures, and no must edge failed. “Not applicable” means the application emits no must edge.

Application	Edges	Sites cited	Judged	Clean of judged	Must read and passed	Must conversion
realworld	65	105	65	92.3 %	9/9	7.7 %
alfresco	55	79	23	43.5 %	n/a	0.0 %
ghostfolio	403	685	20	35.0 %	n/a	0.0 %
k8s dashboard	309	441	71	97.2 %	63/125	2.9 %
bitwarden	529	1282	201	59.2 %	1/1	0.9 %
mifos	10811	14300	1076	78.0 %	15/15	1.4 %
All	12172	16892	1456	75.8 %	88/150	1.4 %

one central builder. They are spread over 43 files, 38 of which contribute a single site, so the column reports a habit of the codebase rather than one place to fix.

Two cautions belong with the corpus total. mifos supplies 75.7 % of the in-scope sites on its own, so the corpus figures are close to mifos’s: without it, recovered is 50.0 % and accounted 97.9 %. Historical changes to the route denominator are a separate concern and are recorded in Appendix A; they do not supply a different denominator to this final attribution comparison.

5.5.2 What it cost the soundness property

A coverage gain must be assessed together with the evidence for the emitted edges. Table 5.18 reports the final snapshot’s witness check beside RQ1’s conversion ratio. An edge without a detected failure is not necessarily a pass: the checker can leave its cited destination unread, and the corpus total of those edges is 62 of 150.

The correction sequence explains how the final rules were reached. An earlier extractor, commit f17a302, published 1024 must edges, with 26 detected witness-check failures. Those figures are read from that commit’s own generated results rather than measured with the final tool. The later source review examined a frozen intermediate

cohort of 20 edges, not the final tier. Its recorded fates are retained to explain the fourth correction. Neither historical count supplies a row of the final-snapshot table. Three initial repairs are relevant to that sequence.

The first is a guard the walk never reached. One application routes every feature table through a wrapper function that returns a route carrying an authentication guard, and the guard set of a screen was assembled from the route chain the walk descends; a route a function *returns* is not on that chain, so the guard went unread and everything below it was published as unconditional.

The second is smaller in the source and wider in effect. A `canActivate` element that is a *call* rather than a name was not recognised, and an unrecognised element was skipped, so a route guarded only by a guard factory read as a route with no guard at all. bitwarden guards its root with `redirectGuard()`, so the deterministic edges that point at that root read as unconditional. The rule that replaces it is the one that generalises beyond this corpus: an element the reader cannot name gates the edge anyway, because refusing to read a guard must never be the reason an edge is called deterministic.

The third was invisible until the check itself was corrected. The extractor files the second and later constructs that yield an identical edge as further witnesses of it, and the witness check opened only the first, so the tier was being judged on part of the evidence it rests on; over the current graphs 27.9 % of the sites the edges cite are such later citations, and 58.7 % in the application where the share is largest. Judging every cited site found edges in bitwarden whose later citations targeted `/vault`. The running router places a component there, through a route a helper produces, and neither redirects nor falls through to its catch-all (Section A.5). The extractor of that round did not read that route, matched the literal to nothing it had read, and fell back to the catch-all's redirect to the root, so it held the site at may; it then merged it into a must edge the root already had, and cited it as evidence for it. The address the edge led to was the extractor's fallback, not the application's behaviour. The round answered this by giving an edge the weakest modality over every construct that cites it. All of them moved to the may tier, which is why the witness check found no failure in that round's tier and did not pass the tier of the round before it. That rule did not survive the next correction, for a reason given below.

The fourth was not found by the witness check, because the witness check cannot find it. It compares the destination a cited construct names with the destination its edge names, and never asks whether the construct runs every time its code does, nor whether the router lets the navigation through (the half of Claim 3.1 that Section 3.1 assigns to the demotion rules). A review of the round's tier read every construct its 20 edges cite in the source, 30 distinct constructs cited 36 times, and red-team panels then attacked the corrected rules; the review and the panels were made with the assistant the declaration at the front of this thesis describes, and every defect they reported was reproduced against the code before it was repaired. Three defects explain the round's tier, and the panels found a fourth in the repair of the second.

The first is that the demotion rules were incomplete. They demoted a site for a guard on its target, a cap from attribution, a catch-all route or a target with holes, and for nothing else. A link under `*ngIf`, the case Section 3.1 names as demoted, was

emitted as must, and so was a link inside an `@if` block and a call under a branch, in a callback, in a `catch` clause, or after an `await` or an early `return`. Of the 36 citations, 32 run under such a condition: 10 under a branch, 9 under a template condition, realworld's own `*ifAuthenticated` directive among them, 4 after an `await`, 3 in a callback, 3 after an early exit, 1 in a `catch` clause and 2 before code that may start a navigation that cancels them; a construct under more than one condition is filed under the first the reader finds. The extractor now demotes a site under every condition Section 3.1 lists, which the panels extended to a member no handler calls unconditionally, a component mounted under a template condition, an ICU case, and an address two declared routes can answer, and the last review to any code after a navigation, in a later branch, a member, a service or a `finally` block, that may start another before the first has finished. The second is that the guard the second correction names was still unread. bitwarden's `redirectGuard()` does not sit on the route that renders the root: it sits on a child of that route with an empty path and no component, which Angular matches on every navigation to the root, and the guard set of a screen was assembled from the routes that render it. A guard on such a child now gates the screen above it, as do `canMatch`, `canActivateChild` and `canLoad`, which were not read at all, and 10 edges, every one of bitwarden's, are guarded; each construct they cite also runs under a condition, so either correction alone moves them. The repair first gated an edge on every guard its destination carries, and a panel showed that this demotes navigations the router lets through untouched: Angular does not run the `canActivate` or the resolvers of a route a navigation keeps again, so a navigation between two children of a parent guarded by `canActivate` runs the children's guards and not the parent's, and gating it on the parent's treats every navigation as an entry from outside the application. The extractor now computes, for each source and destination, the guards the router runs: `canDeactivate` on the levels the navigation leaves, `canMatch` on every level it matches, `canActivate`, `canLoad` and the resolvers on the levels it enters, and `canActivateChild` on the levels above an entered one, where a level is kept when source and destination pass through the same route object, the level has no parameter the navigation could change, and it does not ask to run its guards on every navigation. A resolver the navigation awaits holds the edge at may, as a guard does, and a guard list the reader cannot name still gates. The third is a navigation to the address its source already displays. Angular's default configuration ignores it, and where the address differs only in a parameter or the query it runs only on some inputs, so it is not a navigation that occurs on every execution. A shell or anchor edge whose destination is one of the screens it stands for is now drawn per screen, must from the others and a may self-loop at the destination, rather than demoted whole, which a panel showed discards transitions that do occur.

Table A.3 in Appendix A.7 records, construct by construct, the condition the extractor now reads. It is produced with the extractor's own readers, so it records the extractor's decisions rather than corroborating them. What stands in for corroboration of the condition rules is a recorded review: 2 readings of the 30 constructs, made separately by the assistant the declaration describes, from the sources and without the extractor's readers or results, kept in `results-must/review.json`. The extractor agrees with the less favourable of them at 30 of the 30 constructs, and

the two readings agree with each other at 30 of them. The two readings are two runs of one model, not independent raters, so their agreement measures consistency and not correctness, and the cohort they read is the round's tier, not the final one. Three further things limit what that agreement shows. Each reader was given the condition rules of this chapter as the definition to apply, so the readings test whether the extractor applies its rules as written and not whether the rules are the right ones. They were made after those rules had been corrected on these same constructs. And they judge only whether a construct runs every time its code does, and its destination: not the guards and self-loops that decide 10 and 1 of the 20 edges in the frozen review cohort.

The edges that survive show why the third correction's rule had to go. realworld's tag page renders the home component, whose Global Feed link to the root runs whenever the page does. Two other links reach the root from the same page, and they do not run whenever the page does: one is under the application's `*ifAuthenticated` directive, and one is inside the `@empty` block of an article list. The weakest-modality rule demoted the edge for them, although the navigation the first construct makes occurs on every execution of its template, which is all Claim 3.1 asks of an edge: the claim is that a must edge corresponds to *a* navigation that occurs, not that every construct reaching the same screen does. The rule had been written for a different question, which construct an edge's certainty may be credited to, and it answered that question by discarding a certainty the edge had. An edge now takes the modality of its most certain construct, which becomes its witness, and every other construct it cites records the modality it earns on its own; the witness check judges the must tier only on the constructs that earn must. The `/vault` citations of the third correction are recorded as may under that rule, which is what they are evidence for. Of the round's 20 edges, 17 have every construct they cite under a condition, 10 of them also guarded; taking guard first, then self-loop, then condition, 10 are guarded, 1 is a self-loop, 6 are conditional, 2 keep a construct that earns must (the Global Feed link, and the logo links of realworld's header and footer, which the shell renders on every screen), and 1 is joined: its own construct earns may, and the edge stays must because the shell's logo links, now drawn per screen, yield it too.

With these corrections the deterministic tier of the corpus is 150 edges with attribution and 17 without it, resting on 25 constructs with attribution and 21 without, in 4 applications; 2 of the 6 have no must edge. It is not the round's tier made smaller. Reading guards as the router runs them lets through the navigations between screens that share a guarded parent, and that is where most of the tier now is. realworld holds 9 edges, its logo links and its Global Feed link into the root. k8sdash holds 125: the logo link and the Create button of the toolbar its chrome component renders above every screen, drawn from every screen under that component except the root, which a redirect answers, the destination itself, and the screens whose `canDeactivate` guard runs when the user leaves them. mifos holds 15, cancel and back buttons that return from a form to the section that opened it and the menu of its system page, and bitwarden 1, a cancel button. The edge count is therefore a poor measure of how much evidence the tier holds: 2 constructs yield 125 of the 150 edges, one per screen the toolbar renders on, and without k8sdash the tier is 25 edges on 23 constructs in 3 applications, which is why RQ1 is

answered in constructs. Those 125 edges are also only as right as the scope of the chrome component: each source is a screen attribution places the toolbar on, and an over-wide scope there would add must edges from screens that do not render it. Over-wide scopes are not tested outside realworld (Section 5.3.1), so the source side of these edges rests on the scope rather than on a check. The must edges also rest on an assumption about the order in which route tables registered apart are matched (Section 6.3): 126 of them, in 2 applications (k8sdash, bitwarden). Without it the tier holds 24 edges on 22 constructs. Each of the 25 constructs was read against the source for the condition around it, the address Angular resolves it to (for a relative link, the route's consumed segments less one per `../`), and the guards and resolvers its navigation runs. The readings are kept in `results-must/tier-reading.json`; the census refuses to report a must construct that file does not list. Unlike the review of the round's tier, this reading was made once and with the extractor's decisions in view, so it is a check that each decision can be defended from the source, not an independent second reading. The witness check finds 0 failures in the 150 must edges and records 62 edges whose cited destination it cannot read. The unread edges are not passes. All 62 of them are k8s dashboard toolbar edges, the same edges whose source side rests on the chrome component's scope and on the route-order assumption, so these weaknesses fall on one set of edges and are not independent; outside k8s dashboard every must edge was read and passed. The source reading above and the mechanical check are distinct evidence: the reading examines every construct the census reports, with the extractor's decision visible, whereas the checker resolves only the source forms it supports. Neither is an independent proof of Claim 3.1.

The tier is easy to misread as a property of the applications. The extractor never evaluates a guard, so a guard that can refuse a navigation demotes it whatever the guard would decide. Extracting the corpus again with every guard taken to let the navigation through, and every other rule kept, gives 194 must edges in 6 applications instead of 150, resting on 69 constructs: 44 of them run a guard, and for 40 of those every guard the navigation runs is one the source screen's own routes carry by name, read off the route configuration, so a user on that screen has passed it at least once on the way there. 29 of the guarded edges are ghostfolio's, which names its authentication guard on each route rather than once on a parent, so that a navigation between two signed-in screens runs it again. The small tier therefore says what the extractor can assert without evaluating guards. Evaluating them would move at most 44 edges from may to must, those on which the navigation runs a guard. A reader who expected static extraction to recover a large deterministic graph should take this as part of the answer: some of the transitions are there, behind guards a static reader does not evaluate.

The consequence for RQ1 is reported on the final snapshot rather than inferred from a historical conversion rate. The constructs that earn must on their own number 25 with attribution and 21 without. Historical instruments counted guarded or conditional constructs as deterministic and sometimes counted every citation of a must edge as deterministic evidence. The final rule requires each counted construct to earn must itself.

Claim 3.1 ranges on this corpus over 150 final must edges. The source review and

the mechanical check provide different evidence: 62 edges have a cited destination the mechanical checker cannot read, while the source review examines the constructs the census lists with the extractor's decisions in view. The implementation's demotion rules are also exercised by a mutation sweep (173 mutants at `920e1aa`), in which every deletion fails some test. That was not so on the sweep's first run in the final revision, at `c8eec9b`: 20 of its 164 mutants survived, all of them rules the router fixes had added without a test. Tests now cover 18 of them. The other 2 were retired as equivalent: each deletes a condition already implied, by the shape of a route table or by another check of the extractor, so no input can tell the rule from its absence. Writing those tests also exposed a defect, a relative link that climbs above the root (Section 6.3). The sweep measures whether the tests detect deletions of the rules, not whether the rules are a complete account of runtime navigation. These checks do not constitute a proof of soundness.

Read over the whole graph rather than the must tier, the figures are counted per cited source location and not per edge, because that is what the check opens: the 12172 edges of the round's corpus graphs cite 16892 locations between them, an edge citing several wherever several constructs yield it. Of those locations only 6.5 % are clean, because 88.2 % of them are over-approximations for which the check claims no target agreement by design. That is reported beside the must column because the must tier is still a small slice of the graph, 150 edges of 12172, and a precision figure for a slice is not one for the graph. Nor is it a defect rate: on the 1456 locations the check does judge (every one that is neither an over-approximation nor among the 535 it cannot read), 75.8 % are clean. The three figures answer three different questions. The first is the one that bears on Claim 3.1, and it bears on it in one direction: a must edge whose cited site named another destination would refute the claim, while the claim's quantification over inputs is carried by the demotion rules and not by this check (Section 3.1). The unit is kept explicit, since locations and edges are not counts of the same thing.

One caveat belongs with the 88.2 % figure, because "by design" reads as a property of the method and most of it is a property of one line. Of the 12172 edges in the corpus, 44.8 % (5454 edges) come from a single call site in mifos, `view-transaction.component.ts:135`, which navigates to a template literal whose static prefix is empty:

```
this.router.navigateByUrl(`${accountUrl}/actions/${name}`)
```

With nothing before the first hole there is no prefix to narrow on, so the site over-approximates to every declared route in an application that declares six hundred of them. mifos supplies 88.8 % of the corpus's edges on its own, and that one line supplies half of those. The whole-graph figure therefore does not show that the method over-approximates at this rate. It shows that an unbounded fan-out is cheap to emit and expensive to report, and that an edge count is the wrong unit for anything the thesis claims. It is also an argument for a prefix floor: a site whose static prefix is empty tells the analysis nothing, and the model would lose nothing by recording it as a dynamic sink instead of as 5454 may edges.

Table 5.19: The route configuration the framework holds, against the extractor's node set, as recorded by the runtime probe of 29 and 30 October (Table 4.4). *Routes* counts the route entries the configuration holds once every lazy subtree is expanded, and *mounted* the addresses among them at which the router places a component on the primary outlet. *Not mounted* counts nodes the extractor declares at an address the framework mounts nothing at. *Not minted* counts addresses the framework mounts and the extractor has no node for, and *prefix only* those further addresses the framework's configuration holds where a component is placed but no screen of the address's own is: in 1 the route is a prefix with no empty-path child, and in 2 its empty-path child is a redirect, and in neither case is the extractor wrong to mint no screen there. The column is reported so that *mounted* less *nodes* is accounted for in full. *Incomplete* is the extractor's own statement, from its soundiness notes, that part of the route set was beyond it.

Application	Routes	Mounted	Nodes	Not mounted	Not minted	Prefix only	Matcher only	Declared incomplete
realworld	14	10	10	0	0	0	0	no
alfresco	123	30	27	0	3	0	0	yes
ghostfolio	377	326	84	0	242	0	1	yes
k8s dashboard	140	67	67	0	0	0	0	no
bitwarden	170	99	98	0	1	0	0	yes
mifos	935	614	606	0	5	3	0	yes
All	1759	1146	892	0	251	3	1	

RQ1's conversion ratio and the recovered column of Table 5.17 answer different questions. Conversion counts constructs whose own witnesses earn must; recovered counts sites witnessed by an edge of any tier. In the final snapshot 2 of the six applications have no must edge, and 25 constructs earn must across the corpus. The broader recovery measure therefore must not be read as deterministic recall.

5.5.3 What is minted at an address the application does not mount

Across the corpus the extractor declares 0 screens that the framework mounts nothing at. Over 6 applications, 1759 route entries and 892 nodes, every address the extractor claims is an address the framework's own configuration mounts. This is external evidence that no node is minted at an address the application does not mount, which is a precondition of the must-tier property rather than part of it; since an address behind a guard that always refuses still counts as mounted, the probe could not expose such a node either. The two static readers could not produce this evidence, because an address both of them place at the wrong prefix is an address they agree on.

That evidence is bounded by what a census keyed on addresses can represent, and the bound should be stated rather than inferred from the table. An address is counted as mounted when the configuration names it as a literal pattern and places a component there on the primary outlet. 65 route entries in the corpus place a component and are counted in neither column, because they have no such pattern: 1 decides

Table 5.20: Every route entry of Table 5.19, in the one bucket it falls in. *One address* counts entries that a surviving entry at the same address already answers for. *Named outlet* and *Matcher* are the two exclusions that drop a component the application places (64 and 1 entries, 65 in all), and the rest place no component at any address. The generator refuses to write this table or Table 5.19 unless the buckets sum to the route total and every exclusion that drops a component-bearing entry is named here.

Application	Routes	Mounted	One address	Redirect	Named outlet	Empty outlet	Matcher	No component
realworld	14	10	1	0	0	0	0	3
alfresco	123	30	3	2	66	0	0	22
ghostfolio	377	326	9	1	0	0	1	40
k8s dashboard	140	67	0	3	29	0	0	41
bitwarden	170	99	7	9	13	0	0	42
mifos	935	614	17	32	0	0	0	272
All	1759	1146	37	47	108	0	1	420

its address through a `UrlMatcher`, and 64 sit on a named outlet, where the address is `(name:path)` and not a path. Table 5.20 shows every route entry in the bucket it falls in, and the generator refuses to write either table unless the buckets sum to the route total and unless every exclusion that drops a component-bearing entry is named, so a future exclusion cannot quietly swallow a screen. The matcher case is the more important of the two: a navigation to `/admin/users` in the running application is answered by that component, so the mounted column is a lower bound by at least one address, and how many addresses the matcher claims is not published here, because reading its body by hand is not a measurement. The outlet entries were then put to the running applications, because an exclusion that drops a component should be shown to drop something no address reaches rather than argued to. Navigating the primary address each of them sits under (`/personal-files` in alfresco; `/namespace`, `/event` and `/clusterrole` in k8s dashboard; `/login`, `/signup` and `/login-with-device` in bitwarden) is recognised as the primary screen in every case. The observer reads the primary outlet's chain only, so whether an outlet component is also activated at those addresses is not measured. Addressing the outlets directly in Angular's own compound form, `/personal-files(viewer:view/abc123)`, `/namespace(actionbar:)`, `/login(secondary:)` and `/login(environment-selector:)`, produced no recognition event at all: the router stayed at the address it held before the attempt, which in all four was a sign-in screen. That shows the four compound addresses unmatched from a signed-out session, not that no address reaches an outlet, and the measurement does not settle how the applications reach them. The exclusion therefore rests on the ledger, not on the probe: the alfresco viewer's entries are the ones Section A.6 already declares outside the node set, and the rest are outside it for the same reason, a region filled by a navigation that names the outlet rather than by a primary address. Those navigations are the construct the ledger excludes as addressing a secondary outlet, so the two exclusions are the same fact seen from the two ends.

Two further limits belong with the 0 figure, because it is the strongest claim in this chapter and should not be read as more than it is. The first is that "mounted"

is a property of the configuration and not of what a user sees: some addresses the extractor mints hold a component that renders only application chrome, and the address's own screen is reached by a redirect or by an empty-path child. The probe measures two such addresses directly: in k8s dashboard the root redirects, and in bitwarden the root places chrome and no screen of its own. The same shape elsewhere in the corpus is counted as a loss for one application and as agreement for another, decided by what the extractor happened to mint there. The second is that where two routes sit at one address and a `canMatch` guard chooses between them, no static reader can say which answers: bitwarden's `/vault` is such a pair, the census keeps the first-declared component and the running router activates the other. Neither limit moves a published number, because the census counts addresses and both members of that pair are leaves. Both are reasons to read Table 5.19 as a statement about the route configuration rather than about the rendered interface.

It was not so when the measurement was first taken. The comparison then returned two addresses in alfresco, `/view/:nodeId` and `/search-libraries/view/:nodeId`, and they were a blind spot the two static readers held in common: both assembled the routes beneath a named outlet against the primary path, both held both addresses, node coverage was 100 % over a set that contained them, and no comparison between the two could expose it. Section A.6 had declared the construct as an open case, and declaring it is not the same as being sound over it: a consumer reading the graph finds a node at `/view/:nodeId`, and a navigation to that address in the built application is answered by the catch-all route. An outlet is inherited, so nothing beneath a named-outlet route has a primary address either; the extractor now carries the outlet down the walk and mints no screen for such a subtree, which is a loss and no longer a claim. The named-outlet limitation stands as a limitation on what the graph contains, and Table 5.19 is the measurement that moved it out of the other column.

The check also found one case nobody had declared. Against the route set as the static readers held it before this round (the independent parser's output, which is the one of the two whose per-application sets are committed), the same comparison returns 10 addresses in alfresco, bitwarden: the two in alfresco above, and eight more in bitwarden. Seven of those eight are the report screens and the eighth is a member screen. Both readers placed them at the application root (`/breach-report`, `/sponsored-families`), where the running router mounts the first at `/reports/breach-report` and the second three segments deeper, under `/organizations/:organizationId/members/`. The cause was the same in both readers and had been arrived at independently in each: a lazy route names a module through a barrel, the barrel re-exports a feature module, and the feature module imports the routing module that holds the table. Each reader followed one hop from the module the route named, found no table, and left the file to the scan that reads unclaimed tables at the root. Neither could have caught the other, because both were wrong in the same place, and the node coverage both reported was 100 % against a denominator that carried the same eight wrong addresses. Both now follow the chain to the table, bounded at three hops and refusing at every hop a file the route itself sits in, a file another route loads, and a file whose every table is registered at the application root.

5.6 Dead links, found from the source

A site whose target the application mounts nothing at is a dead link, and the method finds candidates for such links at no extra cost. The candidates are found statically; each is then checked against the route configuration the runtime probe recorded before it is reported. The clearest case is mifos. A transactions tab links to a journal entry with

```
[routerLink]="['/', 'journal-entry',
'view', 'S' + transaction.id]"
```

and the application does declare that screen, at `/accounting/journal-entry/view/:id`, inside the accounting module. The link omits the parent prefix, so the URL it builds is one no route matches. mifos declares `{ path: '**', component: NotFoundComponent }`, so the link does not fail invisibly: it renders the not-found screen, which a user can mistake for missing data rather than a broken link. The same shape repeats across the application. Links to `/loans-accounts/\ldots` name a subtree the application declares only under an owner (`/clients/:clientId/` and `/groups/:groupId/`) and never at the root, and links to `/savings-accounts/\ldots` one declared only under `/centers/:centerId/`, `/clients/:clientId/` and `/groups/:groupId/`. Four calls name segments the application mounts under other names: `/users/create` (mounted as `appusers`), `/system/scheduler-jobs` twice (`manage-jobs`) and `/system/global-configurations` (`configurations`), and three further calls of the same shape name two more addresses: `/organization/users/create`, from two different create-user components, and `/organization/data-tables/create`. A test that never opens the tabs holding these links would not find them, and a crawler would find them only if it reached the screens that hold them; reading the route configuration finds them without running anything.

The list is not one defect class, and the split matters for what each part is worth. 18 of the candidates are an absolute link written against a path the application does not mount, usually the name the server uses rather than the one the router does; those are the application's. 5 come from a *relative* `routerLink`, such as `[routerLink]="'edit'"` or `[routerLink]="['../../transactions']"`, where the target depends on resolving the link against the screen it fires from, and getting that base wrong is this analysis's gap before it is the application's defect. 4 are neither, because the result files do not record enough of the construct to say which it is; they are counted and not classified, rather than assigned to the half that would flatter the finding. The last two groups are reported as candidates and weighted lower. Counted as places in the source rather than as targets the three groups are 18, 3 and 4, and the difference is one relative link in a shared template that resolves against three owning screens, so a single mistake yields three candidates at one place in the source.

A further 69 sites are not published at all. The reason is the boundary of the measurement, not the way the sites were read: they sit outside the directory the measurement is scoped to, in ../../libs, ../aca-shared, ../browser, in the repositories of 2 applications, and their files enter the analysis only because those repositories declare

a `paths` alias for them. Most are the source of a *sibling front end*, a browser extension built from the same monorepo, whose own router declares the same addresses a census would have called dead. An accusation needs two things: that the application mounts nothing at the target, and that the link is reached in the application measured. For a file belonging to another front end the second is not merely unestablished but false, so the route set these sites would be judged against is the wrong one, and publishing them would report one application's valid links as another's defects. They are counted here and left out of the census.

Chapter 6

Discussion

6.1 Key findings

Static extraction of a transition graph from Angular source fails in two different places, and the first of them turned out to be the tool's rather than the method's. Node extraction is complete on all six applications of the corpus against the readers of the route configuration (Table 5.2): every screen those readers find declared has a node, and the runtime method observes no node placed at an address where the framework mounts nothing, which shows that no such error was observed, not that none can occur. Completeness in that sense is bounded by what a route table can be read to declare, and the runtime method is also the method that bounds it: the route configuration it reads holds 251 mounted addresses with no node, 242 of them in ghostfolio alone, so 2 of the six are complete in the stronger sense of holding a node for every mounted address in the runtime configuration (Section A.5). Both readings are reported, because the weaker one is what the loss classification of this chapter is computed against and the stronger one is what a user of the graph would want. That was not so when this work began. Two defects of the extractor accounted for most of the corpus then: a route whose path is a constant reference, and a route table behind a lazy boundary it did not cross. Each produced no node at all, so that every navigation site belonging to such a screen was lost together with every site pointing at it. Section 5.5 reports what repairing them took; on the final instrument used for the main RQ1–RQ3 results, that class of loss is empty.

What remains is lost at the edge, and it is mostly the same everywhere: the navigation sites exist in the source, the screens they would connect exist in the graph, and the extractor cannot decide which screens a site belongs to. The exceptions are the sites whose target screen has no node, most of them in ghostfolio (Table 5.9). That is the loss class scope attribution addresses, and it is the largest in every application of the corpus (Table 5.4). The structural relation between the two levels survives the repair. Attribution still cannot draw an edge into a screen that has no node, which is why the chapter reports the two levels separately. The ranking between them, however, is a fact about an extractor and not a limit of the approach.

The two levels are not independent. Attribution cannot draw an edge into a screen that has no node, so node extraction bounds what attribution can achieve. A complete set of nodes, in turn, yields few transitions while the navigation sites that

live outside the component a route renders stay unattributed, which is the state the baseline of Table 5.4 is in. An evaluation that reported only one of the two would have described a tool that works better than it does, which is why Chapter 5 reports node coverage beside the conversion ratio instead of the ratio alone.

The ranking of the defect classes used to depend on which unit the corpus is counted in, and it no longer does. Attribution is the largest class both per application and over the corpus total (Table 5.4); on the extractor this evaluation began with, node defects dominated the total because the largest application contributed most of the constructs and lost them at the node level. The earlier disagreement between the two views shows a general hazard: one large application can dominate anything that is aggregated, so the per-application view says what a developer would hit on a given codebase and the corpus view says where engineering effort pays off across codebases. Reporting only the total would have hidden the attribution problem that most applications show, and reporting only the per-application ranking would have hidden the weight of the largest.

The answer to RQ3 has to be read against what the loss rules can and cannot delimit. Because a lost construct is assigned to the first class that applies (Table 4.2), a construct inside a shared component that only appears in screens without nodes is counted as an attribution loss, although attribution alone cannot recover it. The measurement shows the converse only in a small way: of the conversions attribution made, 292 came out of a construct the rules had already assigned to an attribution class, none out of a node class, and two out of the residual class to which no rule applied (Table 5.8). The misclassification therefore runs mostly one way on this instrument: the class can hold a construct attribution alone cannot recover, and only two constructs attribution recovered were classified elsewhere, neither in a node class. The class is thus a diagnostic partition of the losses and not a bound on what attribution recovers: it can hold constructs attribution cannot recover, and the recovery is not contained in it. On realworld attribution recovered every one of its attribution losses, must tier and may tier together (Table 5.6), and Table 5.9 records no remaining loss there under any heading. The attribution-class losses that remain in the other applications are accounted for by Table 5.9, and three of its reasons are not attribution problems at all: the target screen has no node, the owning component is never reached from any screen, or the site sits in a service or a guard by a receiver or provider form the call graph does not resolve. No construct that the baseline converted was lost (0 leave the converted classes), so no conversion of the baseline was lost. The gain does widen the must tier: after the fourth correction of Section 5.5 that tier holds 17 edges without attribution and 150 with it, the added edges being the links of realworld's and k8sdash's shells placed on every screen the shell renders. On the instrument before that correction every must edge of the round's tier passed the witness check (20/20 after attribution, as the round's tables printed it), a tier that held none of k8sdash's edges, so that run says nothing of the edges attribution added there later. Whether those edges occurred on every input is the question the fourth correction answered, and for most of them the answer was no. On the final instrument the check finds 0 failures among the 150 must edges and cannot read the destination of 62, all of them k8sdash's toolbar edges, so of that application's must edges only about half are read by the check.

The part of the model that an earlier instrument did not implement had a measurable price. Navigation sites in services, guards, interceptors, resolvers and effects are counted separately, and in absolute terms they are a small class across the corpus, yet they are not spread evenly: in one application they are almost as large as the shared-component class (Table 5.4), and before the call graph was added they were the only remaining loss that the analysis as specified would have recovered. The call graph with dependency injection, which the current extractor implements (Section 5.3.3), was thus required for applications that navigate from services; the dispatch relation is still not implemented.

RQ4 gives a conditional answer, and the condition is RQ1. Leaving aside the rename operator, whose detection is close to guaranteed by construction, the three operators that can fail are detected in 45 of 45 scored mutants on the attributed graph: every deleted `routerLink`, 20 of 20 retargeted `Router.navigate` calls and 11 of 11 removed guards (Table 5.15). That clean result came only after two corrections to the harness, both found by reading the undetected mutants: a retarget that replaced only the head of its commands array, and a guard whose screen was matched by class name to a different class of the same name (Section 5.4). Each undetected mutant on the previous run was the harness's error rather than the graph's, which says the harness needed the reading; it does not show that the remaining operators would survive a larger or differently drawn sample. Guards are what the first correction of Section 5.5 was about, but no per-operator result from the earlier instrument survives, so how far this operator moved cannot be said.

An earlier state of this extractor was read directly off the graph for this class: in realworld the edge entering `/editor` carried no guard although the route declares `canActivate`, while the edge entering `/editor/:slug` carried it, and the difference between the two was put down to the empty path. That reading was right about the graph it was taken from and wrong about the cause. On the instrument this thesis ships both edges carry `requireAuth`, and the empty path is not what separates a detected guard mutant from an unobservable one: what decides that is whether any edge enters a screen the guarded route yields, which is now the exclusion rule (Section 5.4). The class "guards unreliable in both directions" of Chapter 5 rests instead on what Section 5.5 measured, a guard carried by a route a wrapper function returns and a `canActivate` element written as a call; the mutation run neither confirms nor refutes it.

Detection is conditional on coverage in a second sense, which the exclusion rule makes visible, though to a far smaller extent than on the instrument this work began with. Against the attributed graph only 6 of the sampled mutants cannot be scored at all, all of them guards the graph can record on no edge (Table 5.14); against the baseline graph 22 cannot. The distance between those two counts is where RQ1's coverage shows up in this experiment: the constructs that attribution puts into the graph are largely the ones the baseline run had to exclude. Either count is a coverage result and not a sensitivity one. The conclusion is therefore conditional: given that a construct is represented in the graph, every scored change to it was visible in the graph, for the operators sampled on three applications (45 of 45 fallible mutants), and the probability that it is represented at all is what RQ1 measures. For change

impact analysis this form of claim is appropriate, because an impact query is only ever asked about the part of the application that the model represents, and the graph states which part that is through its witnesses.

The three mutated applications do not declare their routes the same way, two as standalone arrays and one in NgModule routing modules, and no operator fails under one style and not the other (Section 5.4). The measurements support no comparison beyond this. The corpus was not stratified by routing style and the other three applications were not mutated, so nothing more general about the two styles follows from this thesis.

Two uses of the extracted graph are supported by what was measured. One is change impact analysis. Every edge carries the file and line of the construct that produced it, as an invariant of the model, so a set of changed lines can be mapped to the edges whose witnesses fall inside them and from there to the screens those edges leave and enter. This is the screen-level counterpart of what Chianti does with atomic changes, affected tests and methods [62], and it sits in the impact analysis tradition that Arnold and Bohner describe [4, 12, 44]. RQ4 provides evidence that the mapping reacts to the injected edits: for the constructs the graph represents, an authored change in the source produces the predicted change in the graph. That is a property such an analysis needs, not a measure of its accuracy, which would ask whether every affected screen is named and how many are named in excess.

The other is checking in continuous integration at no execution cost. The graph is recomputed from source on each commit, the extraction is deterministic and carries no state between runs (Section 4.3), and nothing has to be installed, served, authenticated against or crawled. Properties of the navigation structure can then be asserted on every commit in the same way a type check is: that a screen still has at least one inbound deterministic transition, that a screen that was reachable from the shell still is, that a route path a test depends on has not been renamed. A guard assertion is the one obvious member of that list the graph cannot yet carry, because a guard is recorded on the edges entering the guarded screen, so for a screen that no edge enters there is nothing for such an assertion to read (Section 5.4). An assertion that passes for the wrong reason is worse than no assertion. A dynamically built model cannot be used this way at the same cost, because it needs a running application, test data and an exploration budget, and it has to be rebuilt for each version [51, 52, 37].

Both uses stop at the same place, which is runtime dynamic routing. Where a navigation target is computed from a value, where the route table is assembled at startup from configuration rather than declared, and where a user interface state is an overlay that no route describes, the static graph is either silent or over-approximates into the may tier, which carries no soundness claim. Under the assumptions of Claim 3.1, the must tier is a lower bound on the navigation an application can perform; the may tier is not, and a may edge may name a transition the application cannot make. A query that must not miss a transition therefore cannot be answered from the graph alone, and the graph's own witnesses and declared approximations are what tell a consumer which queries those are.

6.2 Limitations and threats to validity

The results are measurements of one tool, on one corpus, with five methods, performed by the person who wrote the tool. Each of those four choices limits what the figures support.

The scope is validated in both directions on one application only. Every other method in Chapter 4 checks a node, an edge or a witness; none of them can be asked whether the set of screens attribution computes for a site is the set it fires from. Two methods can. The hand-read structural reference answers it in both directions, but for one application of six and only as agreement with the analysis's own relations (Section 5.3.4): there the analysis finds 100.0 % of the (site, screen) pairs of that reference, at 100.0 % precision against it; the rule the call graph replaced, which joined a navigating service over its injectors, asserted 9 pairs outside it at one site, and the call graph was written after that finding and scored on the same ground truth, so the clean figure is consistent with the repair but does not test the call graph independently. The levels that stand for what earlier work reaches already recover 94.7 % (shell rule) and 99.5 % (with the enclosing routes) of the reference pairs with none outside it. Beyond realworld only the runtime method applies, on 5 of the six applications, and it answers half of the question, on an earlier graph than the final call graph and on 30 of 806 screens (Section 5.3.1): of the 33 (screen, target pattern) pairs the applications render on the screens it could judge, 29 are predicted by the edges leaving that screen, a test of the screen's pooled outgoing destinations rather than of each site's scope, and the 4 that are not turn out to be sites attribution declined to place at all rather than sites it placed wrongly.

What remains unvalidated is the other half and the reach. A site attributed to too *many* screens produces edges whose witnesses all pass, because the construct really is at that location and really does name that target; the oracle cannot convict it, because a predicted link missing from the DOM is equally well a link the template renders only to a signed-in user. The predictions it could in principle convict are those that name a destination exactly, rather than over-approximating to every address a static prefix admits. There are 7 of them on the screens it could judge, against 102 that over-approximate, so the direction is unrefuted but also close to untestable by this method. And the oracle judged 30 screens of 806: without a backend, 776 of them redirect to a sign-in page and render another screen's links. Both limits would be lifted by driving the applications far enough to render their interiors, that is, by seeding a backend, signing in, and visiting a guarded screen in a known state. Of the possible future work, this would do most to change what this thesis can claim about its own contribution. The shell rule is capped at *may* where the mounting route is invisible (Section 3.3), which limits what an over-large scope can claim, but that is a design precaution and not a measurement. What can be measured without a running application is the width of the scopes rather than their truth, and Section 5.3.2 measures it: the computed attribution is narrower than assigning every site to every screen by a factor between 1.7 and 18.4, and on k8sdash the average site is attributed to 59 % of the screens. A scope that wide says little about where a site fires, even if it is sound, and the coverage attribution recovers

comes at the cost of that width. The same untested direction bears on the must tier: a must edge's source is a screen the scope places its site on, so most of k8sdash's must edges, one per screen its toolbar is attributed to, are as right as that one scope.

Construct validity. The conversion ratio divides the constructs witnessed by a deterministic edge by the in-scope navigation constructs that the independent parser finds in non-test source. Query-only calls and the two exclusions of Table 4.2 are taken out of the denominator, but it still over-counts: it contains constructs in code that the application never mounts, such as components whose selector no template uses, which the inspection of the remaining losses identified. That over-count pushes the ratio down, but the denominator also under-counts, so the ratio is not a bound in either direction. The true figure cannot be computed without deciding reachability for each construct by hand and recognising every navigation form.

The under-count is the more fundamental threat, because it bounds what the measurement can see at all. The parser recognises navigation by four syntactic patterns: a `routerLink` attribute or binding, and a call to `navigate` or `navigateByUrl` on a receiver whose name contains *router*. Navigation expressed any other way is in neither the numerator nor the denominator, so it is counted as neither a loss nor a success. An application that wraps navigation in a helper, so that its components call `this.nav.goHome()`, contributes one construct at the wrapped call site and none at the places that invoke it. An assignment to `location.href`, a redirect driven by a server response, and navigation performed inside a third-party component are all invisible on the same grounds. The conversion ratio is therefore recall against an enumerable syntactic proxy for an application's navigation, not against its navigation, and the completeness of that proxy is not itself measured. The test-source cross-check is the only method that can see past it, because a URL the application's own end-to-end specifications name is an address its authors expect to exist, whatever the source looks like. It reads the specifications without running them, so a URL there is named rather than observed to be reached, and it covers one application of the six.

Node coverage carries a stronger form of the same threat, and it points the opposite way. Its denominator is the screens the independent parser can address, so a screen whose path neither tool can read is outside the ratio rather than counted against it: refusing to read more of an application raises its reported coverage. The effect is not hypothetical. Three of one application's declared screens assemble their paths inside a template literal from an exported constant; while neither tool read them, that application's coverage was complete, and reading them is what moved its denominator (Section 5.5). Nothing in the ratio itself would have said so. The defence adopted here is that every coverage figure in this thesis is printed beside the count of routes the parser knows are there and cannot address, in the same table, so that a figure bought by refusing to look is visible as one. For this kind of analysis, a coverage ratio reported on its own cannot be taken at face value.

The two units also stand in a many-to-many relation, which the metrics handle unevenly. One construct in a component that many screens render yields one anchored edge that expands to many, which is why the ratio counts constructs and not edges (Section 4.3.2). In the other direction several constructs can yield one edge, and the

extractor records the extra ones as merged witnesses so that each is traceable. For most of this work the conversion ratio credited a construct cited by any witness of an edge, primary or merged, while the witness check re-opened only the primary witness, so a construct could count towards the ratio on evidence the precision check never examined. On realworld 38.1 % of the citations in the attributed graph are merged ones, and 58.7 % on the application where the share is largest, so the gap had practical consequences: the check reported a clean deterministic tier over part of the evidence that tier rests on, and wrong targets in bitwarden sat in the part it did not read (Section 5.5). The check now judges every location an edge cites, each location records the modality it earns on its own, and the must tier is judged on the locations that earn must, so a construct counts towards the ratio only on evidence the check has read. The ratio and the precision figure still answer different questions, and that, rather than the former gap, is why the chapter reports them separately.

The graph also loses guard information, though not where merging happens. Constructs are merged into one edge only when their recorded guard texts are equal, since the guard is part of the edge's identity, so a merge never discards a recorded guard. The loss comes earlier: a structural condition in a template, or a condition around a navigation call, demotes the edge to may but is not written into its guard (Section 3.2), so the guard of a screen-to-screen edge names only the router's guards. The graph is in that respect a weaker object than the source it came from, and a consumer that reads a guard off an edge learns which router guards run, not under what condition in the application the navigation is reached. The loss breakdown has a second construct threat. The rules assign each lost construct to the first class that matches, so the precedence order is part of the measure. A construct in a shared component that is used only inside screens with no node is an attribution loss under the present order and a node loss under the reverse order. The sizes of the classes are comparable within one application and under one order, which is what Chapter 5 claims, but they are not independent quantities, and a different order would move constructs between classes without any change to the tool.

Internal validity. Extraction runs on source trees whose dependencies are not installed, so the TypeScript compiler cannot resolve types that come from the framework or from third-party packages. Type-directed steps such as narrowing an injected receiver are weaker than they would be in a fully installed project, and the measured figures are properties of that configuration rather than of the analysis in principle. The mutation experiment has small samples per operator, in single digits for several of the operator and application pairs, with a seeded quota rather than exhaustive generation; one mutant moves a per-operator rate substantially, so the pattern of which operators are detected is more trustworthy than the rates themselves, and the published advice on reporting mutation experiments is followed for that reason [58]. The exclusion rule that removes mutants whose construct is absent from the graph is applied using the graph itself, which couples the sample to the tool's own coverage; the excluded mutants are reported rather than dropped silently, but the sample is not independent of the artefact under test. The test-source cross-check extracts URLs from test source text, which under-samples click-driven navigation and can collect strings that no test ever visits, so it measures node existence and nothing finer. On the instrument these figures are measured on the independent

parser follows every lazy specifier in the corpus, and the application whose node coverage it once based on two screens is now read in full by both readers; what the differential test has lost in exchange is independence, since several of the forms it reads it learned from the extractor (Section 4.3.2). That is the reason the runtime method was added, and why the chapter rests the external check on it rather than on the agreement between the two readers of the same text.

External validity. The corpus is six open-source Angular applications pinned to single commits. Proprietary applications are not measured, and they may differ in size, routing style and the mechanisms they use for navigation, which is the threat that open-source corpora are known to carry [35, 36]. Two further limitations come from how the corpus was assembled. The selection is tool-driven: `SAP/spartacus` was rejected because its routing is configuration-driven in a way the extractor does not support (Section 4.1), so the corpus is biased towards applications whose routing the tool already handles, and a figure measured on it is likely to be optimistic with respect to the population of Angular applications. The candidate pool also showed an inverse relation between application size and end-to-end test density, so no application offers both a large route tree and a dense test suite. The evidence from outside the extractor's own reading is of three kinds and is not spread evenly: URLs named in end-to-end test sources, which are read and not run, come from alfresco alone, a medium-sized application; the router configuration that the running application holds was read on all 6 applications; and the links rendered on screens were observed, on an earlier graph and in part, on 5 of them. The findings about scale therefore rest on the runtime router configuration and on the extractor's own reading, and on no test that was observed to visit an address. The results are Angular results. The graph model and the lattice do not mention Angular, and Section 6.4 argues about other routers, but that argument is not a measurement.

Oracle independence. The route parser shares no code with the extractor, which makes it independent and says nothing about whether it is right. That independence is also partial, and Section 5.5 says where it stops. For the route-path forms the second round added (a table behind a freezing call, a path assembled from constants, a module reached through a workspace alias), the two tools were taught the same rules because each was wrong in the same place. Their agreeing about such a path is therefore weaker evidence than their agreeing about a path each reads its own way. Its first version did not look inside wrapper calls, did not follow a lazily loaded module to the routing module it imports, and walked lazily loaded tables without their prefix, and it therefore reported that every declared screen of mifos had a node; after the correction it finds a far larger set of declared screens of which under a tenth have a node, with the extractor unchanged in both runs (Section 4.3.2). The change in the figure came from the oracle. A differential test also only shows that two implementations disagree, never which of them is right, and each disagreement in this study was resolved by reading the application source, by the same person who wrote both implementations.

Experimenter bias. I wrote the extractor, and I also designed the evaluation, selected the corpus and defined the defect classes. The mitigations actually applied are the following. The six selection criteria were fixed before any results were

obtained, and the rejected candidates are reported with their reasons; one rejection was on tool support, a ground outside the criteria, and two retained applications do not meet the test-suite criterion (Section 4.1). The construct counts and the declared screens come from a parser that shares no code with the extractor, with the qualification above: for the path forms added in Section 5.5 the two resolve the same constructs by the same rules, so that mitigation does not cover them. The witness check is mechanical and re-parses the source at the claimed location, which makes it a check on the witness and not on the address: an edge whose target node sits at a path the application does not mount satisfies it. The mutation oracle is the authored delta, so it does not consult the tool's output to decide what the expected change is. The corpus commits, the extractor commit and the harness version are pinned, the evaluation re-runs with two scripts at one commit of the extractor (Section 4.3), and the conversion, coverage, loss, remaining-loss, before-and-after and mutation tables of Chapter 5 are generated from their result files instead of being entered by hand. The defect-class table, the oracle table and the inspection of the unreached components are written by hand. Determinism and the absence of accumulated state were checked before any measurement. Eleven protocol corrections are reported in full (Section 4.3.2), including the three that made the reported figures worse. Several controls were not applied: there was no blinding, no second annotator for the loss classification and no independent audit of the residual constructs that were inspected by hand, and the defect classes were defined after the first observations rather than in advance. The source-level readings of the must tier, two readings of the round's 30 constructs and one of the final 25, were made by the AI assistant declared at the front, which also helped write the extractor and the route parser, and the final reading was made with the extractor's decisions in view; no human second reader checked them, so their agreement with the extractor shows consistency with the stated rules and not independent correctness. The repairs of Section 5.5 were also found on, and are measured on, the same six applications, and no held-out application was measured, so the post-repair figures are fitted to this corpus. The requirement of two lines of evidence per defect class is the strongest control that was applied to that last point, and it is the reason one class is still reported as a candidate. It is a weaker control than it first appears: Table 5.3 shows that four of the five accepted classes rest on one named method, the rest of their evidence being observations rather than a second instrument, so for those four the control is one method plus judgements I made myself.

The runtime check of Section A.5 removes one threat and introduces others of its own. It removes the one that mattered most, which is that two readers of the same route files can agree on a wrong address and the agreement be read as a result. What it brings in its place is first a selection effect: the method needs an application that builds and boots, and every application in this corpus did, but a corpus chosen partly for being buildable is not a corpus chosen at random. Second, it reads the route configuration rather than the rendered page, so every address it calls mounted is one at which the router would place a component, not one at which a visitor would see it; a route behind a guard that always refuses, or one whose component throws on render, counts the same as one that works. Third, the build was a development build in each case, because Angular removes the debug

surface the method reads when an application declares itself in production mode, and a production configuration can differ from a development one in ways a route table does not show. Fourth, where a workspace application composes the library a measurement is scoped to, the framework's set is a superset of the library's by construction, and part of any difference is scope rather than reading. Two of alfresco's three differences are of this kind: `/login`, declared by the shell application, and `/view/authentication-confirmation`, declared by a package that shell installs. The third is not: `/about` is declared inside the measured library, in a JSON plugin manifest that no reader of the code can see.

It might seem that none of these weakens the claim the check is used for, because none could make a mounted address look unmounted. That reasoning has the direction backwards. The claim is that an address the extractor mints is one the framework mounts, and it is tested by finding the extractor's set inside the framework's. Three of the threats make the framework's set *larger* than the set of addresses a visitor can reach in the measured scope: a route behind a guard that always refuses is counted as mounted, as is one whose component throws, and a route the composing shell or its packages declare outside the measured library. A larger containing set makes containment easier to satisfy, so these weaken the no-invented-address claim. The build configuration is a different kind of threat, because a production configuration can add routes as well as drop them, so it does not move the set in one direction. Both results of the check are therefore statements about the development router configuration that was recorded, under the abstraction of Section A.5. On that configuration, the failures the earlier cohort showed are real: the two alfresco addresses of the first comparison, and the 10 the comparison returns against the static readers' route set as it stood before this round, are absent from the recorded configuration. The final extractor mints 0 such addresses, which is reported as the absence of a failure on that configuration rather than as a proof of its impossibility.

6.3 Declared approximations

Two of the approximations this section declares were found by the runtime check of Section A.5 rather than by reading the analysis, and they are stated first because both are places where the analysis is silent rather than approximate.

A route may be declared outside the language. alfresco's about page is registered by a JSON manifest that names a path and an identifier, which an extension registry binds to a component class at run time. No reader of the application's TypeScript can see such a route. This one raises no note, because nothing in the code is unreadable: the route is not in the code at all. A tool that wanted to cover it would have to read the framework's plugin format as a second source language, which is a different undertaking from reading a router configuration and is not attempted here.

Which of several tables a prefix mounts may be decided outside the sources. mifos holds two route tables for its users module, one per identity provider, and selects between them with a constant assembled from `window.env`, a value injected into

the page when the application is deployed. Where the sources fix the condition of a conditional, the extractor reads it: a flag written in an environment file is resolved and the arm it selects is read. Where they do not, as in this case, it declares the subtree unread. The loss is declared and it is 5 screens wide, which is the price of the rule that decides the question: the table a prefix mounts is the one its route names, or the one the module registers with `forChild`, or the only one in the file, and otherwise none. Reading every table a file holds instead would have given this prefix the right addresses, by accident of the two tables agreeing, and gives any prefix every table a file holds, which mints the cross product of prefix and path wherever two tables differ.

Where such a condition *is* fixed by the sources, the value read is the one the checked-out sources carry. An Angular build may substitute one environment file for another, so a constant a production build fixes differently selects a different arm there, and the graph is then a graph of the configuration that was read. The measurements in this thesis are taken over the sources as checked out, and the runtime check of Section A.5 reads a development build for the same reason, so the two agree about which configuration is under measurement.

The thesis claims soundness only for the deterministic tier and only over the declared route set (Claim 3.1). Following the soundiness position of Livshits et al. [48], the claim is paired here with an explicit list of the constructs the analysis does not decide, so that a consumer of the graph can tell which queries it may ask. The list separates constructs that are in scope, where a failure is a defect that Chapter 5 counts, from constructs that are out of scope, where no completeness is claimed at all.

In scope are route entries whose path is a string literal together with the component they render, `routerLink` bindings whose value is a literal or an array of literals, calls to `Router.navigate` and `Router.navigateByUrl` with a literal target, the nesting relation obtained from element selectors resolved within the importing file's scope, the primary `<router-outlet>` with the shell rule that follows from it, and guards named on a route as labels on the edges entering that route's subtree. Everything in this group is the analysis's responsibility. The failures measured in Chapter 5 are failures inside it: route paths that are constant references are not resolved, route tables behind a lazy boundary are not followed, and some nodes are not linked to the component that renders them. These are bugs with known causes rather than approximations to be declared away, which is why RQ5 could repair them; Section 5.5 reports what repairing them cost.

Out of scope are the following constructs, each with the reason it is approximated and the effect on the graph.

- **Components created programmatically, including dialogs and overlays.** An overlay is a user interface state that no route declares, so it has no screen node, and no edge records that a screen opens or dismisses it. The component that implements it is reached from no template, so its navigation sites have the empty scope. Without attribution they contribute nothing. With it, each such component that navigates, and each navigating component nested in its template, is given an unplaced origin that carries its file, and its

navigation sites become edges from that origin to their resolved destinations, held at *may*; these are the edges Section 3.1 counts as leaving a node that is not a screen. A navigation in a service that only such a component injects receives no scope and is recorded only as a note. What the graph lacks is therefore the overlay state, the transitions into and out of it, and the screens from which its exits fire, not the destinations of those exits.

- **Named outlets and targets of the form `{outlets: ...}`.** An auxiliary route adds a second active route and a second mount point, encoded in the URL as `(name:path)` [27]. The model of Chapter 3 represents this as a second mount, and the adapter does not implement it. A route that names an outlet other than the primary one, every route beneath it, and a route whose outlet the reader cannot fix to a string mint no node; the last kind is counted in the note that marks the route set incomplete. A commands array that carries an `{outlets: ...}` element yields no edge and no note. Only `navigateByUrl` parses a URL: an auxiliary group there is dropped and the edge to the primary path is held at *may*, since the whole navigation fails when the auxiliary route does not match and that route is not read, and a group that also carries the primary outlet is not rebuilt and yields nothing. A `routerLink` string or the first element of a `navigate` command is split on slashes alone, as Angular splits it, and a later string element is one segment whatever it holds, so `/inbox(popup:compose)` written there is one segment and reaches whichever route matches it, a parameter route if there is one. The test-source cross-check shows the auxiliary URLs among its misses.
- **Navigation relative to the active route (`relativeTo`).** The target is a function of the route of the component that holds the site, so it has no single value in the source, and attribution is what makes it resolvable: the analysis resolves it once for each screen the edge is emitted from. A template `routerLink`, and a `Router.navigate` call whose options are an object literal with a `relativeTo` key, are resolved against the path of that screen, applying a leading run of `.` and `..` segments; a call without `relativeTo`, and `navigateByUrl`, are resolved against the root. A target with more `..` segments than the route it starts from has segments is one Angular rejects with an error, so it yields no edge and a note; from a route with no segment of its own Angular resolves the target as absolute, and the extractor does the same. Angular also applies a `..` that follows another segment, as in `a/../b`, where the extractor keeps it as a literal segment; such a target then names an address no route declares, and the edge falls to the catch-all or to a note rather than to a wrong screen. The base is right only when the screen is the route of the component that holds the site, and only then can the edge be must. Everywhere else it is held at may: for a component rendered under a route it does not own, whose own route the analysis does not track; for `relativeTo` written in shorthand, passed through an options variable, or given a route other than the component's own, such as `this.route.parent`, none of which the reader evaluates, which is to say anything but a constructor parameter or an `inject` call that yields the component's `ActivatedRoute`; for a link that binds its own `[relativeTo]` input; and for an edge emitted from a node without a

path of its own, a scope anchor, the shell or an unplaced origin, where a bare relative target is matched as if it were absolute. Such an edge can name the wrong screen, which the may tier admits and the must tier does not, and a target that matches nothing under the assumed base is recorded with a note that says the base was assumed.

- **Templates passed as values (`TemplateRef`) and rendered elsewhere.** The analysis does not follow a template value from where it is declared to where it is instantiated. Elements and `routerLink` bindings inside an `<ng-template>` are read from the text of the template that declares them, so they enter the nesting relation and the edge set as part of the declaring component and are attributed to its screens. Because nothing says whether or where the template is instantiated, every such link is held at *may*. The attribution is right when the template is instantiated within the declaring component's own subtree, as with a template passed to a child's input or to a dialog opened from that screen; when the value is carried to a component rendered on other screens, through a service or an outlet in the shell, the edges are attributed to the declaring screens rather than to the screens on which the template appears. Recovering the place of instantiation is a dataflow problem over the values that carry the template, the same obstacle that Section 6.4 identifies for JSX.
- **Dispatch through NgRx effects and router event subscriptions.** The dispatch relation is specified in Chapter 3 and not implemented, so the screen that dispatched an action is never recovered. The extractor stands in for it at the levels that keep the call graph or its class join, *full* and *class*: every navigation call in a file inside the application directory that contains a `createEffect` call is given $\top$ and offered from the shell at *may*, or into the shell's `u_` sink when its target is fully dynamic. This is sound and coarse, since an effect can fire on any screen and the edge is drawn from the shell rather than from the screen that dispatched the action. At the levels below those, the stand-in is not applied. For an effect outside the application directory, or one declared with the older `@Effect` decorator, the navigation is treated as any service call: it is placed by the call graph or the class join where they reach it, and recorded as a note where nothing does. A subscription to `Router.events` is not treated as dispatch: a navigation inside it is attributed to the scope of the class that contains it, as any other call site is, and because it runs in a callback the subscription invokes, it is held at *may* (Section 5.5).
- **Navigation inside signal effects and observable subscriptions.** The firing condition is a data dependence in a graph the runtime builds, not a control-flow condition in the program text (Sections 2.7.5 and 2.8). The model gives such edges the modality *unknown*, with the read set recorded as the symbolic guard. The extractor implements neither the modality nor the read set. A navigation inside an `effect` body, a `subscribe` callback or any other function something else calls is attributed to the scope of its class like any other call, and held at *may* because it runs only when that function is called (Section 5.5); a fully dynamic target goes to the `u_` sink as any other does. What the graph records is that the screen can be left for that destination, not under what condition.

- **Computed navigation targets.** A target built at run time from values cannot be enumerated statically in general, and the analysis treats it according to the static text it contains. A target with no static part, a `routerLink` whose value is an interpolation among them, is recorded as an edge of modality *unknown* and confidence 0 into the `u_` sink of the node it fires from. A target with a static leading part (the head of a template literal, the left operand of a string concatenation, or the constant elements before a trailing variable element of a commands array) is over-approximated into the *may* tier, with one edge to every declared route that extends that prefix by at least one segment. An empty prefix therefore connects the site to every declared route except the root, as one of the protocol corrections showed (Section 4.3.2). The over-approximation is not complete: a prefix segment matches only an identical route segment, so a route with a parameter in that position is not a candidate, and a prefix that matches no route yields no edge and no sink, only a note that gives the count as zero. A commands array whose variable element is followed by constant elements is read as a pattern and drawn as a single *may* edge to the first declared route that matches it, with the variable element matching only a parameter segment, so a route whose segment in that position is a constant is not reached, and no note records the omission.
- **Injection tokens and provider bindings.** The call graph resolves a call on a field through the field's declared type: the class it names, the classes a provider declaration binds it to with `useClass` or `useExisting`, or the subclasses of an abstract class no provider binds. Three things are not read. A binding is taken application-wide rather than from the injector that resolves the token, so where two components provide a token differently both implementations are joined, which widens a scope and never narrows it. A factory or value provider (`useFactory`, `useValue`) and a provider array a function returns from outside the application bind nothing the pass can follow, so a call through such a token contributes no edge, and the member it would have reached falls back to the join over every component that injects its class, or to no scope at all if none does. A receiver is resolved when it is a field of the class, an `inject(T)` call, or a local variable or parameter that is initialised with `inject(T)` or declared of an application type; a receiver of any other form, such as a value returned by another call or an untyped local, is not resolved. Over the corpus the pass resolved 5574 calls on application types to the class the type names, 4937 through a provider binding and 18 to subclasses, and left 1111 unresolved, in 2 of the six applications.
- **Configuration-driven route registration.** A table passed as an array to `RouterModule.forRoot`, `forChild` or `provideRouter` is read wherever the registration runs, including inside a provider factory the application calls, which is how alfresco's folder-rules library mounts `/rules`; a table held in a variable is also found by its shape, whatever registers it. What is not recovered is a route that exists only once configuration has been evaluated at startup, and the effect on the graph depends on whether the source declares the route object. Where a route object is declared and its `path` is supplied by configuration, the path is counted as unreadable, no node is minted, and the route set is declared a lower

bound. Where the table is computed by a function and passed to a registration call, provided through the `ROUTES` token by a factory or as an inline value, installed with `Router.resetConfig`, or declared in a JSON extension manifest such as the one that registers alfresco's about page (Section A.5), the routes are absent from the graph and no note is raised, so the route set is not declared a lower bound on their account. The independent parser behaves identically on each of these forms, which is why the two readers agree on routes neither can see. This approximation bounds the corpus as well as the tool, since it is why one candidate application was rejected (Section 4.1).

- **Guards that would let the navigation through.** A guard is a function of the application's state, and the extractor does not evaluate it. A guard the navigation runs holds the edge at *may*, even when the guard would always pass. The guards a navigation runs are the `canMatch` of a route its address is matched through, the `canActivate`, `canLoad` or resolver of a route it enters, the `canActivateChild` above those, and the `canDeactivate` of a route it leaves. A route the source and target share is taken to be reused unless its path has a parameter, `runGuardsAndResolvers` asks for more than a parameter change, or, under the default mode, the navigation does not keep the segments the route or one above it consumes, since a matrix parameter there may change; a parameter or matrix parameter that does not in fact change makes this too cautious. A guard is identified by the text of its expression, so two guards that compute the same thing are two guards. Content a component projects through `<ng-content>` is read as its host template places it, so a link projected into a component that renders the projection only under a condition is read as unconditional; `mat-menu` and `mat-tab` are the only library containers read as rendering their content lazily. A route component behind a conditional outlet is conditional, but a component nested in it is judged by its own placement, and the application root's outlet is not read.
- **Navigations a later one may cancel.** This item and the one on activation with a modifier key, two below, are the execution model of Claim 3.1: the claim holds only on executions they do not describe. A navigation is demoted when code that runs after it, before the event is handled, may start another (Section 3.1). Whether that code may navigate is read through the calls the reader can resolve to a body in the application: a member of the class, a function of the file, a method the checker resolves on a typed receiver. A call into a library, through a receiver the checker cannot type, or into code the application loads at run time is taken not to navigate, so a navigation followed by such a call can be published as must although the call cancels it. The reader does not follow a navigation a handler schedules for later (a timer, a promise it does not await), which is the same approximation in time rather than in the call graph. Nor does it read the other listeners one event reaches: a click handler on an ancestor that the event bubbles to, or a `(click)` binding that navigates on the same element as a `routerLink`, either of which can start a second navigation. Callers are followed within the class; a member another class calls is not. An earlier version of the rule read only the statement right after the navigation, and only when that statement was itself a navigation call;

the one call in the corpus's must tier is a single-statement handler, so the rule's widening left that tier as it was.

- **Children of a route a function returns, mounted at the empty path.** The first empty-path child under a screen, and the first empty-path child below that one, render on the screen's own address, and their links are drawn from it. Where the empty-path route is written inside a function that returns it, as `withShell([...])` does, the walk does not place the component the function's children name under the screen above: its links stay on an unplaced origin at *may*. The graph loses those edges and asserts nothing false about them.
- **Activation with a modifier key.** A `routerLink` on an anchor leaves a click made with a modifier key held, or with a button other than the primary one, to the browser, which opens the address in a new browsing context rather than navigating the current one, and so does an anchor whose `target` names another context. The model reads every activation of a link as a navigation of the application, so a must edge from an anchor holds for a plain activation and not for these.
- **The order in which Angular matches routes.** The router takes the first route, in declaration order, that matches an address; the matcher prefers an exact path to a parameter route and consults a redirect only when no route matches. Wherever two declared routes, or a route and a redirect, can answer an address, the edge is held at *may* rather than decided, and it may name the route Angular does not take. One exception rests on an assumption. Route tables an application registers apart are joined in the order their providers are registered, which the extractor does not read. It takes a catch-all in one such table to be matched after every route of the others, the order Angular's documentation recommends and the one the corpus applications appear to keep, and it names each catch-all it dismissed this way in a note. Extracted again without the assumption, the must tier holds 24 edges on 22 constructs instead of 150 on 25: the other 126 edges, on 3 constructs in 2 applications (k8sdash, bitwarden), rest on it.
- **Ambiguous selectors in the nesting relation.** When a selector belongs to more than one component, the candidates are narrowed to those the rendering file imports; where none is imported, the analysis joins over all of them and records a note. Such a join can attribute a site to screens on which the component it belongs to is never rendered. Attribute selectors are matched as element selectors are, so a directive is in the nesting graph wherever its attribute appears. When the question is whether a component renders an outlet through a component it places, an ambiguous placement counts only if every candidate renders one, since the reader cannot say which candidate the build instantiates.

Table 6.1 gives the prevalence of the approximated constructs in the three applications that were cloned for the mutation run. The in-scope kinds are not repeated, because the construct denominator of Table 5.1 already counts them with the parser. The counts come from `approx-counts.mjs`, which is part of the harness and runs over the application sources with tests excluded. It counts call sites, which is a different procedure from the parser that produces the construct denominator of

Table 6.1: Prevalence of the approximated constructs in three of the six applications, produced by `approx-counts.mjs` in the harness. The figures are call-site counts over the application sources with tests excluded, taken at the pinned corpus commits. They are a different unit from the construct denominator of Chapter 5, which comes from the route-oracle parser.

Construct	realworld	alfresco	bitwarden
`dialog.open`	0	3	0
auxiliary outlets	0	7	0
`relativeTo`	1	1	7
`TemplateRef`	2	0	11
`createEffect`	0	49	0

Chapter 5, so the two are not comparable term by term. Three of the six applications were counted, so no corpus-level prevalence is claimed.

Two things follow from the counts. Set against the construct denominators of Table 5.1, which are 40, 34 and 131 for these three applications, the approximated kinds are few in realworld and bitwarden, so the approximations do not explain the coverage figures of Chapter 5 for those two. In alfresco the approximated sites outnumber the 34 constructs, almost entirely through `createEffect`, so for that application the approximations cannot be set aside. The two numbers are not the same unit, since the denominator counts navigation constructs while the `createEffect` figure counts effect declarations, whether or not they navigate. The approximated kinds are also concentrated: auxiliary outlets appear in one application of the three, NgRx effects (`createEffect`) only in that same application, templates passed as values mostly in another, and relative navigation everywhere but in small numbers. An approximation that costs nothing on one application can therefore be the dominant one on the next, which is an argument for declaring the list rather than reporting a single aggregate.

The call-site counts also understate the structural cost of an approximation. A handful of `dialog.open` call sites can open many different components. The inspection of the owner components that attribution never reaches found about sixty components across the corpus that are opened programmatically, most as dialogs, whereas `approx-counts.mjs` finds three `dialog.open` call sites, all in alfresco, in the three applications it covered. The two figures cover different scopes and are not directly comparable. The right unit for the cost of this particular approximation is the number of components and navigation sites it strands, not the number of calls that strand them.

6.4 Transfer to other frameworks

All measurements in this thesis concern Angular. How much of the approach would carry over to React with React Router and to Vue with Vue Router [61, 81] is argued below.

Table 6.2: Constructs that provide the input relations, by router.

Concept	Angular	React Router	Vue Router
Outlet	`<router-outlet>`	`<Outlet />`	`<router-view>`
Several outlets	named outlets, encoded in the URL	none	named views, not in the URL
Declarative link	`routerLink`	`<Link to>`	`<router-link :to>`
Imperative navigation	`Router.navigate`	`useNavigate()`; redirect from a loader	`router.push`
Route guard	per route (`canActivate`, ...)	loader that throws a redirect	per route (`beforeEnter`) and global (`beforeEach`)
Access to services	hierarchical injectors	context and hooks	`provide`/`inject`, composables, plain imports
Reactive trigger	signal effect	`useEffect`	`watch`, `watchEffect`

What follows is a qualitative analysis and not a measurement. It compares the documented behaviour of the three routers with the input relations that the analysis needs, and what it yields are hypotheses about where the work would move, at the level of evidence of a design argument. Nothing in it shows that the approach performs on React or Vue applications as it does on Angular ones. Establishing that would require a corpus for each framework and the methods of Chapter 4.

The analysis has a framework-independent part and a framework-specific part. The graph model, the lattice of scopes, the fixpoint, anchored edges and the expansion query do not mention Angular. What is specific to Angular are the four input relations of Section 3.2: component nesting, router outlets, the call graph with dependency injection, and dispatch relations. Moving to another framework means deriving the same four relations from that framework's constructs. Table 6.2 lists the corresponding constructs.

Outlets transfer directly: in all three routers a parent route renders a placeholder where the matched child is inserted. The components above the placeholder form a shell that is present on every child screen, so the scopes $\top$ and $\mathit{subtree}(r)$ have the same meaning. Vue's named views are simpler than Angular's named outlets, because they do not appear in the URL; they add a second mount point but no auxiliary route. React Router has no equivalent, so this class of defect does not arise there.

The call graph is easier in React and Vue. The hardest Angular-specific step is resolving an injected service through the injector hierarchy. In React, a navigation function is usually obtained from `useNavigate()`, and a hook can only be called while a component renders, either directly or through other hooks. Every such navigation site therefore has an owning component that can be found by following

hook calls, and no injector needs to be resolved. Vue composables behave the same way inside `setup`. The case that remains is code that imports the router instance directly, which Vue applications often do in stores and HTTP interceptors. This is the analogue of Angular's service case: the site has no component of its own, and its scope is the join over its callers. Since the link is a plain module import, it can be resolved with ordinary import resolution, as in the JavaScript call graphs of Feldthaus et al. [23].

Redirects do not transfer, and the asymmetry was found by porting the model's reading rather than by arguing about it. Angular's validator refuses a route that declares `redirectTo` beside `children` or a `component`, so a redirecting Angular route has neither a layout nor a component to lose and the model's step from "redirects" to "renders nothing" costs nothing. Vue Router applies a record's redirect only when that record is the last one matched, so a redirecting parent with children does render its component for a matching child. Carrying the Angular reading into the Vue extractor dropped those components; the extractor now keeps the component of a redirecting record that has children. It declares one residue as a note at the parent rather than modelling it as an edge: navigating to the parent's own address lands on the redirect target with that component unmounted. The same porting exposed a second Vue-only hazard with no Angular counterpart: a table that mixes the Vue 2 catch-all `'*'` with the Vue 3 form `'/:pathMatch(.*)*'` folds both to one address, and the extractor keeps the record declared first; where that is the Vue 2 form, which Vue Router 4 rejects when the router is created, the surviving component is the wrong one. The collision is declared, so the graph says that a record was dropped at that address; which of the two it should have kept is not something the table alone determines.

Guards move to different scopes. A redirect thrown or returned from a React Router loader runs when its route is entered, during the navigation that is trying to reach it, and again when its params or the search change or after an action revalidates it; a parent whose match is unchanged is by default not revalidated on a navigation between its children, the analogue of Angular's route reuse. It plays the role of `canActivate`, and as for an Angular entry guard its source is the screen being left, not the loader's route: a navigation from `/public` to `/admin` that the loader redirects to `/login` leaves `/public`. Its scope is therefore the join of the origins of the navigations that may run the loader, as the relation T gives for entry guards, and not $\mathit{subtree}(r)$; where those origins are not recovered, the conservative scope is $\top$. Vue's global `beforeEach` guard corresponds most closely to a `canActivateChild` on a root route with no component; a redirect from it is anchored at $\top$, which the lattice already contains.

Component nesting is harder in React. Angular and Vue keep templates in a separate language, so the nesting relation can be read from element names. In React, components are created in JSX, which is ordinary JavaScript. A component can be stored in a variable, passed as a property or as `children`, and rendered under any condition the language can express. Recovering the nesting relation then becomes a data flow problem in its own right, closer to points-to analysis than to reading a template. This is the main place where a React version of the analysis would need

new work.

The reactive boundary is the same. A navigation inside `useEffect` or a Vue watcher fires when its dependencies change, so, like a signal effect in Angular, it would be given modality *unknown*.

The lattice, the fixpoint and the anchored representation transfer unchanged. Of the input relations, outlets and guards transfer with small changes, the call graph becomes simpler, and component nesting becomes harder in React. These are expectations derived from the routers' documented behaviour. Whether they hold on real applications would need a corpus and the same evaluation, which is left for future work.

6.5 Future work

An oracle for the screen set on a *large* application is now the cheapest addition that would strengthen the thesis most. The hand-built ground truth of Section 5.3.4 supplies one for the smallest application in the corpus, and it was worth building: it convicted the class-level service join of attributing a site to 9 screens it cannot fire from, which no other method here could do, and the call graph that replaced the join was written in answer. What it cannot do is carry to an application of six hundred screens, where reading every site by hand is not practical and where most of the corpus's (site, screen) attributions are made (Table 5.11); the scopes widest relative to their application are k8sdash's, where the average site is attributed to 59 % of the screens, and no reading by hand covers those either. Driving such an application into a signed-in state and reading its rendered links, or annotating a stratified sample of its sites rather than all of them, are the two ways to get there. Recording, for each attributed site, the relation that supplied its scope would narrow the verdict further: the ground truth here shows which *sites* are wrong, and that would show which *rule* made them wrong without having to re-extract once per rule.

The two defects that capped the larger applications were the most urgent, because nothing else in the analysis could be evaluated properly while they stood. Both were repaired in the course of answering RQ5, so what follows describes what the work was rather than what remains: resolving constant-valued route paths means following a path expression to its definition and folding it, which is a local analysis over the route tables and turned the application that had collapsed to a single node into one with a node for every declared screen. Following lazy boundaries means resolving a `loadChildren` or `loadComponent` specifier to the file it names and continuing route extraction there, including through a routing module that a lazily loaded module imports. The independent route parser follows lazy boundaries after its correction (Section 4.3.2), and it finds 84 declared screens in ghostfolio, where every route path is a constant reference (Table 5.2), so the work is known to be feasible on this corpus, and the parser's output gives the expected result to check against. The gain is not limited to the nodes recovered. Some of the attribution losses that remain, most of them in ghostfolio (Table 5.9), are constructs whose scope was found but whose target screen has no node because no reader can address it, and those would be

repaired by reading more of the route configuration rather than by any change to the attribution pass.

The part of the model that Chapter 3 specifies and the prototype leaves out comes next: the dispatch relations that carry NgRx effects and router event subscriptions, and the injector hierarchy that the implemented call graph reads past, since it takes every provider binding as applying application-wide (Section 6.3). The call graph itself follows class hierarchy analysis against Angular's provider declarations, in the style the framework-modelling literature recommends [17, 68, 3]; reading bindings relative to the injector that resolves them would narrow the scopes it computes, and the fixpoint and the complexity bound already cover it. Guards that return a `UrlTree` belong to the same group, since a redirecting guard is itself a navigation site and not only a label.

The witness check exposed a defect in how the guard set of a screen was assembled from the route chain: the assembly ignored a guard carried by a route a wrapper function returns, which is the construct through which one application in the corpus routes every one of its feature tables. Reading it is what took that application's deterministic tier down to 1 on the round's instrument, and took the corpus tier from 26 witness-check failures to 0. Under the router's own guard semantics, which came later, the same guard no longer gates navigations between the screens below it, and mifos holds 15 must edges again. The demotion rules that followed found that most of the edges that passed did not occur on every input either (Section 5.5). The mutation operator that removes a guard is the obvious second test for it, and on this corpus it reaches only the guards the graph already records on an edge (Section 5.4). The wider treatment of guards in Chapter 3, where the guard of an edge is the conjunction of the conditions on the containment path with the guards the router runs on that navigation, is the more substantial piece of work and would subsume it.

Dialogs and overlays should become user interface states. They are the largest declared approximation in structural terms, they strand entire components rather than single call sites, and the model needs only a second kind of node together with edges that open and dismiss it. The route-derived definition of the screen set would have to be widened, which is the reason this is not a small change to the adapter.

The runtime tier is the complement to the whole static approach rather than an extension of it. The model already admits observation identifiers as witnesses alongside source locations, so an observed transition can enter the graph without being confused with a derived one. Setting that tier against the dynamic lineage, where the state flow graph comes from firing events on a running application [51, 52, 75], would answer the question this thesis cannot: how much of what a crawler finds the static graph already contains, and how much of what the static graph contains a crawler never reaches. The Android work has the same structure, combining static analysis with runtime traces where the static window transition graph is imprecise [46].

The corpus needs to grow in three directions. An automated search pipeline over a larger pool of repositories, with the published filters for repository quality applied mechanically [35, 36], would replace a hand-screened set of ten candidates and would weaken the tool-driven selection bias of Section 6.2, provided that applications the

extractor handles badly are kept in the corpus rather than rejected. A case study on a proprietary industrial application, reporting only aggregate figures, is the most direct answer to the external validity threat, since an open-source corpus cannot show enterprise routing style. A minimum density of user interface tests, rather than the presence of test configuration that criterion (4) of Section 4.1 asked for, should become an explicit selection criterion whenever execution-based ground truth is needed, since the candidate pool showed that this density falls as applications grow, and without such a criterion a study ends up with its oracle and its scale on different applications.

A React or Vue adapter, evaluated with the same harness on a corpus of the same shape, would turn Section 6.4 from an argument into a result. The predictions are specific enough to be falsified: outlets and guards should transfer with small changes, the call graph should become easier because a navigation function is obtained through a hook or a composable, and component nesting should become harder in React because JSX makes the nesting relation a dataflow problem.

The defect class "node not linked to its component" still has one confirmation, from the comparison between the route parser and the graph, and the rule adopted in Chapter 5 admits a class only with two. A second method has to reach it independently, and the test-source cross-check is the natural candidate, since a screen whose node exists but carries no component link should still be visited by tests while contributing no outbound edges. Until then the class stays a candidate, and the share of the corpus total that it accounts for stays provisional with it.

The use that motivated the thesis is still unmeasured. Agents that drive a user interface or a repository work without a navigation model: mobile testing agents pass the current screen to a model and replay the action it returns [47, 73], application-specific knowledge for them is obtained by dynamic exploration [74, 72], and coding agents navigate a repository through an interface whose design measurably changes their performance [32, 77, 82]. Whether a graph of screens and guarded transitions, recomputed from source and carrying a witness on every edge, improves either kind of agent, or improves automated test generation, is a controlled experiment that this thesis does not attempt. The graph and the harness are the artefacts such an experiment would need, and the defects quantified in Chapter 5 are what it would have to control for, since a model that is missing half of an application's screens would confound the comparison.

Chapter 7

Conclusion

This thesis set out to measure how much of the navigation structure of an Angular application can be recovered from its source code without running the application, to explain where that recovery fails, and to repair the largest class of failure with an analysis that attributes a navigation site to the screens from which it can fire. The selection criteria for the corpus were fixed before any results were obtained, though one candidate was rejected on tool support and two retained applications do not meet the test-suite criterion. The protocol was corrected eleven times while it was being developed, including the definition of the conversion ratio and of its denominator (Section 4.3.2), and the result tables of Chapter 5 come from the harness run on pinned commits.

The answer to RQ1 is that the loss happens at two levels: node extraction is now nearly complete, whereas almost no navigation construct becomes a deterministic transition. In all 6 applications every screen the static readers find declared has a node, and the runtime method observes no node placed at an address where the framework mounts nothing; that shows no such error was observed, not that none can occur. That completeness is over the route set a static reader can read, and not over the addresses the framework mounts: the same runtime method finds 251 mounted addresses with no node at all, 242 of them behind one spread of computed routes in ghostfolio that the extractor declares unread, so only 2 of the six are complete against the runtime configuration. Below that level almost no navigation construct becomes a deterministic transition once the demotion rules Claim 3.1 rests on are applied: 21 constructs without attribution and 25 with it (two recoveries on realworld and two k8s dashboard constructs raised from may to must), in 4 of the six applications and none on the other 2. Most of the tier's edges are one application's toolbar: 2 constructs there yield 125 of the 150 edges, and the rest of the corpus holds 25 on 23 constructs. 126 of the edges also depend on an assumption about route-table order that the extractor does not read; without it the tier holds 24 edges on 22 constructs. Evaluating guards would widen it: with every guard taken to let the navigation through, the tier would hold 194 edges on 69 constructs, of which only 44 edges run a guard. Node extraction was the bottleneck when this work began and no longer is; Section 5.5 records what repairing it took.

The answer to RQ2 is that the failures fall into five defect classes, each with at least two lines of evidence of which at least one is a named evaluation method; for four of them, including attribution, the second line is an observation rather than

a second instrument (Table 5.3), so for those four the two-line requirement is a weaker control than the count suggests. There is also one candidate class that has a single confirmation. Assigning every lost construct to exactly one class, the first whose rule matches, makes attribution the largest class in every application of the corpus once the node defects are repaired; a different order of the rules would move constructs between classes, so that ranking holds under the order the loss rules state (Section 6.2).

The answer to RQ3 is that scope attribution recovers 292 of the 454 constructs the loss rules assign to it, 64.3 % of that class, with some recovered on every application in the corpus, and roughly doubles the graph, from 5919 edges to 12172 on the current graphs; and that almost all of those conversions are navigation the graph reports as possible rather than certain. After the fourth correction of Section 5.5 the attributed run holds 150 must edges against 17 without attribution. The difference is the links of two application shells (realworld's header and footer logos and k8sdash's toolbar), drawn from every screen the shell renders. The witness check finds 0 failures among the must edges and cannot read the destination of 62, all k8s dashboard toolbar edges added by attribution, the same edges that rest on the shell scope and on route order; it reads no guard, so an edge that should be conditional would pass it. The tier is too small for that to be a statement about precision in general. Separately, no construct the baseline converted lost its conversion when attribution was enabled (0), and none was moved from must to may (0); that is retention of what the baseline represented, not precision of what attribution added. The computed screen sets were checked in both directions on one application and not on the corpus: read site by site on realworld, the analysis names 100.0 % of the (site, screen) pairs of the hand-read structural reference with no pair outside it, a reference partly completed from the graph's output, after the one over-approximating rule that reading exposed had been replaced by the call graph and the route reader corrected where it had shown one screen's attributions missing. Elsewhere the running interface puts half of that question to 5 of the six applications, of which two rendered any judged link, and nothing here puts it to ghostfolio. The scopes are also wide: narrower than attributing every site to every screen by a factor between 1.7 and 18.4, and on k8sdash the average site is attributed to 59 % of the screens. A scope that wide says little about where the site fires, even if it is sound. RQ3 is therefore answered in part. Coverage rises and the baseline's conversions are retained. Whether must-tier precision holds is not established: the check finds no failure among the edges it can read, but it cannot read the destination of 62 of the 150, and whether the scopes attribution adds are too wide was tested on realworld alone.

The answer to RQ4, on the three applications that were mutated, is that the graph responds to authored source changes in the constructs it represents. Of the 45 mutants from the three operators that can fail, 45 are detected, removed guards 11 of 11 among them; a deleted `routerLink` is detected almost by construction, so the evidence rests mainly on the retargets and guard removals, and all 6 exclusions are guard mutants, so 11 of the 17 sampled are scored. The rename operator, close to guaranteed by construction, brings the total to 67 of 67. Without attribution 50 of 51 are detected. Those two sets are not the same, because a change to a construct the baseline graph holds no edge for cannot be scored against it; on the 51 mutants both

graphs can score the figures are 51 and 50, and the one disagreement is a limit of the harness, not of the graph. Most of attribution's effect on this experiment is therefore the widening of what can be scored at all; a gain in sensitivity on what both graphs represent is not shown. Responding to the measured changes is a property change impact analysis needs, not a measure of its accuracy: the experiment does not ask whether the graph names every affected screen, how many it names in excess, or how well it would select tests. The 6 mutants that cannot be scored are guards the graph can record on no edge: for 5 of them no edge enters the screen, which is RQ1's coverage gap, and 1 guards a route that yields no screen. The samples are small, and the full detection was reached only after the undetected mutants of the previous run were read and found to be errors of the harness.

The thesis contributes a measurement and the apparatus that makes it checkable, and a mechanism whose specification outruns its evidence. The mechanism is scope attribution, specified as a dataflow analysis over the powerset lattice of screen sets and implemented over three of its four input relations, with a soundness property for the deterministic tier that is stated and mechanically tested, though not proved, over the declared route set and paired with an explicit list of the constructs the analysis does not decide (Claim 3.1, Section 6.3). It recovers 64.3 % of the loss class it addresses, almost all of it onto the uncertain tier, and against the structural reference read by hand on one application its full step, the call graph with its fallback join and stand-ins, finds the one pair (of 188) that a reimplementation of earlier work's mechanisms misses, taking recall from 99.5 % to 100.0 % with no pair outside the reference. That result comes after two repairs the same reading prompted and a completion of the reference from the graph's output, and it was measured on realworld alone. Nothing in its formulation depends on a particular extractor; what an extractor supplies is the screens and the navigation sites it ranges over, and Chapter 5 shows that what the mechanism can recover is bounded by how completely those are read. The measurement is the part that stands on its own. Static navigation extraction is measured on a corpus of six Angular applications pinned to fixed commits, with five methods. Four of them read the applications' sources: a witness check that re-parses the source at each claimed location, a differential test against an independently written route parser, a cross-check against the URL literals in the applications' own end-to-end specifications, and a mutation experiment whose ground truth is the authored change. That shared footing is also their common weakness, since two readers of the same text can be wrong in the same way. The fifth reads the route configuration the framework itself holds at run time, and was added because the first four can agree on a wrong address. The failures are classified into defect classes that are admitted only on two lines of evidence, at least one from a named method, and every construct the extractor loses is assigned to exactly one class, which makes the classes comparable within an application and across the corpus. The corpus and the harness are reusable artefacts: the evaluation re-runs with two scripts against pinned commits of the extractor and of every application, and the conversion, coverage, loss, remaining-loss, before-and-after and mutation tables of Chapter 5 are generated from their result files. The few figures that compare the present instrument with the one the work began with are generated too, but from that earlier commit's own results read out of version control, since re-running the extractor now would measure

a different tool. The defect-class table, the oracle table and the inspection of the unreached components are written by hand.

What the measurements establish is that a declared route table is a usable starting point for static navigation extraction and not a sufficient one. Node extraction is a precondition for attribution, since a construct whose source screen has no node cannot be attributed; with node extraction repaired, the remaining loss is mostly attribution, and scope attribution recovers every attribution loss on realworld and most of mifos's, and under half of them on the other four, where owners no screen reaches, targets only the run time knows and services the call graph does not place remain (Table 5.9). Where a construct is in the graph, every scored change to it was visible in the graph, for the operators sampled on three applications and with the deletions near-guaranteed (45 of 45 fallible mutants), which is a property that change impact analysis and commit-time checking need; where it is not, a change to it cannot be scored, and the excluded mutants of RQ4 restate the coverage gap RQ1 measures.

RQ5 asked whether the losses belong to the applications or to the tool, and the answer is that almost every loss belongs to the tool. The causes fall into three groups: defects of the extractor that were repaired, a construct whose information is in the source but which the extractor does not yet support, and information that is fixed only at run time or lies outside the route abstraction. One defect class does not: ghostfolio computes a route's path in a custom `UrlMatcher`, which no static reader of a route table can resolve. The runtime method adds a second case outside a static reader's reach, mifos's choice between two route tables by a value injected at deployment; both stay declared blind spots rather than repairs. The second group has one member, and it is the largest single loss of nodes: ghostfolio's spread of computed routes, a construct the tool could read and does not, which the extractor declares unread and the round did not repair. The first group holds every other defect class, each a construct the extractor could read and did not: a route table reached through an NgModule, a route factory, a path named by a constant, an enum or an i18n segment, a workspace alias, a loader that references its module instead of importing it. With those repaired, 6 of the six applications hold a node for every screen the static readers find declared, 2 hold one for every mounted address in the runtime configuration (the runtime method finds 251 mounted addresses with no node, 242 of them in one application), and 98.8 % of the navigation sites that remain in scope are accounted for: recovered as a transition, recorded as a destination only the run time knows, or anchored at a source the analysis cannot place. 75.7 % of those sites are mifos's, so the corpus figure is close to one application's. The sites that remain each name a reason, enforced by a check that fails when any does not, though the check enforces that a reason is recorded and not that it is correct, and some name a route the application never declares. The repairs also had an unwelcome cost, which is reported with them. The earlier instrument published a deterministic tier of 1024 edges with 26 witness-check failures in it, and four corrections changed it to 150 edges, each of the first three defects masked by the one before it: a guard carried by a route a wrapper function returns; a `canActivate` element that is a call rather than a name; an edge that several constructs cite taking the modality of the first of them; and gaps in the demotion rules, namely a navigation that runs

under a condition, guards the reader did not read (one on an empty-path child, `canMatch`, `canActivateChild`), and a navigation to the screen already displayed. The first three left 20 edges with 0 witness-check failures, as the round's tables printed them, which is not the same as satisfying the claim: the fourth, found by reading every construct the tier cited rather than by the check, kept a construct that earns must in only 2 of them, and 1 more stayed must only because a construct it had not cited yields it (Section 5.5). The others did not become correct targets; they stopped claiming to be deterministic, which they had no evidence for. The fourth correction also replaced the third's merge rule, which had demoted an edge whenever any construct citing it was uncertain: an edge is now as certain as the most certain construct that yields it, and that is why realworld's Global Feed and logo links remain. Its guard rule, first written to gate an edge on every guard its destination carries, was corrected again to the guards the router runs on that navigation, and the rest of the current tier needs it: the logo link and Create button of k8sdash's toolbar, drawn from every screen under its shell but the root, the destination itself and the screens whose `canDeactivate` runs; the cancel and back buttons and the system menu of mifos; and a cancel button in bitwarden, each moving between screens under one guarded parent. The k8sdash edges need the shell attribution as well as this rule, and how much either contributes alone is not measured. On 2 of the six applications there is no must edge, so RQ1's stricter measure reads zero there, and Claim 3.1 ranges over 150 edges in the corpus, with 0 witness-check failures and 62 destinations the check cannot read, every construct behind them read against the source by the AI assistant (Section 5.5), which is not an independent proof of the claim; the two readings of the round's tier are two runs of one model, so their agreement measures consistency, not correctness. What the small tier measures is what the extractor can assert without evaluating a guard: taking every guard to let the navigation through gives 194 must edges, and for 40 of the 44 that run a guard, every guard the navigation runs is one the source screen's own routes carry, so the user has passed it at least once on the way to that screen. That trade, a small tier in place of unsupported claims, is the end of a sequence in which each claim that did not survive its own test was withdrawn. The size of the tier is bound to the choices that produced it: a navigation must occur on every execution the claim ranges over, the demotion rules decide which uncertainty removes it, and the approximations of Section 6.3 decide what the extractor reads. Under another semantics, demotion policy or extractor the tier would be another size, so the result is not a limit of static navigation analysis in general.

What they leave open is also specific. The dispatch relation is specified and not implemented, and the call graph is implemented without the injector hierarchy, reading every provider binding as if it applied everywhere. Where the model has no implemented relation, approximations stand in: a guard is scoped to the routes that name it, an effect to $\top$, and a service member no caller reaches to the join over the components that inject its class. This is coarser than the model and leaves a class nothing reaches with no scope at all. One defect class still waits for its second confirmation. The repairs were found on this corpus and fitted to it, and no held-out application measures how they generalise. The main gap is in the evaluation rather than the analysis. Asking the running applications which links

they render, on an earlier graph than the final call graph, found 29 of the 33 rendered (screen, pattern) pairs among the destinations of the edges leaving each screen, which checks pooled destinations and not each site's scope, and shows the 4 it misses to be sites attribution declined to place rather than sites it placed wrongly. What that cannot do is convict a scope of being too large: only 7 of the predictions at those screens name a destination exactly, so the direction is close to untestable by this method. Reading one application's sites by hand did convict one: the rule that joined a navigating service over its injectors asserted 9 pairs outside that application's structural reference, all at one site. The call graph that replaced the rule asserts none there, but it was written with that site in view, so the reading that convicted the old rule cannot acquit the new one; the levels that stand for earlier work's reach recover 94.7 % and 99.5 % of the reference pairs while asserting none outside it, against 100.0 % for this analysis. The runtime method judged 30 screens of 806, because without a backend the rest redirect to a sign-in page. Driving the applications into a signed-in state is the work that would decide the other half. Whether a graph of screens and guarded transitions, recomputed from source and carrying a witness on every edge, helps the consumers that motivated the work, agents that drive a user interface or a repository and automated test generation, is not measured here; the extractor, the corpus and the harness are what such a measurement would start from, and the defects quantified in Chapter 5 are what it would have to control for.

Appendix A

Validating the measurement

This appendix holds the parts of the evaluation that establish what the measuring instruments can and cannot see. Nothing here is a result of the analysis; it is the record that the figures of Chapter 5 were produced by instruments whose own failures were looked for and found.

A.1 Nodes the two static readers disagreed about

Table 5.2 also reports the other direction of the same comparison, which the differential test produces and which an earlier version of the results chapter left out. Every figure in this section is the first round's, measured with the parser as it stood then; Section 5.5 repeats the comparison with both tools corrected and the numbers change considerably. As the first round measured it, the parser holds no declared screen for 197 of the 338 nodes in the corpus, so node extraction is inaccurate as well as incomplete. For k8s dashboard the figure is almost entirely the parser's own limit, 37 of its 38 unmatched nodes being the lazily loaded entries it cannot follow. Mifos is not explained that way: it has one blind spot and 120 unmatched nodes. Table A.1 decomposes all 197 by the kinds of route the parser found at the node's path, joined on the parser's own normalisation.

Three groups come out of it. For 43 nodes the parser could not follow the route that would have declared them, so those are the oracle's limit and not the extractor's, and 37 of the 43 are k8s dashboard. For 11 nodes the parser did find a route, but one it classifies as a path prefix or a redirect. Section 3.1 admits only component-bearing routes into the screen set, so the implemented node set contains 11 elements the model excludes: one in realworld, two in alfresco, six in bitwarden and two in mifos. An example is `/profile` in realworld, whose route declares `loadChildren` and no component. These nodes make $screen(r)$ answerable for a route the model treats as a path prefix only.

The remaining 143 are the substantial group and have no declared route of any kind at the node's path. They are concentrated in mifos, which has 117 of them against a single parser blind spot, and in bitwarden, which has 23. Either the extractor emits nodes for paths it assembles differently from the parser, or the parser misses route tables it does not report as blind spots. The two methods disagree here without either being able to adjudicate, and at the close of the first round it was

Table A.1: Why each unmatched node has no declared screen, as the first round measured it (Table 4.4). A node is placed in the first row that applies, and the categories are exclusive.

Why the node has no declared screen	rw	alf	gf	k8s	bw	mif	Total
No route declared at that path	0	1	1	1	23	117	143
Parser could not follow the route	0	3	0	37	2	1	43
Route is a prefix or a redirect, so outside S	1	2	0	0	6	2	11
Declared screen, so a join artefact	0	0	0	0	0	0	0
Unmatched nodes	1	6	1	38	31	120	197

the largest single piece of unexplained behaviour the evaluation had found, and it underlay every figure that counts nodes.

The second round resolved it. For almost all of these nodes the first of the two explanations held, which is the opposite of what an earlier draft concluded. 138 of the 143 sit at an address that is the tail of a route the parser had already declared in full: the extractor had reached the table without its parent prefix and minted `/premium` where the parser declared `/settings/subscription/premium`. That is the extractor assembling the path differently, not the parser missing a table. The other 5 are the catch-all each of those five applications declares, `path: '**'` with a component, which the extractor mints at the path `*` while the oracle's policy keeps it out of the screen set: there the two tools disagree by convention, not over a table either of them missed. The direction the group fell in matters beyond the bookkeeping, because it decides whether the measuring instrument or the tool was wrong. The repairs of Section 5.5 are accordingly almost all in the extractor. The figures above are the first round's and are kept as the record of what that round saw. Section 5.5 repeats the comparison with both tools corrected. The corpus is then left with 2 nodes that no declared screen matches, of which 2 sit at the path `*`, where the extractor mints the catch-all the application declares and the oracle counts no screen. The parser had the smaller share of the problem, counted in Section 5.5: a route array behind a module it refused to follow, a path assembled from a constant it did not resolve, a lazily loaded table walked without its parent. Reporting the disagreement rather than attributing it was the right decision at the time, and investigating it produced most of Section 5.5.

On the instrument the figures of Chapter 5 are measured on, this disagreement is gone: the parser declares a screen for every node the extractor mints except the wildcard catch-all route, which it does not count as a screen, and the runtime method confirms that the framework mounts a component at every address the extractor claims. This section therefore records a past state rather than a standing defect: the state the differential test was in when it was the only check available, and the reason a third, independent reader was added.

A.2 How the declared-screen set moved

Table 5.16 gives node coverage before and after. 6 of the six applications now hold a node for every declared screen. The denominator of those ratios deserves more than a column of its own, because on this corpus the declared-screen *count* turns out to be a poor summary of the declared-screen *set*: between the oracle as it stood before this round and the oracle reported here, 5 of the six applications' declared sets changed, and 1 of those changed while the count stood still. A moving denominator can distort a coverage figure without anyone intending it. A denominator that keeps its size while every address inside it changes can do the same without the count column changing. Each of the five is accounted for below from the generated subtraction rather than from a hand-written statement.

In ghostfolio the change could not have been noticed from the count. Its count did not move: 84 declared screens before, 84 after. Every address in it changed. 84 paths left the set and 84 arrived, because the earlier oracle recorded the verbatim expression a route's path was written as rather than the address it resolves to. It stored `/internalRoutes.account.path`, where the application mounts `/about` and its siblings. A reader comparing the two counts would have concluded, wrongly, that nothing about ghostfolio's route set had changed.

k8s dashboard carries the largest move, and it was missing from this account until the provenance of the comparison was fixed. It went from 2 declared screens to 67, with 65 arriving and none leaving. The earlier oracle reported 37 blind spots for this application, each of them a `loadChildren` target it could not resolve. It therefore declared the two screens its two eagerly registered tables carry (`/login` and the application root) and nothing beneath any lazy module. The oracle reported here declares the whole tree with no blind spot. The earlier figure was a near-vacuous denominator, so coverage against it was 100 % of almost nothing. That is a fact about the measuring instrument and not about the application.

bitwarden went from 97 declared screens to 98. 3 were Storybook fixtures, throwaway tables such as `/vault` and `/sm/:organizationId` that exist only to render one component in isolation; neither tool reads them now, and the oracle reports how many files it set aside. 40 were inside the application. 26 of those left the count for ghostfolio's reason: they were recorded under a verbatim expression rather than an address. The earlier run stored `/AuthRoute.Login`, for instance, because the enum behind it was unreadable. These screens leave that count and arrive in this one at the addresses they actually mount. The rest are the mifos defect in miniature: the earlier run read their path segments correctly but placed them at the project root, as `/password` and `/session-timeout`, because it could not follow the lazy table that carries them; they are declared in this run at the addresses the application mounts, under `/settings/security/` and `/vault/`. These are the same screens at a different address rather than screens gained or lost. A count of the denominator cannot make that distinction; a diff of the set can. 44 arrived, from two causes: a route table wrapped in `Object.freeze` that both tools learned to read, and the report and member subtrees, which were declared at the application root until Section A.5 showed the running router mounting them elsewhere. Against the

98 screens that remain, the graph holds a node for every one.

An earlier draft reported bitwarden against 159 declared screens and decomposed that figure in detail. This paragraph replaces that decomposition, which was wrong in every part. The 159 came from a file produced by a revision of the oracle from the middle of this round, dropped into the directory the tables are generated from and read as though it were the earlier oracle's output. Nothing in the generator could detect this: the file is internally consistent, as every revision's output is, and an absent file was silently read as an application whose denominator did not move. The comparison is now made against the earlier oracle's own committed output, read from the repository rather than from the directory the run writes into, so the revision each column belongs to is a fact about the repository and not about what a working directory happened to hold.

The two remaining moves are small and are printed for the same reason. alfresco went from 25 declared screens to 26: 0 left the count and 1 arrived, because of the workspace-alias repair rather than because anything was set aside. mifos went from 610 to 605, with 8 paths leaving the count and 3 arriving: the routes of its reporting module, which the earlier run placed at the project root as `/`, `/:filter` and `/run/:name` because it would not follow a `baseUrl` import, now sit at `/reports` and beneath it. Fictitious addresses left the set and real ones took their place, and because more left than arrived the denominator moved down. The net change is too small to move any percentage in Chapter 5, and that is why it is printed. A denominator that moves without being mentioned is how the previous figure survived three rounds of review, and the only defence that generalises is to generate every part of the subtraction so that it can be checked.

One caveat of Chapter 4 can now be quantified. The oracle and the extractor agree about an edge only where both read the same source, and 636 edges across 3 applications (527 of them in bitwarden, 0 of those on the deterministic tier) have their only witness in a file outside the application directory. The check records that the witness lies outside but not the route by which the file came to be read, so the figure is reported here without an explanation. The files are of the kind a monorepo puts beside an application: a shared library and, in bitwarden, the repository's other front ends. Both tools read those files, so their agreeing there is not independent evidence of anything; the witness check reports the figure rather than excluding the edges. The edges are kept because a precision figure is a claim about the edges this tool emitted, wherever it read them, and a reader who wants the figure over the measured directory alone can subtract the count. They are not kept on the grounds that they are clicks in the application measured: the files in bitwarden belong to a sibling front end built from the same repository, and whether its components are reached in the application under measurement is not established here. The dead-link census of Section 5.6 makes the opposite decision for the opposite reason. Calling a link dead asserts something about an application, and such a claim needs the link to be reached in the application being measured, so there the boundary decides and not the count.

The reason to distrust that last sentence lies in the mechanism rather than in this application, and this round supplied an illustration. Three of bitwarden's declared

screens were, for most of this round, outside the denominator entirely. Their paths are assembled inside a template literal from an exported constant. Neither tool could read them, although resolving that one exported constant would have been enough, and a denominator counted without them gave this application a coverage of 100 %. Both tools read them now and the figure is correct, but the figure itself gave no sign of the earlier error: a coverage ratio cannot tell a screen neither tool can address from a screen that does not exist. The only defence is to report what was set aside, which is what the blind-spots column is for and why it is printed beside every coverage figure in Chapter 5 rather than in a footnote.

That so many reach every declared screen is a weaker result than it sounds, because of what the blind-spots column counts: the routes a tool knows are there and cannot address. Reaching every screen one can see shows that the walk finds the route tables, not that the graph is complete. The accounting below covers what the graph does with the navigation.

A.3 Every remaining site names a reason

21 sites remain unexplained, and Table A.2 gives the reason for each. The reasons are produced by a script that exits non-zero if any site reaches the end of it without one, so a site the analysis cannot explain is a failing state rather than a silent remainder. The largest group, 10 of the 21, reflects the attribution gap of Chapter 3. Each is a class the extractor parsed and found a navigation site in but could place under no screen, because no relation the implementation builds reaches it. Either the call graph of Section 3.2.3 resolves no call into it and no component it reads injects it, or it is an effect the stand-in for the unimplemented dispatch relation does not cover (Section 6.3). The reason given is the extractor's own note and not this harness's inference. This matters because the alternative wording, that nothing in the application injects the class, would be a claim about the application, whereas the correct claim is about the tool. A second group concerns a target the extractor read and could not place: 4 sites whose target matched no route. They are split by what can be claimed of each: 0 a relative target with no base to resolve against, 0 inside a subtree one of the tools could not read, none in an application whose route set is a lower bound, and 4 that nothing accounts for but the application. Each is decided here by testing the target against the independent oracle's route set, pattern against pattern, and not by repeating whatever note the extractor had left at the site: the note says no node of the extractor's matched, which is a claim about the extractor.

A.4 The four dead-link corrections

Four corrections belong with this finding, and all four came from attempts to refute it. Three of them are the same mistake made in three places, and that repetition is the more useful result.

A target that no route declares does not necessarily go nowhere. An application with a ** route catches every unmatched URL, and it does so in one of two ways: a

Table A.2: Why each remaining site is not a transition. The script that produces this table fails if any site has no reason.

Sites	Reason the site is not a transition
10	A class the extractor could not place under any screen
6	Over-approximated at the site
4	The target names no route the application declares
1	A service; whether anything injects it is not established
21	All, none of them without a reason

`redirectTo` forwards the URL to another screen, and a `component` renders one. The extractor follows both, holds the resulting edge at may, and records a note saying the target matched nothing and where the catch-all sent it. The third round followed only the redirecting flavour. Neither application the surviving count comes from uses that flavour, so the correction held for the one application it had been written against and failed for the two that needed it.

Resolving the target and calling the link dead are separate claims, and the first version of this finding went wrong by conflating them. bitwarden declares its authentication paths through an enum the extractor had failed to read, so targets such as `/login` matched nothing for a reason that was the extractor's, and the wildcard then forwarded every one of them and raised that application's recovered figure by eight points. That enum is read now, and so is the wrapper call that holds it, so those paths resolve rather than being forwarded; what remains is the mechanism. An earlier version of this paragraph named three bitwarden paths assembled inside a template literal instead; those are read now, which is what moved its denominator, and what is left there is a boundary rather than a path. The note says which of the two it is, because a dropped subtree and an unreadable path are not the same loss. Angular does forward those URLs, so the edge is drawn either way and what had to change was the claim attached to it. The extractor counts the declared paths it could not read and records one note per application saying the route set is a lower bound, and that caveat is written into every wildcard note from that application.

This round changed what the caveat does to a candidate. A single unreadable declaration used to void every claim of this kind across the whole application, which is severe and was deliberate: the alternative is a census that reports an extractor's blind spot as an application's defect. It is also a harder question than the census asks. Whether an application's whole route set was read is one question; whether *this* target is served is another, and the runtime check of Section A.5 answers the second directly, because the configuration it reads is the one the framework holds. Each caveated candidate is therefore put to that configuration. 24 are withdrawn because the framework serves a screen at the target, which makes them a measurement of what the extractor failed to read rather than of what the application got wrong; 19 are withdrawn because the target is matched by a route that places no screen of its own, and 0 because it is answered by a redirect. 0 sites keep the caveat. A matcher route leaves undecided only what it could claim, namely the addresses under its own prefix. Reading it as leaving the whole application undecided withheld twelve

findings on a ground the harness's own navigation refutes: ghostfolio's matcher sits at `/admin` and reaches nothing above it. Where the configuration still declines, a navigation is asked instead: ghostfolio's `/admin/users` is behind that matcher, and the recogniser places a component there, so the three sites naming it are withdrawn rather than left undecided. What survives is a candidate whose target the framework answers with nothing but its catch-all.

From a navigation site, a route neither tool can read is indistinguishable from a route the application never declares. This thesis credited the difference to the application three times before the mechanism was general enough to stop it. A route array built by a spread (`children: [...adminRoutes]`) was not followed, so the routes it carries were minted at the project root and links to them matched nothing. ghostfolio mounts its user administration screen with Angular's `matcher` property rather than `path`:

```
{ component: GfAdminUsersComponent,
matcher: usersMatcher, ...}
```

where `usersMatcher` consumes a URL whose first segment is `users`. Neither tool read the `matcher` property at all, so `/admin/users` appeared in no route set, and the previous round of this chapter published it as the one dead link it had confirmed by hand. The hand check had searched the application for `path: 'users'`, which is not how that route is spelled. alfresco declares `nodes/:nodeId` and loads its children through a tsconfig `paths` alias, which neither tool resolved, so `/nodes/:nodeId/rules` was absent from both route sets and two live links to it were reported as dead. All three are now read. The three fixes are specific to their cases, but one rule generalises: both tools now record the holes they cannot read (a matcher route, an unresolvable lazy boundary, a path expression that is not a literal), and no claim that a path is undeclared is counted inside one.

The route-set test read only the first quoted segment of a commands array, so it tested a prefix as if it were the target: `['/nodes', id, 'rules']` was tested as the path `/nodes`. alfresco declares no route at that path, but it does declare `/nodes/:nodeId/rules`. The whole array is now read as a pattern, its computed segments as holes, and matched against the declared routes pattern to pattern. That correction cuts both ways: it cleared alfresco's two candidates and it found ten more sites in mifos that the first-segment test had passed over.

After all four corrections, the 167 sites a catch-all route answers divide as follows: 0 come from an application whose route set is a lower bound and whose addresses no measurement can decide, 24 have a target the framework's own configuration serves with a screen (a defect, but the extractor's), 19 a target it matches with a route that places no screen, which could never have had a node, and 0 a target it answers with a redirect. none name a route the application declares as a screen and the extractor minted no node for, none name a route it declares as a *path prefix*, 0 address a subtree the oracle could not open, 2 are a pattern with a hole where the first segment should be and so name no single address, 18 are a relative target the analysis had no base to resolve, 12 name the application root, for 0 the record names no target at all, so nothing is established about where it points, and 69 sit outside

the directory the measurement is scoped to and are the subject of the paragraph above; 23 remain. Of the 4 unexplained sites whose target named no route, 0 are again a relative target with no base, 0 address an unreadable subtree and none come from an application whose route set is a lower bound at all; 4 remain. Those two residuals are the dead-link candidates: 27 sites at 25 places in the source, 23 distinct URLs, in 2 applications (bitwarden, mifos). The count is of sites rather than of defects: one wrong prefix written into a shared template is one mistake and several sites, and one site resolved against several owning routes is one place to go and look and several targets. Every category in the enumeration above except the last of each arm is a limit of one of the two tools rather than a defect of an application. The residual is therefore what is left once the tools no longer blame the applications for their own blind spots.

One of those categories used to be unsettled, and is now measured. A route the application declares as a path prefix answers no URL of its own unless it also declares an empty-path child, and the previous round could not say which of the sites it excused on that ground addressed a prefix the application really answers. The framework's own configuration answers this per address. Every one of the 19 sites now excused on that ground has a target the framework matches with a route that places no screen of its own. A navigation to one of them, mifos's `/collections`, activates a last route carrying no component at all, so the address answers with the application's chrome and an empty outlet. That is what the exclusion claimed from the start, and it is now a measurement rather than an inference.

The result is a candidate list rather than a verdict. The preceding four corrections are evidence that the tools' blind spots are not yet all known, and from the outside a route that both the extractor and the oracle fail to read looks the same as a route that does not exist. What can be said is narrower than the previous round claimed. Each candidate was checked against the application's own route files by hand, which is how the `appusers`, `manage-jobs` and `configurations` mismatches above are quoted; no candidate was checked by navigating to it in a running application, which is the only check that would settle one. Half of the objection has since been answered, and not by the route files. Every candidate's target is put to the route configuration the framework itself holds, read out of the running application by the check of Section A.5. The premise a dead-link claim rests on, that the application mounts nothing at that address, therefore no longer rests on two readings of the same files. The classification that premise uses was itself held against what the framework does, on 41 navigated addresses, of which it agrees with 41. That is what keeps the census from being empty: both applications it draws from declare part of their route set unread, which by the rule above would suppress every candidate they carry. What is still unchecked is the other premise, that the link is reached at run time at all, and one step below it: a target is matched against the framework's patterns rather than navigated to one address at a time, so a candidate's own URL has not been through the recogniser. What the method contributes is the list rather than a verdict: a static transition graph puts every such candidate on one page, with a file and a line each, instead of leaving them to be met at run time.

A.5 Asking the running framework

Every result so far rests on two readings of the same route files. The witness check tests a must edge's destination against the independent parser's route set, and Section 4.3 states the condition under which that test is vacuous: where both tools read a route table at the wrong prefix, they agree on a wrong address and the check passes. Agreement between two readers of one text is evidence only to the extent that the readers can differ. This section reports a third reading that does not read the text at all.

A.5.1 How the framework was asked

Each application was built in its development configuration and loaded in a headless browser. Once it had bootstrapped, the live `Router` was found through Angular's debug surface and its route configuration read directly. Every lazily loaded subtree was expanded by calling the application's own `loadChildren` and taking the routes out of whatever it resolved to: the array itself for a standalone subtree, and the route providers of the module for one that loads an `NgModule`. Nothing in the procedure parses a route file. The answer is the configuration the framework holds, produced by the application's own code, so where it disagrees with both static readers the disagreement cannot be an artefact of a reading they share.

No server was run. The method needs the application to boot, not to work: an address is counted as mounted when the router's configuration places a component there on the primary outlet, whether or not a guard would admit a visitor or a resolver would find data. All 6 applications bootstrapped and 1759 route entries were read, with 0 lazily loaded subtrees left unexpanded and 0 applications raising an uncaught error. 2 logged errors to the console, mostly failed requests to the server that is not there. This is expected and does not bear on the route configuration, since the configuration is assembled before any of those requests is made. 1 route in the corpus decides its address through a `UrlMatcher`; that route and anything beneath it have no address any of the three readers can resolve to a string, so they are outside both sides of the comparison rather than filed under the parent's address. Table 5.19 reports the result.

Reading a configuration is itself a reading, and the one this comparison depends on is which of its entries answer an address with a screen. That reading is checked against what the framework does. The harness navigates a running application to an address and records the recogniser's verdict: the event Angular emits when a URL has matched. That event fires before any `canActivate` guard runs, so such a guard sending every visitor to a sign-in page does not hide what was matched; `canLoad` and `canMatch` guards run earlier, during matching, and one that refuses removes its route from the match, so the verdict is the match those guards allow. Over 6 applications and 42 addresses, chosen to include every address this chapter names, the classification taken from the configuration agrees with the navigation on 41 of the 41 it decides, and declines to decide 1, which is the address the `UrlMatcher` route claims. The comparison of Table 5.19 does not use that classification; it decides

for itself which entries answer an address. The same navigations are therefore held against its own list as well, on the two implications that bind it: an address the framework serves with a screen it must hold, and an address answered only by the catch-all it must not. That agrees on 31 of 31. The addresses are those this chapter reasons about and not a sample of the corpus: the 251 screens the extractor does not mint are 42 navigations away from being individually checked, and are not claimed to have been. The comparison turns on two of its answers. First, an address whose route carries children places no screen of its own: a navigation to mifos's `/collections` activates a last route with no component at all, so the address answers with the application's chrome and an empty outlet. Second, an application with a catch-all answers every address, so the configuration mentioning an address is no evidence that anything is mounted there. In mifos a URL that matches nothing reaches a not-found component, and in bitwarden and kubernetes/dashboard it is redirected to a screen.

A.5.2 What the framework mounts and the extractor does not

The other direction is not symmetric with the first, and the thesis does not treat it as such. A screen the extractor does not mint costs recall; the graph says less than it could. An address it mints that nothing mounts would be a claim about an application that is false. The corpus holds 251 of the first and 0 of the second.

Of the 251, 242 are one construct in ghostfolio. Its product comparison pages are not written down: a list of competitor records is mapped to routes and spread into the table, and each path is a template string over a field of the record. The extractor reads the table, mints the parent, and declares the spread a route source it could not read.

5 are in mifos, and they are one construct as well: its users module holds two route tables, one for each identity provider the deployment may use, and chooses between them with `!environment.OIDC.oidcServerEnabled`. That constant is not fixed by the sources. It is computed from `window.env`, injected into the page at deployment time, so no reader of the code can say which of the two tables the application mounts, and the extractor declares the subtree unread. The two tables happen to declare the same five addresses with different components, and reading either would have named those five addresses correctly. The extractor used to do so by walking a table the module does not register under the prefix at all. That it arrived at the right addresses was an accident of this application: the same reading gives every prefix every table a file holds, and where two tables differ it mints the cross product. The addresses are lost here and the loss is declared, which is the trade-off this analysis is designed to make.

3 are in alfresco and no two of them share a cause. Two are outside the directory the measurement scopes to, though for different reasons: `/login` is declared by the shell application that composes the measured library, and `/view/authentication-confirmation` by an installed package that shell provides. The second is outside it because of a design choice the extractor states: it does not follow a dependency into

a package directory, because a package declares no screen of the application under measurement.

The third is inside it, and `/about` is not declared in TypeScript at all. The library ships `about.plugin.json`, whose single entry is

```
{ "id":  "app.about", "path":  "about", "layout":  "app.layout.main",
  "component":  "app.about.component" }
```

and an extension registry binds the identifier `app.about.component` to a class at run time. The route exists in data; no reader of the application's code can see it, and the extractor's analysis is over the code. The probe sees it because the framework assembles the table from the manifest before the probe reads the router. This construct and mifos's runtime-selected table are both recorded in the approximations of Section 6.3. alfresco's route set is a declared lower bound on other grounds, so no claim of completeness is made over it either way.

The remaining 1 is bitwarden's vault, whose route is produced by a helper that chooses between two components by feature flag.

One construct in ghostfolio therefore accounts for almost all of this column. This does not mean the graph is missing those screens by accident: the construct is one spread, the extractor says it could not read it, and the 242 pages behind it are generated entries that differ only in a path segment and a data key. It does mean that the cost of a declared blind spot can be far larger than the number of blind spots suggests. A count of notes cannot show this, but this comparison can.

A.5.3 Nothing is missing that was not declared

0 applications hold a screen the framework mounts and the extractor has no node for without the extractor having said that part of its route set was beyond it. Every application in Table 5.19 that has a missing screen is an application that declared one. The correspondence is per application and not per cause, which is the weaker of the two readings and the only one measured here. alfresco declares that it could not read ten spreads, but one of its 3 missing screens, `/about`, has nothing to do with a spread: it is declared in a JSON manifest and in no route file at all. The measurement supports the claim that no application is missing a screen while asserting that its route set is whole. It does not support the claim that every missing screen is behind the construct its application named. Until this measurement the thesis could assert neither.

On this corpus the correspondence happens to hold in the other direction as well: 0 applications declare part of the route set unread and turn out to be missing nothing, and 2 declare nothing and are missing nothing. That is not a property of the analysis and is not offered as one. A declaration costs the route set whatever the construct behind it happened to carry, and one that cost nothing is a tool being careful rather than a tool being wrong; mifos's declaration cost nothing until this round, when the extractor stopped reaching the addresses behind it through a table the module does not register, and the same note then stood for five lost screens. The

direction that matters is the other one: a screen missing with no declaration would be a tool asserting a completeness it does not have, and there are none of those. It also changes what the dead-link census of Section 5.6 can rest on. That census used to suppress every candidate from an application whose route set is a declared lower bound, and a per-application licence was added to lift the suppression where this comparison found the declared set complete after all. On the figures above that licence lifts the suppression for 0 applications: both applications the census draws from are missing a screen of their own (mifos 5 and bitwarden 1), so neither can be licensed. A licence of that shape asks a harder question than the census needs, namely whether a whole route set is complete, and it loses the census of any application that misses one screen anywhere. Section 5.6 therefore asks the question per candidate instead, of the same framework configuration: a link whose target the framework serves nothing at is a dead link whether or not the rest of the route set was read, and one whose target it does serve is not a dead link however the extractor read it.

When this check was first run, the correspondence clearly failed. Two applications with unread route sources were asserting complete ones: alfresco, whose document viewer is assembled by a helper call spread into the table, and bitwarden, whose vault route is built the same way. Each spread was skipped silently. Bitwarden's silence had already had a cost: with its caveat absent, that application's sites entered the dead-link census on a route set that was missing a screen. Both readers now record a spread they cannot follow as a route source they did not read. Their rules for doing so are not the same: the extractor declares the spread whose expression it cannot resolve to an array, and the independent parser declares every spread, since it follows none. The second is the more conservative of the two and neither is silent, which is what the property requires.

A.6 What a route table's evidence does not establish

One assumption underlies every approximation in the reader of route tables, and the others are instances of it. A route table that no site in the application claims (no `loadChildren` names it, no spread or `children` carries it, no registration call takes it) is read as a table the application mounts at its root. The assumption is what lets either tool find an application's routes however they are declared, and it is an over-approximation: a table registered nowhere the compiled build reaches is mounted at no address at all, and reading it at the root puts its entries into the graph as screens. It is taken deliberately, because the alternative loses the routes of every application whose root registration neither tool recognises, and the cases below are the places where it has been narrowed rather than abandoned.

Three further approximations in that reader share one shape, and they are listed together because each was found by attacking a repair of the previous one. All three are declared in the tool's own notes; none was closed, and the reason in each case is that closing it would refuse a construct the corpus actually uses.

The first concerns which file owns a lazily loaded route array. An Angular module loaded through `loadChildren` usually keeps its routes in a separate routing module that it imports, so the reader follows one import to find them. Electing that owner no longer trusts the *shape* of an array: a menu or breadcrumb configuration carrying `path` properties is never elected, and a file claimed beside an elected owner is reported as an incomplete route set rather than dropped silently. However, a `Routes`-typed array is evidence only that its author declared it to some router, not that *this* application mounts it. A sibling application's typed table, reached through a workspace alias and used as data, is therefore still elected and still minted under the lazy prefix, with a deterministic edge into it. Requiring the elected owner to lie inside the application directory closes that case and simultaneously refuses a lazily loaded route array that legitimately lives in a shared library outside it, which is a pattern the corpus does use. No corpus application distinguishes the two, so the gap is left open and declared rather than traded for a construct that is known to occur.

The second is the same boundary one step earlier: the `loadChildren` specifier itself is not bounded, so a loader whose import resolves into another project's alias root mounts that project's table under this application's prefix. The obvious guard would require the loaded file to belong to the same project root as the route that loads it. It was applied experimentally and refuses a lazily loaded module that one corpus application keeps in an alias root, turning several of its declared screens into declared blind spots and lowering its reported coverage. By path alone the analysis cannot tell a shared lazy library from another application's route table, and that is the limit.

The third is a reading, not a defect. A routing module imported by two lazily loaded modules and loaded by no route is read as a table of the project root rather than as children of both importers. The alternative reading matches Angular more closely but lets one route adopt another's table, which is how the first approximation above was reached; the conservative reading is taken, and it costs the affected routes their prefix.

A fourth was found by asking what the test that keeps a lazy route from adopting the application's root table actually read. It is reported here as a repair rather than as a limitation, because the question produced a measurement that no reading of the code would have given. The test read the candidate file's own call to `forRoot` on `RouterModule`, Angular's own discriminator for a root table. That made it sound where it fired, and it is also why it fired nowhere. An application that registers its root with the standalone `provideRouter` API is not recognised at all, and two of the six have no `forRoot` call anywhere. An application that declares its root array in one file and registers it in another, as ghostfolio and kubernetes/dashboard both do, is recognised in the registering file, which holds no route, and not in the file a lazily loaded module could adopt. Instrumenting the predicate turned the limit into a measurement. When this repair was made (commit `c0fe3b1`, before the final extractor, and not re-measured on it), it was asked 698 times across the six applications in the independent parser and 2798 times in the extractor, and answered yes on none of them. A guard that never fires is worse than no guard, because a commit message and a chapter credit it with a property nothing has shown it to

have.

Both tools now mark root tables per *array*, taken from the argument of either registration API and resolved through the name it is nearly always written as, which is what makes the question answerable in the shape applications actually write.

Several repairs of the same reader stand on constructed fixtures and on nothing else. A file holding a menu beside an imported routing module, a `forChild` given a table declared in another file, a conditional resolved inside a registration call, a root table a spread elsewhere also carries, and a wrapper that takes its own path as a parameter: each of these was reproduced, each produced either an address the application does not mount or a declared loss where nothing was lost, and each is now refused. None of the five occurs in this corpus. Every figure in Chapter 5 is therefore identical with and without them, so no result here is evidence that they work. Their evidence is the fixtures, which fail when the rule each pins is removed on its own. The same holds of the refusals in the independent parser. The test that stops a lazy route adopting a table another route owns was asked, at the same commit, 174 times in the extractor and 3512 in the parser and answers yes in neither, so for those refusals the corpus is evidence about the reading the two tools share and not about the guards that bound it. The per-array root-table test is the exception: at that commit it was asked a few hundred times and answered yes eight times in each reader, and one of those answers is what mints alfresco's `/rules`. A rule the corpus exercises and a rule only a fixture exercises are different kinds of claim, and this chapter says which each is. The reverse also holds: a corpus of six applications is not a sample that makes an analysis sound, and five constructs it happens not to contain were enough to break two of the properties this thesis claims. One further case came with it: a table a feature library both hands to `provideRouter` and exports for lazy loading is mounted at the root *and* under the prefix that loads it, so a claim on its file may not cover both mounts. The root scan now mints a root-registered array whatever claims its file, and alfresco's `/rules` is such a case.

A fifth is the mirror of the first. It is the case in which a conservative refusal becomes an over-approximation. When the only site that loads a route table lies in a test specification, the analysis declines the claim and scans the table from the project root, so its routes are reported at the application's root address. The guard establishes that a site the walk cannot reach may not justify suppressing the root scan; it does not establish that the application mounts the table there. That makes it a further instance of the unclaimed top-level-`Routes` over-approximation declared above, and it occurs nowhere in this corpus.

A sixth is the named outlet. Its history in this work shows the difference between declaring an approximation and being sound over it. A route carrying `outlet: 'name'` is reached as `(name:path)`, so it contributes no screen on the primary outlet. The extractor refused such a route from the start, but only the route that carries the property. Where a named outlet opened a `children` subtree, the routes beneath it were still assembled against the primary path and minted there. alfresco mounts its document viewer that way, and the graph held `/view/:nodeId` and `/search-libraries/view/:nodeId`, two addresses that application serves only through the viewer outlet. The independent route parser read the subtree the same

way, so coverage was 100 % over a set holding both, and no comparison between the two tools could show the gap. This was written down as an open case, on the reasoning that an approximation stated is an approximation a consumer can price.

That reasoning was wrong, as the runtime check of Section A.5 showed. A consumer reading the graph finds a node at `/view/:nodeId`; a navigation to that address in the built application is answered by the catch-all route. The declaration did not make the node true, and the must-tier property admits no exception on the strength of a written caveat. An outlet is inherited, so nothing below a named-outlet route has a primary address either: the extractor now carries outlet context down the walk and mints no screen for such a subtree. What remains is a limitation on what the graph contains: a screen reached only through a named outlet is not an element of the node set, and the transitions into it are not in the edge set. That is a statement about recall, and it is the kind of approximation a declaration can legitimately cover.

One attribution of a blind spot to the component it hides is a filename heuristic and is labelled as one. Where a route prefix the analysis could not follow has literal segments, a component whose path carries all of them as whole directory names is reported as hidden behind that prefix. The heuristic answers the question a reader asks (which screens are missing because of this hole), and it is wrong in two directions that the harness now states rather than hides. A route prefix does not determine a directory name, so the components behind a hole whose directory is named differently are not attributed to it and are reported only as having no node; and a prefix made entirely of parameters has no literal segment, so it is attributed to no component at all. An earlier version compared the prefix to the path as a bare substring, which read `/cluster` into `resourcelist/clusterrole/component.ts` and would have replaced the extractor's own true note about that file with a tool blind spot that does not cover it. No count of the rows that satisfied the broader predicate is given, because the figure depends on which population it is taken over and nothing in this thesis rests on it. What does matter is why the attribution reaches the published reason table in neither form. The obvious explanation, that some earlier test answers those rows first, is wrong. The attribution is asked of all 21 unaccounted rows in the corpus and declines every one of them on its own first two conditions: 10 because the owning file is not a component, and the rest because the graph already witnesses an edge from that file. The comparison this paragraph is about is never reached on this corpus, so what is recorded here is a property of the rule and not of any figure in Chapter 5. The narrower rule is still the one to state, because the broader one was false wherever it was asked.

A.7 The round's must tier, construct by construct

Table A.3 lists every construct the 20 must edges of the round-two instrument cite, with the edge that cites it and the condition the current extractor reads at it (Section 5.5). A construct appears once for each edge that cites it. The condition is read by the extractor's own readers, so the table records the extractor's decisions and a reader checks it by opening each construct at the line given.

Table A.3: The constructs behind the round's must tier and the condition the extractor reads at each. *none* means the construct runs whenever its template or handler does.

Application	Edge	Construct	Condition read
realworld	`/` → `/`	`src/app/features/article/pages/home/home.component.html:23`	structural directive
realworld	`/` → `/`	`src/app/features/article/pages/home/home.component.html:35`	none
realworld	`/` → `/`	`src/app/features/article/components/article-list.component.ts:36`	control-flow block
realworld	`/tag/:tag` → `/`	`src/app/features/article/pages/home/home.component.html:23`	structural directive
realworld	`/tag/:tag` → `/`	`src/app/features/article/pages/home/home.component.html:35`	none
realworld	`/tag/:tag` → `/`	`src/app/features/article/components/article-list.component.ts:36`	control-flow block
realworld	`/login` → `/`	`src/app/core/auth/auth.component.ts:76`	in a callback
realworld	`/register` → `/`	`src/app/core/auth/auth.component.ts:76`	in a callback
realworld	`/editor` → `/`	`src/app/features/article/pages/editor/editor.component.ts:52`	under a branch
realworld	`/editor/:slug` → `/`	`src/app/features/article/pages/editor/editor.component.ts:52`	under a branch
realworld	`/article/:slug` → `/`	`src/app/features/article/pages/article/article.component.ts:117`	in a callback
realworld	`/profile/:username/favorites` → `/`	`src/app/features/article/components/article-list.component.ts:36`	control-flow block
realworld	`n_scope_shell` → `/`	`src/app/core/layout/footer.component.html:3`	none
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:3`	none
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:10`	control-flow block
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:27`	control-flow block
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:59`	control-flow block
realworld	`n_scope_shell` → `/`	`src/app/core/layout/header.component.html:88`	control-flow block
bitwarden	`/verify-email` → `/`	`src/app/auth/verify-email-token.component.ts:53`	superseded by a later navigation
bitwarden	`/verify-email` → `/`	`src/app/auth/verify-email-token.component.ts:66`	after await
bitwarden	`/verify-recover-delete-org` → `/`	`src/app/admin-console/organizations/manage/verify-recover-delete-org.component.ts:43`	under a branch
bitwarden	`/verify-recover-delete-org` → `/`	`src/app/admin-console/organizations/manage/verify-recover-delete-org.component.ts:58`	after early exit
bitwarden	`/recover-delete` → `/`	`src/app/auth/recover-delete.component.ts:66`	after early exit
bitwarden	`/verify-recover-delete` → `/`	`src/app/auth/verify-recover-delete.component.ts:58`	under a branch
bitwarden	`/verify-recover-delete` → `/`	`src/app/auth/verify-recover-delete.component.ts:71`	after await
bitwarden	`/request-sm-access` → `/`	`src/app/secrets-manager/secrets-manager-landing/request-sm-access.component.ts:76`	after early exit

Application	Edge	Construct	Condition read
bitwarden	/settings/sponsored-families → /	src/app/billing/settings/sponsored-families.component.ts:194	under a branch
bitwarden	/setup/families-for-enterprise → /	src/app/admin-console/organizations/sponsorships/families-for-enterprise-setup.component.ts:86	superseded by a later navigation
bitwarden	/setup/families-for-enterprise → /	src/app/admin-console/organizations/sponsorships/families-for-enterprise-setup.component.ts:112	under a branch
bitwarden	/setup/families-for-enterprise → /	src/app/admin-console/organizations/sponsorships/families-for-enterprise-setup.component.ts:187	after await
bitwarden	/setup/families-for-enterprise → /	src/app/admin-console/organizations/sponsorships/families-for-enterprise-setup.component.ts:214	under a branch
bitwarden	/organizations/:organizationId/settings/account → /	src/app/admin-console/organizations/settings/account.component.ts:260	under a branch
bitwarden	/confirm-key-connector-domain → /	../../libs/key-management-ui/src/key-connector/confirm-key-connector-domain.component.ts:116	after await
bitwarden	/settings/security/password → /	../../libs/angular/src/auth/password-management/change-password/change-password.component.ts:193	under a branch
mifos	/callback → /login	src/app/zitadel/callback/callback.component.ts:39	under a branch
mifos	/callback → /login	src/app/zitadel/callback/callback.component.ts:47	in a catch clause

Bibliography

[1] Manuel Alabor and Markus Stolze. Debugging of RxJS-based applications. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Reactive and Event-Based Languages and Systems (REBLS)*, pages 15–24. ACM, 2020.

[2] Lars Ole Andersen. *Program Analysis and Specialization for the C Programming Language*. PhD thesis, DIKU, University of Copenhagen, 1994. DIKU report 94/19.

[3] Anastasios Antoniadis, Nikos Filippakis, Paddy Krishnan, Raghavendra Ramesh, Nicholas Allen, and Yannis Smaragdakis. Static analysis of Java enterprise applications: Frameworks and caches, the elephants in the room. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 794–807, 2020.

[4] Robert S. Arnold and Shawn A. Bohner. Impact analysis – towards a framework for comparison. In *Proceedings of the Conference on Software Maintenance (CSM)*, pages 292–301. IEEE Computer Society Press, 1993.

[5] David F. Bacon and Peter F. Sweeney. Fast static analysis of C++ virtual function calls. In *Proceedings of the 11th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 324–341, 1996.

[6] Engineer Bainomugisha, Andoni Lombide Carreton, Tom Van Cutsem, Stijn Mostinckx, and Wolfgang De Meuter. A survey on reactive programming. *ACM Computing Surveys*, 45(4):52:1–52:34, 2013.

[7] Herman Banken, Erik Meijer, and Georgios Gousios. Debugging data flows in reactive programs. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 752–763. ACM, 2018.

[8] Ira D. Baxter, A. Yahin, L. Moura, M. Sant'Anna, and L. Bier. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance (ICSM)*, pages 368–377. IEEE Computer Society, 1998.

[9] Carlo Bellettini, Alessandro Marchetto, and Andrea Trentini. WebUml: Reverse engineering of web applications. In *Proceedings of the 2004 ACM Symposium on Applied Computing (SAC)*, pages 1662–1669, 2004.

[10] Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding TypeScript. In Richard Jones, editor, *ECOOP 2014 – Object-Oriented Programming*, volume 8586 of *Lecture Notes in Computer Science*, pages 257–281, Berlin, Heidelberg, 2014. Springer.

[11] Justus Bogner and Manuel Merkel. To type or not to type? A systematic comparison of the software quality of JavaScript and TypeScript applications on GitHub. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 658–669. ACM, 2022.

[12] Shawn A. Bohner and Robert S. Arnold, editors. *Software Change Impact Analysis*. IEEE Computer Society Press, Los Alamitos, CA, USA, 1996.

[13] Miao Chen, Tengfei Tu, Hua Zhang, Qiaoyan Wen, and Weihang Wang. Jasmine: A static analysis framework for Spring core technologies. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1–13. ACM, 2022.

[14] Sen Chen, Lingling Fan, Chunyang Chen, Ting Su, Wenhe Li, Yang Liu, and Lihua Xu. StoryDroid: Automated generation of storyboard for Android apps. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 596–607, 2019.

[15] Gregory H. Cooper and Shriram Krishnamurthi. Embedding dynamic dataflow in a call-by-value language. In Peter Sestoft, editor, *Programming Languages and Systems, 15th European Symposium on Programming (ESOP 2006)*, volume 3924 of *Lecture Notes in Computer Science*, pages 294–308, Berlin, Heidelberg, 2006. Springer.

[16] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[17] Jeffrey Dean, David Grove, and Craig Chambers. Optimization of object-oriented programs using static class hierarchy analysis. In Mario Tokoro and Remo Pareschi, editors, *ECOOP'95 – Object-Oriented Programming*, volume 952 of *Lecture Notes in Computer Science*, pages 77–101. Springer, 1995.

[18] Guillaume Demarty, Fabien Maronnaud, Gabriel Le Breton, and Sylvain Hallé. SiteHopper: Abstracting navigation state machines for the efficient verification of web applications. In *Web Services and Formal Methods (WS-FM 2012)*, volume 7843 of *Lecture Notes in Computer Science*, pages 103–117. Springer, 2013.

[19] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[20] Joscha Drechsler, Guido Salvaneschi, Ragnar Mogk, and Mira Mezini. Distributed REScala: An update algorithm for distributed reactive programming. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 361–376. ACM, 2014.

[21] Conal Elliott and Paul Hudak. Functional reactive animation. In *Proceedings of the 2nd ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 263–273, 1997.

[22] Asger Feldthaus and Anders Møller. Checking correctness of TypeScript interfaces for JavaScript libraries. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 1–16. ACM, 2014.

[23] Asger Feldthaus, Max Schäfer, Manu Sridharan, Julian Dolby, and Frank Tip. Efficient construction of approximate call graphs for JavaScript IDE services. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pages 752–761, 2013.

[24] Martin Fowler. Inversion of control containers and the dependency injection pattern. `https://martinfowler.com/articles/injection.html`, January 2004.

[25] Zheng Gao, Christian Bird, and Earl T. Barr. To type or not to type: Quantifying detectable bugs in JavaScript. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 758–769. IEEE, 2017.

[26] Google. Angular documentation: Effects. `https://angular.dev/guide/signals/effect`, 2026. Accessed November 2026.

[27] Google. Angular documentation: Routing. `https://angular.dev/guide/routing`, 2026. Accessed September 2026.

[28] David Grove and Craig Chambers. A framework for call graph construction algorithms. *ACM Transactions on Programming Languages and Systems*, 23(6):685–746, 2001.

[29] May Haydar, Alexandre Petrenko, and Houari Sahraoui. Formal verification of web applications modeled by communicating automata. In *Formal Techniques for Networked and Distributed Systems (FORTE)*, volume 3235 of *Lecture Notes in Computer Science*, pages 115–132, 2004.

[30] Simon Holm Jensen, Anders Møller, and Peter Thiemann. Type analysis for JavaScript. In Jens Palsberg and Zhendong Su, editors, *Static Analysis, 16th International Symposium, SAS 2009*, volume 5673 of *Lecture Notes in Computer Science*, pages 238–255, Berlin, Heidelberg, 2009. Springer.

[31] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[32] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[33] Ralph E. Johnson and Brian Foote. Designing reusable classes. *Journal of Object-Oriented Programming*, 1(2):22–35, June/July 1988.

[34] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665. ACM, 2014.

[35] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, pages 92–101, 2014.

[36] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, 21(5):2035–2071, 2016.

[37] Iva Kertusha, Gebremariem Assress, Onur Duman, and Andrea Arcuri. A survey on web testing: On the rise of AI and applications in industry, 2025. arXiv:2503.05378.

[38] Gary A. Kildall. A unified approach to global program optimization. In *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 194–206, 1973.

[39] Łukasz Kurant. Static components dependency graph detection with evaluation metrics in React.js projects. In *Communication Papers of the 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*, volume 45 of *Annals of Computer Science and Information Systems*, pages 127–134, 2025.

[40] Duling Lai and Julia Rubin. Goal-driven exploration for Android applications. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 115–127, 2019.

[41] Rodrigo Laigner, Diogo Mendonça, Alessandro Garcia, and Marcos Kalinowski. Cataloging dependency injection anti-patterns in software systems. *Journal of Systems and Software*, 184:111125, 2022.

[42] Davy Landman, Alexander Serebrenik, and Jurgen J. Vinju. Challenges for static analysis of Java reflection – literature review and empirical study. In *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 507–518. IEEE, 2017.

[43] Kim G. Larsen and Bent Thomsen. A modal process logic. In *Proceedings of the Third Annual Symposium on Logic in Computer Science (LICS)*, pages 203–210, 1988.

[44] Steffen Lehnert. A taxonomy for software change impact analysis. In *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWPSE-EVOL)*, pages 41–50. ACM, 2011.

[45] Jiawei Li, Jiahao Liu, Jian Mao, Jun Zeng, and Zhenkai Liang. UI-CTX: Understanding UI behaviors with code contexts for mobile applications. In *Network and Distributed System Security Symposium (NDSS)*, 2025.

[46] Changlin Liu, Hanlin Wang, Tianming Liu, Diandian Gu, Yun Ma, Haoyu Wang, and Xusheng Xiao. ProMal: Precise window transition graphs for Android via synergy of program analysis and machine learning. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 1755–1767, 2022.

[47] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1–13. ACM, 2024.

[48] Benjamin Livshits, Manu Sridharan, Yannis Smaragdakis, Ondřej Lhoták, J. Nelson Amaral, Bor-Yuh Evan Chang, Samuel Z. Guyer, Uday P. Khedker, Anders Møller, and Dimitrios Vardoulakis. In defense of soundiness: A manifesto. *Communications of the ACM*, 58(2), 2015.

[49] Magnus Madsen, Benjamin Livshits, and Michael Fanning. Practical static analysis of JavaScript applications in the presence of frameworks and libraries. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 499–509. ACM, 2013.

[50] Magnus Madsen, Frank Tip, and Ondřej Lhoták. Static analysis of event-driven Node.js JavaScript applications. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 505–519. ACM, 2015.

[51] Ali Mesbah, Arie van Deursen, and Stefan Lenselink. Crawling Ajax-based web applications through dynamic analysis of user interface state changes. *ACM Transactions on the Web*, 6(1):1–30, 2012.

[52] Amin Milani Fard and Ali Mesbah. Feedback-directed exploration of web applications to derive test models. In *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, pages 278–287, 2013.

[53] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Efficient JavaScript mutation testing. In *Proceedings of the 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation (ICST)*, pages 74–83. IEEE, 2013.

[54] Shabnam Mirshokraie, Ali Mesbah, and Karthik Pattabiraman. Guided mutation testing for JavaScript web applications. *IEEE Transactions on Software Engineering*, 41(5):429–444, 2015.

[55] Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, Berlin, Heidelberg, 1999.

[56] Frolin S. Ocariza, Karthik Pattabiraman, and Ali Mesbah. Detecting inconsistencies in JavaScript MVC applications. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 325–335. IEEE, 2015.

[57] Naito Oshima and Tomoji Kishi. Model checking method for SPA page transition based on component-based framework. In *Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE 2018)*. KSI Research, 2018.

[58] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier, 2019.

[59] Renaud Pawlak, Martin Monperrus, Nicolas Petitprez, Carlos Noguera, and Lionel Seinturier. SPOON: A library for implementing analyses and transformations of Java source code. *Software: Practice and Experience*, 46(9):1155–1179, 2016.

[60] Upsorn Praphamontripong and Jeff Offutt. Applying mutation testing to web applications. In *Proceedings of the 2010 Third International Conference on Software Testing, Verification, and Validation Workshops (ICSTW)*, pages 132–141. IEEE, 2010.

[61] Remix Run. React router documentation, version 8. `https://reactrouter.com/`, 2026. Version 8.4.0. Accessed 5 October 2026.

[62] Xiaoxia Ren, Fenil Shah, Frank Tip, Barbara G. Ryder, and Ophelia Chesley. Chianti: A tool for change impact analysis of Java programs. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 432–448. ACM, 2004.

[63] Filippo Ricca and Paolo Tonella. Analysis and testing of web applications. In *Proceedings of the 23rd International Conference on Software Engineering (ICSE)*, pages 25–34, 2001.

[64] Gregor Richards, Francesco Zappa Nardelli, and Jan Vitek. Concrete types for TypeScript. In John Tang Boyland, editor, *29th European Conference on Object-Oriented Programming (ECOOP 2015)*, volume 37 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 76–100, Dagstuhl, Germany, 2015. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[65] Bachar Rima, Anas Shatnawi, and Abdelhak-Djamel Seriai. E2E test generation over a source-code-derived workflow space: An Angular case study. In *Software Engineering and Advanced Applications*, Lecture Notes in Computer Science, pages 334–351, Cham, 2026. Springer Nature Switzerland. Published online 27 August 2026; print volume dated 2027. The method is described in this thesis from the publisher abstract and the replication package; the full text was not available.

[66] Guido Salvaneschi and Mira Mezini. Debugging for reactive programming. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, pages 796–807. ACM, 2016.

[67] Guido Salvaneschi, Sebastian Proksch, Sven Amann, Sarah Nadi, and Mira Mezini. On the positive effect of reactive programming on software comprehension: An empirical study. *IEEE Transactions on Software Engineering*, 43(12):1125–1143, 2017.

[68] Manu Sridharan, Shay Artzi, Marco Pistoia, Salvatore Guarnieri, Omer Tripp, and Ryan Berg. F4F: Taint analysis of framework-based web applications. In *Proceedings of the 2011 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 1053–1068. ACM, 2011.

[69] Stryker Mutator contributors. Stryker mutator: Mutation testing for JavaScript, TypeScript, C# and Scala. `https://stryker-mutator.io/`, 2026. Accessed 2026-10-21.

[70] Oksana Tkachuk and Sreeranga P. Rajan. Method and system to extract a navigation model for analysis of a web application, 2013. US Patent 8,402,319 B2, Fujitsu Limited.

[71] Chenxu Wang, Tianming Liu, Yanjie Zhao, Minghui Yang, and Haoyu Wang. LLMDroid: Enhancing automated mobile app GUI testing coverage with large language model guidance. In *Proceedings of the ACM on Software Engineering (FSE)*, 2025.

[72] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering*, 50(4):911–936, 2024.

[73] Siyi Wang, Sinan Wang, Yujia Fan, Xiaolei Li, and Yepang Liu. Leveraging large vision-language model for better automatic web GUI testing. In *Proceedings of the 2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 125–137. IEEE, 2024.

[74] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. AutoDroid: LLM-powered task automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 543–557. ACM, 2024.

[75] Rahulkrishna Yandrapally and Ali Mesbah. Fragment-based test generation for web apps. *IEEE Transactions on Software Engineering*, 49(3):1086–1101, 2023.

[76] Hong Yul Yang, Ewan Tempero, and Hayden Melton. An empirical study into use of dependency injection in Java. In *Proceedings of the 19th Australian Conference on Software Engineering (ASWEC)*, pages 239–247. IEEE, 2008.

[77] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024.

[78] Shengqian Yang, Haowei Wu, Hailong Zhang, Yan Wang, Chandrasekar Swaminathan, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. *Automated Software Engineering*, 25(4):833–873, 2018.

[79] Shengqian Yang, Dacong Yan, Haowei Wu, Yan Wang, and Atanas Rountev. Static control-flow analysis of user-driven callbacks in Android applications. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, pages 89–99, 2015.

[80] Shengqian Yang, Hailong Zhang, Haowei Wu, Yan Wang, Dacong Yan, and Atanas Rountev. Static window transition graphs for Android. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 658–668, 2015.

[81] Evan You and Eduardo San Martin Morote. Vue router documentation, version 5. `https://router.vuejs.org/`, 2026. Version 5.3.1. Accessed 5 October 2026.

[82] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, Singapore, 2023. Association for Computational Linguistics.

[83] Tao Zhang, Fuzhong Hao, Yunfan Wang, Bo Zhang, and Guangwei Xie. VeriFlow: A framework for the static verification of web application access control via policy-graph consistency. *Electronics*, 14(18):3742, 2025.

[84] Carlos Zimmerle, Kiev Gama, Fernando Castor, and José Murilo Mota Filho. Mining the usage of reactive programming APIs: A study on GitHub and Stack Overflow. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*, pages 203–214. ACM, 2022.